Title: A 32-bit RISC-V ASIC implementation

A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT
FOR THE DEGREE OF

Master of Technology

IN THE
FACULTY OF ENGINEERING

BY

SANIDHYA SAXENA, TOMS JIJI VARGHESE

GUIDED BY

KURUVILLA VARGHESE

DEPARTMENT OF ELECTRONIC SYSTEMS ENGINEERING
INDIAN INSTITUTE OF SCIENCE, BANGALORE

MAY 2025

Synopsis

This project focuses on the design and development of a synthesizable 32-bit RISC-V processor core, adhering to the RISC-V Unprivileged ISA Specification (RV32I/M/A/F) as well as the Machine Mode Privilege Specification. The processor is developed with a clean and modular architecture suitable for ASIC integration and optimized for use in embedded systems and real-time applications, where low-latency response and deterministic execution are critical.

What sets this core apart from other RISC-V implementations is the inclusion of Physical Memory Protection (PMP) support, allowing secure access control to different memory regions—a key requirement for embedded and safety-critical systems. In addition, a fully functional Debug Module has been implemented, compliant with the RISC-V Debug Specification. This includes support for single-step execution and register/memory access during halt, and a JTAG interface for external debugger connectivity such as GDB.

With its balance of standards compliance, functional completeness, and low-overhead integration, this 32-bit RISC-V processor core serves as a robust and flexible foundation for a wide range of research and industrial applications in embedded system design.

Acknowledgements

We express our deep gratitude towards our guide, **Professor Kuruvilla Varghese**, for his constant encouragement, valuable suggestions and guidance throughout the project. We thank our reviewers, **Professor Chetan Singh Thakur** and **Professor Hardik J. Pandya**, for their time to time suggestions, ideas, and valuable inputs, which helped us rectify mistakes and explore new directions.

Contents

List of Figures

List of Tables

Nomenclature

BPU Branch Prediction Unit

BTB Branch Target Buffer

CLINT Core Local Interrupt controller

CSRs Control and Status Registers

DMI Debug Module Interface

DTLB Data Translation Lookaside Buffer

DTM Debug Transport Module

ECALL Enviroment Call

EX Execute Unit

GLS Gate Level Simulations

ID Instruction Decode unit

IF Instruction Fetch unit

ITLB Instruction Translation Lookaside Buffer

JTAG Joint Test Action Group

mcause Machine Cause exception register

MEM Memory Unit

mepc Machine Exception program counter

MMU Memory Management Unit

MRET Machine Return Instruction

mstatus Machine status register

mtval Machine Trap value register

mtvec Machine trap vector address

PHT Page History Table

PMP Physical Memory Protection

PPN Physical Page Number

RAS Return Address Stack

RTOS Real time operating System

TLB Translation Lookaside Buffer

VPN Virtual Page Number

WB Write Back

AIM:

The primary aim of this project is to design and develop a 32-bit RISC-V processor core targeted for ASIC implementation, incorporating essential architectural features required for modern embedded and low-power applications. The processor will support advanced functionalities such as Memory Protection and Management (PMP/MMU), Control and Status Registers (CSRs), Interrupt and Exception Handling, and Debug Support compliant with the RISC-V privilege and debug specifications. The goal is to build a standards-compliant, synthesizable core that can be used as a soft IP for academic research or industrial integration into SoCs, with a focus on scalability, verification, and ASIC readiness.

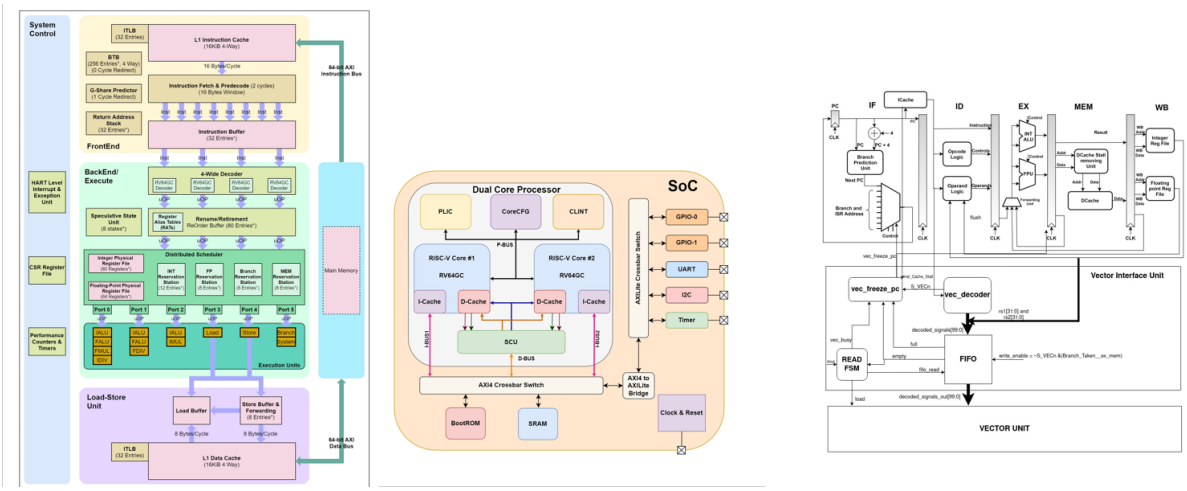

Figure 1: Available designs at RC Lab

Some of the RISC-V architecture based processor and SOC developed by the Reconfigurable Computing lab are 64-bit 4-wide Out-of-Order processor core, SOC based on Dual-core processor, 32-bit RISC-V prcoessor with additions like Vector unit. These architectures were made for FPGA and need to be re-designed for ASIC. We will be designing a 32-bit RISC-V processor from ASIC development point of view, and various parameterisation like Memory Size, Tunable cache parameters will be added to the design. These blocks can be re-used as an IP in future projects for RC Lab, as well as can be integrated in the previous designs for hardware configuarbility. These blocks will be individually synthesized and the final placement and routing will be done for the complete design.

Chapter 1

Pre-study

This report details the pre-study conducted to understand the design and architecture of a RISC-V processor and its implementation using the Chisel hardware construction language. We embarked on a comprehensive exploration through:

- **Market Survey:** We investigated existing RISC-V-based controllers to identify industry trends, potential application areas, and current performance benchmarks.
- **Product Survey:** We analyzed commercially available controllers (if applicable) to understand their functionalities, resource utilization, and potential limitations.
- **Wish List Specification:** Based on the market and product surveys, we formulated a detailed wish list outlining the desired features and capabilities of our RISC-V controller.

1.1 Market Survey

RISC-V (RISC Five), an open-source instruction set architecture (ISA) is rapidly transforming the computing landscape. Unlike traditional proprietary ISAs, RISC-V offers a royalty-free and modular standard. This fosters a wave of innovation across diverse computing domains, including the Internet of Things (IoT), edge computing, and Artificial Intelligence (AI). RISC-V's low-power consumption and inherent scalability make it ideal for resource-constrained IoT devices. Additionally, its flexible nature allows for customization to specific processing needs encountered at the edge of computing networks, and its modularity facilitates the development of specialized hardware accelerators for AI applications.

1.1.1 Market Trends

The pre-study revealed a burgeoning market for RISC-V processors, driven by several key trends:

- **Soaring Adoption**: Across diverse sectors, the open nature of RISC-V is fueling rapid adoption. This openness allows for customization, reduces time to market, and provides cost savings for developers.
- **Application Expansion**: Initially popular in embedded systems, RISC-V is now making inroads into high-performance computing domains such as servers, networking equipment, and even some mobile devices. This diversification showcases its growing versatility.
- **Thriving Ecosystem**: The RISC-V ecosystem is flourishing with the development of essential tools like software tools, compilers, operating systems, and hardware platforms. This robust ecosystem fosters innovation and further adoption.
- **Industry Backing**: Major tech giants, including Google, NVIDIA, Western Digital, and Alibaba, are lending their support to RISC-V. This industry backing signifies its potential to become a mainstream architecture in the future.

1.1.2 Growing Demand for Customization:

RISC-V's open architecture is a key driver behind its growing adoption. This openness unlocks a new level of customization for companies developing Application-Specific Integrated Circuits (ASICs). Beyond the traditional benefits of ASICs, RISC-V empowers a more granular level of control, allowing for:

- **Tailored Solutions**: Traditional ASICs offer customization, but RISC-V's open architecture unlocks a new level. Companies can fine-tune their ASIC designs, optimizing performance, power efficiency, and other critical metrics to their exact application requirements.
- **Industry-Specific Optimization**: Different industries have unique needs. RISC-V empowers companies to develop bespoke solutions that meet the stringent demands of sectors like IoT, automotive, aerospace, and telecommunications. By incorporating RISC-V

cores tailored to their specific needs, companies can achieve significant performance and efficiency gains.

1.1.3 Booming Ecosystem Supports RISC-V ASIC Design

RISC-V's open architecture fosters hardware customization and the development of a comprehensive ecosystem. This ecosystem plays a critical role in facilitating the broad adoption of RISC-V within ASIC design by providing essential tools and resources that streamline the development process. An examination of the following elements reveals the key components of this maturing ecosystem:

- **Comprehensive Development Tools**: A growing suite of development tools caters specifically to RISC-V-based ASICs. This includes tools for Register Transfer Level (RTL) design, verification, synthesis, and physical design, ensuring a smooth workflow for ASIC designers.
- **Diverse IP Core Offerings**: Companies specializing in RISC-V IP cores are providing a rich set of options for ASIC integration. These offerings encompass various configurations of RISC-V cores, peripheral IP (Intellectual Property) blocks, and interconnect IP. This comprehensive library empowers ASIC designers to efficiently build complex systems-on-chip (SoCs) tailored to their specific needs.

1.1.4 Prominent Players in the Maturing RISC-V Ecosystem

The pre-study identified several prominent players shaping the maturing RISC-V ecosystem:

- **SiFive**: A leading provider of RISC-V IP cores, SiFive offers a comprehensive portfolio of customizable processor designs catering to diverse application needs.
- **Andes Technology**: This company specializes in developing high-performance, low-power RISC-V processor cores for a wide range of applications, including those in the Internet of Things (IoT), automotive, and Artificial Intelligence (AI) sectors.
- **NVIDIA**: Following its acquisition of Arm, NVIDIA's growing interest in RISC-V signifies its potential as a viable open-source architecture.

- **Microchip Technology**: Microchip contributes to the RISC-V ecosystem by offering microcontrollers based on the RISC-V architecture, targeting applications in the IoT and embedded systems domains.

1.1.5 Acceleration of Prototyping and Time-to-Market:

RISC-V's open architecture offers significant advantages in terms of development time and prototyping for ASIC design:

- **Streamlined Prototyping**: The open nature of RISC-V allows designers to experiment with various configurations and architectural choices readily. This flexibility fosters faster prototyping cycles, enabling rapid design exploration and optimization. As a result, companies can iterate on their designs quickly, accelerating the product development process.
- **Agile Development Compatibility**: RISC-V aligns well with agile development methodologies. Its open nature facilitates quick modifications and easier integration of feedback from stakeholders during the design phases. This iterative approach reduces the time needed to refine the design and ultimately shortens the time-to-market for ASIC products.

1.1.6 Cost-Effectiveness and Scalability with RISC-V

RISC-V presents compelling advantages for ASIC design in terms of both cost and scalability:

- **Reduced Development Costs**: Unlike proprietary architectures that require licensing fees, RISC-V's open-source nature eliminates these upfront costs, making it a more cost-effective option for developers.
- **Scalable Architecture**: RISC-V offers remarkable scalability. The same core architecture can be adapted to create a wide range of solutions, from resource-constrained, low-power designs ideal for IoT devices to high-performance processors suitable for data centers. This scalability allows ASIC designers to leverage a familiar architecture while tailoring it to meet the specific needs of their application without significant architectural overhauls.

1.1.7 Challenges and Considerations for RISC-V Adoption

While RISC-V presents enticing opportunities, it's crucial to acknowledge the challenges associated with its adoption in ASIC design:

- **Established Market Competitors**: The dominance of entrenched architectures like x86 and Arm in specific markets presents a significant hurdle for RISC-V's widespread adoption. Overcoming these established players and their mature ecosystems requires ongoing innovation and industry support for RISC-V.
- **Intellectual Property Considerations**: Although RISC-V boasts an open-source core, challenges related to intellectual property can still arise when integrating RISC-V cores into commercial products. Careful assessment and management of IP licensing terms associated with specific RISC-V implementations are necessary.
- **Verification and Validation Complexity**: Verifying complex ASIC designs remains a significant challenge, regardless of the chosen ISA. Integrating RISC-V processors necessitates robust verification strategies to ensure the design's correctness and reliability. Designers need to invest in thorough verification methodologies for RISC-V-based ASICs.
- **Evolving Ecosystem**: The RISC-V ecosystem, while rapidly maturing, lacks complete maturity in certain areas. Aspects like optimized libraries, established design flows, and comprehensive tool support are still under development. ASIC designers must carefully evaluate the maturity of the RISC-V ecosystem in relation to their specific needs and project timelines.

1.2 Comparative Study

In this comparative study of RISC-V microcontroller cores, we observe a diverse design space optimizing for different trade-offs between power, performance, and area. The **VexRiscv core**[6] offers exceptional area efficiency (˜1.4k LUTs) and ultra-low power consumption, making it ideal for FPGA-based designs. Similarly, the **NEORV32**[7] is optimized for low-power embedded systems with modest performance (0.28–0.77 CoreMark/MHz) and a configurable area footprint (3.5k–5k LUTs). The **Codasip L110** [8] strikes a balance between code density, performance, and area, achieving 20% smaller code sizes and improved perf/Watt compared

to similar-class cores. Meanwhile, **SweRV EH1** [9] introduces a dual-issue, 9-stage pipeline that delivers competitive performance (˜4.9 CoreMark/MHz) with moderate area usage (˜10–20k LUTs), suiting high-throughput industrial workloads. On the higher end, **SiFive U84** [10] target application-class performance. The U84 boosts IPC by 2.3× over its predecessor, while the P670 delivers >12 CoreMark/MHz at up to 3.4 GHz, yet remains 50% smaller than ARM Cortex-A78 in area. Most notably, **Micro Magic's RISC-V core** [11] achieves an unprecedented 13,000 CoreMark at just 200 mW and 5 GHz, representing a cutting-edge balance of speed and power. Collectively, these cores demonstrate the flexibility and scalability of RISC-V across embedded to high-performance domains.

1.3 Wish Specification

This project focuses on developing a RISC-V core specifically designed for embedded systems and custom hardware projects. The core prioritizes achieving a balanced trade-off between three critical factors:

• **Performance**: The core should deliver efficient processing capabilities.

• **Power Consumption**: Low power consumption is crucial for battery-powered applications.

• **Area Footprint**: Minimizing the core's physical size on a chip optimizes cost and resource utilization.

The RISC-V core will leverage a Harvard architecture to enable simultaneous instruction and data memory accesses. This approach enhances overall processing efficiency. Additionally, an optimizing folded 6-stage pipeline will be implemented. This pipeline maximizes overlap between execution stages and memory accesses, reducing stalls and improving performance.

Optional features include Branch Prediction, Instruction Cache, Data Cache, Debug Unit and optional Multiplier/Divider Units. Parameterized and configurable features include the instruction and data interfaces, the branch-prediction-unit configuration, and the cache size, associativity, replacement algorithms and multiplier latency. Providing the user with trade offs between performance, power, and area to optimize the core for the application.

1.3.1 Key Specifications:

1.3.1.1 High Performance 32-bit CPU design with Optional Extensions

- **Parameterized 32/64bit data**: The core will be a high-performance 32-bit CPU design with the option for parameterized 32/64-bit data handling.
- **Fast and Precise Interrupts**: The core prioritizes efficient interrupt handling, ensuring timely responses to critical events within the system.
- **5-Stage Pipeline**: This core implements an optimized folded 6-stage pipeline to maximize overlap between instruction execution and memory access stages. This approach minimizes stalls and improves overall processing efficiency.
- **Memory Protection Support**: The core will offer memory protection support for enhanced system security.
- **Optional/Parameterized Caches**: Instruction and data caches can be optionally integrated to reduce memory access latency for performance-critical applications. The size, associativity, and replacement algorithms of these caches will be configurable.

1.3.1.2 Area and Power Efficiency

The design will prioritizes both minimizing the core's physical footprint (area) and reducing its power consumption:

- **Parameterized Data Width**: The datapath width can be configured, allowing users to optimize the balance between performance and power consumption for their specific application. Wider data paths can process more data per cycle but require more hardware resources and consume more power.
- **Clock Gating Logic**: The core will employ clock gating logic to reduce dynamic power consumption. This technique essentially shuts off the clock signal to inactive portions of the core, minimizing unnecessary power usage.
- **Small Silicon Footprint**: The design prioritizes a compact layout to minimize the core's physical size on the chip. This not only reduces manufacturing costs but also allows for more efficient integration with other components on the same die.

1.3.1.3 Instruction Set Architecture (ISA)

The RISC-V core will support the base integer instruction set architecture (ISA) defined by the RISC-V specification. This core ISA provides a foundation for efficient execution of various workloads.

- **Basic Integer Instructions**: The core will support fundamental instructions for arithmetic, logical, data transfer, and control flow operations. This includes essential instructions like branch, jump, and call instructions for program branching and subroutine management.
- **Optional Branch Prediction**: A branch prediction algorithm can be optionally integrated to improve the efficiency of branch instructions. Branch prediction attempts to forecast the outcome of a conditional branch instruction, potentially reducing pipeline stalls and improving overall performance.
- **Custom ISA Extension Support (Optional)**: The design may offer the flexibility to incorporate user-defined RISC-V instruction extensions. These extensions can be tailored to specific application requirements, potentially accelerating performance for specialized tasks. However, the decision to include custom extensions requires careful consideration of development complexity and potential compatibility limitations.

Chapter 2

Study

2.1 Processor Design

Processors are essential parts of any computer system, playing a central role in how instructions are processed and executed. Computer architecture refers to the design and structure that connects both hardware and software components to form a complete computing platform. It defines how the system fetches, decodes, and executes instructions. There are many different processor designs, each offering different trade-offs in terms of power, performance, and area (PPA), making them suitable for different types of applications. At the core of any processor design is the Instruction Set Architecture (ISA), which outlines the functions, data types, registers, and other elements visible to programmers. The ISA serves as the interface between hardware and software.

2.2 Instruction Set Architecture

The Instruction Set Architecture (ISA) defines the set of commands that a processor understands and executes. While multiple processors can implement the same ISA, their internal architectures can vary significantly. In this project, we are using the RISC-V ISA, which is emerging as a widely adopted, open, and standardized architecture suitable for both academic research and industrial development[12]. Our focus will be on a 32-bit RISC-V design, although we also have access to 64-bit versions.

RISC-V has gained popularity primarily because it is a free and open standard, enabling unrestricted innovation and development. Its modular structure is another key advantage—it is built

around a solid, fixed base, while optional extensions can be added as needed. This avoids the continual revision cycles seen in legacy ISAs like the x86 family.

A comparative analysis was conducted between RISC-V and other instruction sets such as MIPS, ARM, Intel, and general RISC architectures [13]. One of RISC-V's major strengths lies in its open-source nature, which supports a growing software ecosystem and allows the ISA to be used as a reusable IP block for a wide range of applications.

Despite the specific format used in the core we can identify four main subsets of instructions:

1. *Integer Arithmetic Instructions*: These include operations such as addition (ADD, ADDI), subtraction (SUB), shifts (SLL, SLLI, SRL, SRLI, SRA, SRAI), bitwise logic (AND, ANDI, OR, ORI, XOR, XORI), and comparisons (SLT, SLTI, SLTU, SLTIU). Register-to-register operations use the R-type format, while register-immediate operations use the I-type format.

2. *Control Flow Instructions*: These manage the execution flow by altering the program counter. Instructions like Jump and Link (JAL) and Jump and Link Register (JALR) support unconditional jumps and are encoded in UJ-type and I-type formats respectively. Conditional branches include BEQ, BNE, BLT, BLTU, BGE, and BGEU. Additional conditional operations such as BGT, BGTU, BLE, and BLEU can be achieved by swapping operands in the existing branch instructions.

3. *Load and Store Instructions*: As a load-store architecture, RISC-V restricts memory interactions to dedicated instructions. The STORE family (SW, SH, SB) writes data from registers to memory, with word (32-bit), half-word (16-bit), and byte (8-bit) support. The LOAD family (LW, LH, LHU, LB, LBU) reads data from memory into registers, with the "U" variants indicating zero-extension for smaller data types instead of sign-extension.

4. *Control and Status Register (CSR) Instructions*: These SYSTEM instructions, encoded in the I-type format, are used for managing the processor's internal state. CSRRW allows atomic read/write operations between CSRs and registers. CSRRS and CSRRC perform bitwise set and clear operations on CSRs. Immediate versions—CSRRWI, CSRRSI, and CSRRCI—function similarly but use an immediate value rather than a register operand.

2.2.1 Standard extensions

The most important feature of RISC-V is the possibility to easily add other custom instructions to enlarge the ISA and adapt it to specific objectives. Despite these "non-standard" extensions,

there are several other standard ISA extensions that have been developed during years and can be added very easily to our particular implementation making the final RISC-V a very powerful core, and they are:

- "M" Standard Extension for Integer Multiplication and Division;
- "A" Standard Extension for Atomic Instructions;
- "F" Standard Extension for Single-Precision Floating-Point;
- "D" Standard Extension for Double-Precision Floating-Point;
- "Q" Standard Extension for Quad-Precision Floating-Point;
- "L" Standard Extension for Decimal Floating-Point;
- "C" Standard Extension for Compressed Instructions;
- "B" Standard Extension for Bit Manipulation;
- "J" Standard Extension for Dynamically Translated Languages;
- "T" Standard Extension for Transactional Memory;
- "P" Standard Extension for Packed-SIMD Instructions;
- "V" Standard Extension for Vector Operations;
- "N" Standard Extension for User-Level Interrupts;

2.3 Single Cycle vs Pipeline design

Majority of the processor designs have two components

1. Datapath
2. Control Path

Single cycle processor is a processor in which the instructions are fetched from the memory, and then executed, and the results are further stored in a single cycle, whereas in pipelined implementation, each step in execution takes one clock cycle. All the instructions take different

time to complete depending upon their type, and in single cycle the next instruction cannot be fetched before the execution of previous instruction completes. Hence, the maximum clock frequency will be less and concourrent execution is not possible.

Pipelined processor saves time by overlapping instruction substages. Since, all RISC-V instructions are the same length, this makes it much easier to fetch instructions in the first pipeline stage and to decode them in the second stage. The more pipeline stages a processor has, the more instructions it can process at once and the less of a delay there is between completed instructions, but the overheads and stalling penalty will increase as the number of stages will increase. The below table 2.1 shows the comparision between the two.

	Single Cycle	**Pipelined Processor**
Hardware Required	Less	More (Due to Registers)
CPI	1	>=1 (Due to Stalls and Hazards)
T_{clk}	Baseline	Less (Due to pipelining)
Throughput	1	>1

Table 2.1: Comparision between Single cycle vs Pipelined

The proposed architecture will be five stage pipelined so as to achieve the benefit of maximum throughput and performance.

1. Instruction fetch
2. Intsrutcion Decode
3. Instruction Execute
4. Memory
5. Write Back

2.4 Other available designs

Numerous processors based on the RISC-V instruction set architecture (ISA) have already been developed, with many more currently in progress. RISC-V is an open-source ISA that fosters innovation through collaborative development. It provides a flexible framework for processor

	Rocket	**Shakti**	**Current Design**
Bits	32/64-bit	32/64-bit	32-bits
Stages	5	5	5
Extension	MAFD	IMAC	MAFD
Functional Units	4	3	5
Frequency	1.3Ghz(28nm,FD-SOI)	70Mhz (22nm,FinFet)	52Mhz(65nm)

Table 2.2: Application core comparitive study

design, supporting both 32-bit and 64-bit address spaces, along with a compressed ISA tailored for application-specific extensions [14].

One prominent example is Rocket [15] an open-source processor featuring a six-stage pipeline architecture based on the RV64G ISA. The Rocket SoC generator can produce multiple SoC design variants from a single base configuration. The Rocket core itself is a 64-bit, five-stage, single-issue, in-order scalar pipeline that fully supports the RISC-V ISA.

Another notable development is the Shakti series of processors, created by researchers at IIT Madras. These processors are designed with applications in space and nuclear research in mind. Variants such as Shakti-C, Shakti-T, Shakti-I, and Shakti-E reflect different computing core implementations tailored to specific use cases.

In parallel, the ARM architecture—although not based on RISC-V—shares a similar emphasis on simplicity and energy efficiency. ARM is built on a load-store architecture and uses a mix of fixed 32-bit and variable-length Thumb instructions. It also includes a large set of general-purpose registers. Advanced features such as hardware virtualization, superscalar pipelines, and out-of-order execution are integral to many ARM cores [16]. Table 2.2 shows the comparision of our design with other available RISC-V cores.

2.5 Previous RISC-V Architecture

Previous design of RISC-V Architecture is shown below in figure 2.1 with five stages of pipeline viz. Fetch, Decode, Execute, Memory and write back.

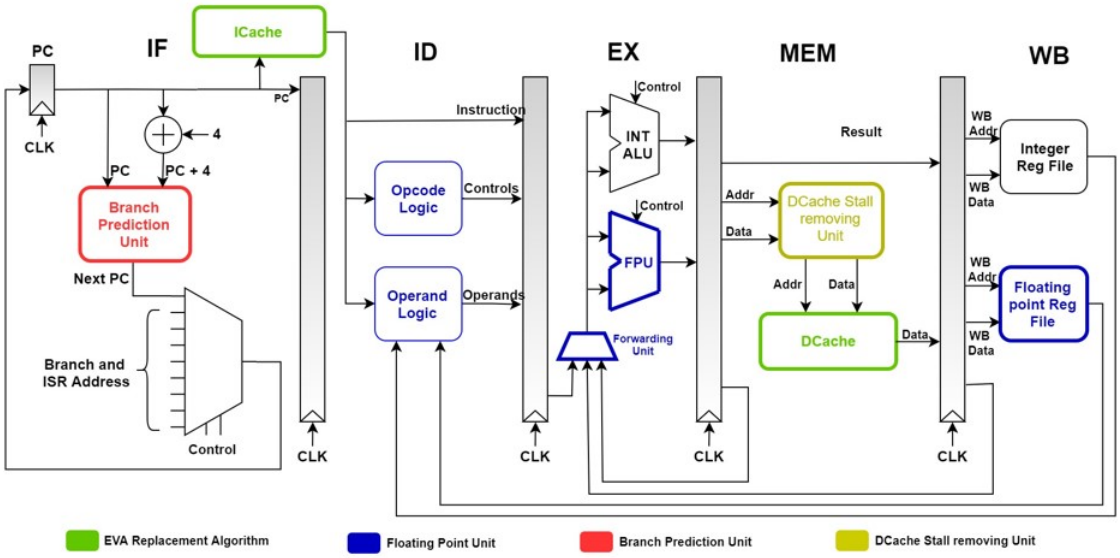

Figure 2.1: Previous Architecture

The baseline architecture, inherited from the previous batch, is a classic 5-stage pipelined RISC-V processor design consisting of the stages: *Instruction Fetch (IF)*, *Instruction Decode (ID)*, *Execute (EX)*, *Memory (MEM)*, and *Write Back (WB)*. Notably, the architecture integrates several performance-enhancing features. In the IF stage, it incorporates a *Branch Prediction Unit* to reduce branch misprediction penalties and a dedicated *Instruction Cache (ICache)* to accelerate instruction fetch latency. During decoding, control signals are generated by Opcode and Operand Logic, which interface with the Forwarding Unit in the EX stage to handle data hazards. The Execute stage features both an *Integer ALU and a Floating Point Unit (FPU)* for supporting mixed-type operations, reflecting floating-point support across the pipeline. The MEM stage includes a *Data Cache (DCache)* to reduce data memory access times, and a DCache Stall Removing Unit, which aims to improve memory pipeline throughput by mitigating stalling due to cache misses. Finally, the WB stage supports both integer and floating-point register files, highlighting the design's ability to execute and store results from a wide range of computational instructions. This architecture served as a strong foundation for further optimizations and extensions in the current work.

1. ***Instruction fetch*:** At the start of each cycle, the address is used to fetch instructions from instruction memory. Following that, a calculation is run to compute the next PC in the cycle. It is decided either to use that as as its next value or, instead, the outcome of the control transfer instruction based on branch prediction logic.

2. ***Instruction Decode***: The 32-bit instruction is parsed into its fields, and two registers

pertaining to the operands are fetched from the register file. Instruction mux, Instruction decoder, Immediate generator, Register file, and the write enable generator are the several blocks that make up the instruction decode stage.

3. ***Instruction Execute***: In the Execute step, the real computing takes place. An ALU and a bit shifter are frequently included in this phase. The bit shifter is in charge of shifts and rotation logic to be performed on the operands.

4. ***Memory Access***: At this point, the load/store instructions read and write memory operands from and to the memory.

5. ***Write-Back***: At this step, the instructions store their results to the respective register file. There will be a onset of data hazard since one of the input registers read in decode could be the same as the destination register written in writeback. The data forwarding technique is used to avoid this.

2.5.1 Hazard Detection and Correction block

Due to data and control dependencies, in pipeline designs we have 2 major hazards:

1. *Data Hazard*

2. *Control Hazard*

When an instruction in the pipeline is dependent on the outcomes of a preceding instruction that has not yet entered the write back stage, it leads to data dependence. Control dependency occurs when control instructions are executed. The choice to branch or not branch for these instructions is made during the execution stage, after two consecutive instructions have entered the pipeline's fetch and decode stages. If no branch is chosen, the execution continues uninterrupted. However, if the branch is taken, the prior two instructions must be drained out before the current branch instruction can be executed, resulting in a control hazard.

1. *Data Forwarding to eliminate data hazard:* Ahead of the source instruction reaching write back stage, data is made available to the ALU for dependent instructions, which is known as forwarding. This data forwarding technique identifies the dependant instruction in pipeline and forwards the data to the dependant instruction thus eliminating data hazards in the pipeline. *We'll be using both IE-IE and Mem-IE data forwarding.*

2. *Adaptive dymanic Branch prediction to eliminate control hazard:* It enhances the accuracy of prediction by taking into account the recent behaviour of previous branches as well. If the instruction is a branch instruction it uses Branch Target Address, which is fetched during the Instruction fetch stage. It also makes use of a Local Prediction Table, which contains two bits that decide whether or not a branch is taken. In comparison to a one bit branch predictor, the branch predictor state changes only after two consecutive mispredictions, resulting in enhanced accuracy.

Figure 2.2 shows the Fully bypassed datapath, which is used to bypass the result either from the IE/MEM stage or from MEM/WB stage based on the next intruction after decode stage.

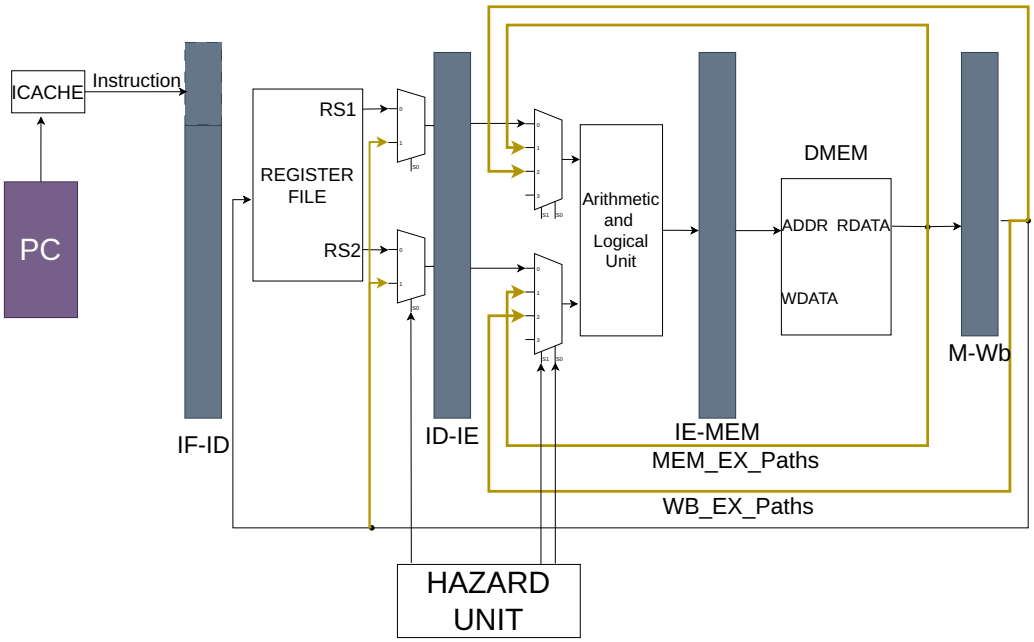

Figure 2.2: Bypass Datapath

If the current instruction is a ALU and ALUi type then we only need bypass and not stall. But if we have LD, JAL, JALR as our current instruction, then we need to stall the next instruction for 2 clock cycles. After 2 clock cycle, these instructions will be in the WB stage and hence we can then use the result of current instruction as an operand for the next waiting instruction. For the 2 cycle the stall signal will be asserted which will hold the instructions.

2.5.2 Branch Prediction Unit

When a program runs certain flow of instruction is fetched and executed in pipeline. However, when a branch instruction is fetched, it may change the flow of the instructions. The execution result of branch instructions determines whether to change the flow in the execution stage, which is in the middle of the pipeline stage. If it changes its flow, the next instructions that are already being executed by other hardware resources prior to the execution stage become useless, which leads to a pipeline flush and degrades the processing speed. Since the branch instructions take more than 20 % of all instructions when running a program, this pipeline flush becomes a huge performance degradation factor in processors.

Branch prediction unit contains three different blocks as shows in the below figure 2.3 : ***branch direction predictor***, ***branch target buffer (BTB***) and ***Return Address Stack***. When instruction is fetched, branch prediction unit will identify whether it is branch instruction. If the instruction is branch, the direction predictor will give the prediction result (Taken / Not Taken). If the branch is taken then the PC is updated by the target address obtained from BTB. The branch prediction unit is usually placed in instruction fetch stage. So, when prediction is correct the next instruction fetched after the branch instruction is from the correct instruction flow and pipeline can execute these instruction without any pipeline flush. But when prediction is wrong, the instruction executed after the branch needs to be flushed.

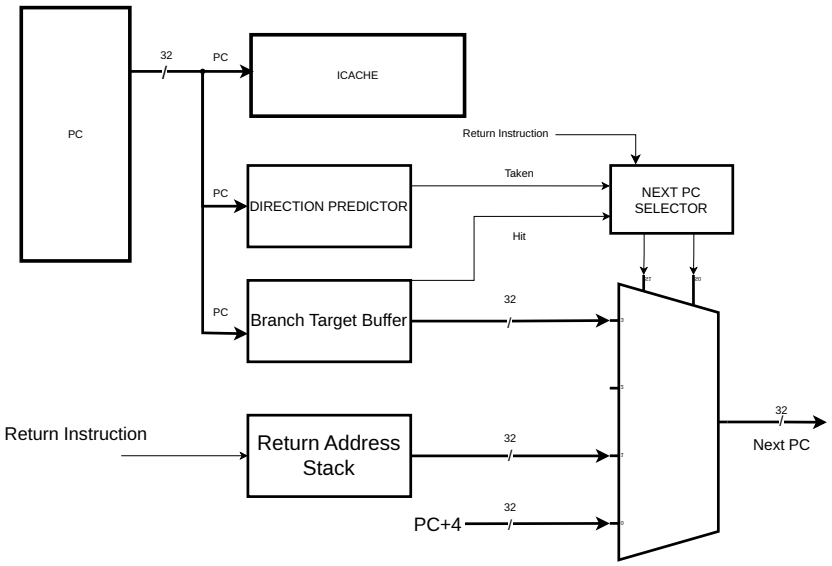

Figure 2.3: Branch Prediction Unit

The Branch Prediction Unit (BPU) in this architecture includes a Return Address Stack (RAS),

which is specifically used to predict the target of function return instructions (ret). The RAS stores the return addresses pushed during function calls (*jal* or *jalr* to *ra*) and pops them during returns, thereby improving prediction accuracy for nested and recursive function calls. This mechanism helps in maintaining pipeline flow and reducing control hazards caused by subroutine returns, especially in deep call hierarchies.

2.5.3 ICACHE

The instruction cache (ICache) implemented in this architecture is a 2-way set-associative cache, designed to enhance instruction fetch performance by reducing cache misses compared to a direct-mapped design. This configuration divides the cache into multiple sets, with each set containing two lines (or "ways"), allowing the processor to store multiple blocks that map to the same index, thereby minimizing conflicts. The use of a 2-way associative structure strikes a balance between hardware complexity and hit rate efficiency, ensuring faster access to instructions while maintaining manageable control logic. This setup is particularly beneficial in pipelined processors where continuous and rapid instruction supply is critical to maintain throughput and minimize stalls.

2.5.4 DCACHE

The data cache (DCache) used in this design is an 8KB, 2-way set-associative, write-back cache, optimized for efficient memory access and reduced latency in load/store operations. The 2-way set-associative architecture divides the cache into sets with two lines per set, offering a good compromise between simplicity and performance by reducing conflict misses when multiple memory blocks map to the same index. The write-back policy employed ensures that write operations are first made to the cache, and only written back to main memory when a cache line is replaced. This reduces memory traffic and improves overall performance, especially in programs with frequent data modifications. The DCache is tightly integrated into the pipeline to support fast and efficient data access, contributing significantly to the overall execution speed of the processor.

2.6 Proposed architecture

The earlier RISC-V processor architecture was designed with a fundamental execution pipeline and support for the base IMA extensions—Integer, Multiplication/Division, and Atomic operations. While this core provided a functional and efficient execution platform, it lacked several critical features necessary for full compliance with the RISC-V privileged architecture specifications. Notably, the previous design did not support CSR (Control and Status Registers), exception and interrupt handling, debug infrastructure, memory protection mechanisms (PMP), and address translation for virtual memory. These missing components limit its practical deployment, especially in real-world operating systems and secure embedded systems.

To address these limitations, our proposed architecture introduces a fully compliant RISC-V core with support for all major privileged functionalities. The processor will be enhanced with modules for **CSR** management, exception and **interrupt handling**, **debugging support**, TLB for virtual memory translation, and cache control features such as flushing and invalidation. We also aim to integrate Physical Memory Attributes (PMA) and **Physical Memory Protection (PMP)** units for enforcing access control and memory policies. The development of this architecture will **culminate in a custom ASIC implementation**, transitioning the design from FPGA prototyping to a silicon-based deployment for performance and power optimization.

2.6.1 Control and Status Registers (CSR)

CSR is an integral part of the RISC-V architecture that provides control over various processor functionalities such as performance counters, trap handling, privilege levels, and configuration registers. In the new architecture, a dedicated CSR block is implemented to handle both read and write operations in a single pipeline cycle. The CSR module decodes instructions like *csrrw*, *csrrs*, and *csrrc* to interact with specific registers. It interfaces closely with the exception/interrupt controller and privilege management, enabling dynamic control of processor behavior across different modes (User, Supervisor, Machine).

2.6.2 Exception and Interrupt Handling

Robust exception and interrupt handling is critical for reliable system behavior and compliance with operating system requirements. The new design includes a centralized trap controller that supports synchronous exceptions (e.g., illegal instruction, load/store faults) and asynchronous

interrupts (e.g., timer, external). It manages control flow redirection using the *mepc*, *mcause*, and *mtval* CSRs and distinguishes between privilege levels for servicing each event. The design ensures deterministic behavior through prioritization and vectored interrupt support, enabling the processor to handle real-time tasks and secure environments.

2.6.3 Debug Infrastructure

The debug module introduces capabilities for runtime inspection, single-step execution, breakpoints, and system halting. This infrastructure is essential for embedded system developers and operating system debugging. The design will conform to the RISC-V Debug Specification, integrating a Debug Module Interface (DMI) and halt/exception debug control. Features include program buffer execution, abstract register access, and interaction with external debug tools through JTAG or other interfaces. The debug system operates independently of normal execution flow, allowing inspection without altering program behavior.

2.6.4 Translation Lookaside Buffer (TLB) with Two-Level Page Table

To support virtual memory and operating system-level address translation, a Translation Lookaside Buffer (TLB) is introduced. The TLB stores recent translations from virtual to physical addresses, significantly accelerating memory access. Our TLB design uses a two-level page table model compliant with the Sv32 and Sv39 schemes defined in RISC-V. This allows efficient address translation for 32-bit and 64-bit systems, with hardware page walkers to fetch entries from page tables on TLB misses. The TLB is tightly integrated with memory management and privilege checking to enforce access rights and improve performance.

2.6.5 Cache Flushing and Invalidation

To maintain coherence and correctness in multi-threaded or I/O-intensive applications, the architecture includes support for cache flushing and invalidation. These mechanisms are implemented through specialized CSR instructions (like *fence.i*, *custom flush*, or *invalidate* commands) that target instruction and data caches. Cache control is particularly important during context switches, self-modifying code, or DMA transfers. Our design ensures that cache operations are synchronized with pipeline execution to avoid hazards and support safe memory sharing.

2.6.6 Physical Memory Attributes (PMA) and Physical Memory Protection (PMP)

Security and memory access control are fundamental for modern embedded systems. To this end, the architecture integrates PMA and PMP blocks. PMP enables software to define access permissions for memory regions, specifying whether a range is readable, writable, executable, or inaccessible at a given privilege level. This forms the foundation for isolating processes and protecting critical system resources. PMA, on the other hand, defines hardware attributes of memory regions (e.g., cacheable, bufferable), allowing the processor to handle various types of devices and memory-mapped I/O safely. These mechanisms are configured via CSRs and tightly coupled with the MMU and TLB.

2.7 ASIC Flow

ASIC (Application Specific Integrated Circuit) is a specialized type of integrated circuit meticulously designed to perform a specific function or set of functions within an electronic system. ASICs are tailor-made for a particular application, offering unparalleled efficiency and performance. There are two primary methods of ASIC design:

1. *Gate Array (Semi Custom) design :* This type of ASIC uses predesigned logic cells called standard cells, such as gates, multiplexers, and flip-flops. Standard cells are made using full-custom design methodology and serve as basic building blocks for ASIC design, ensuring the same performance and flexibility but reducing time and risk.

2. *Full Custom design:* The full-custom method is more complex and costly, but it can do much more than the gate array method. The size of the ASIC decreases significantly as the design incorporates only the necessary gates and electronics, and unused gates are deleted. These ASICs are designed for a specific purpose and support a particular function in the end product.

2.7.1 Chip Design flow

A typical design flow follows a structure shown below 2.4. Some of these phases happens parallelly and some sequentially.

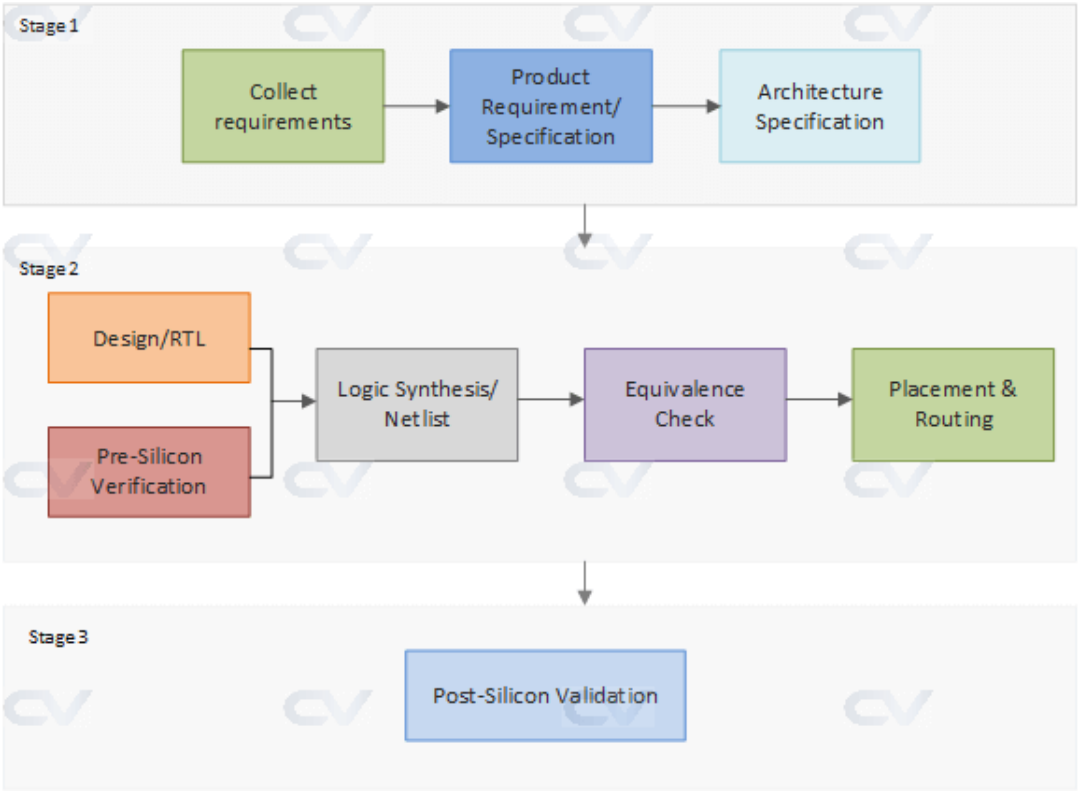

Figure 2.4: Chip Design Flow

Architecture: The Architects based on the reuirement specifications and market need, comes up with a system level view of how the chip should operate. They will decide what components to use, on which frequency should the chip run, what should be the footprint, operating voltages and estimate the power numbers. They also decide how will be the dataflow inside the chip

Digital Design: Since the designs are complex nowdays, we cannot write RTL from scratch and hence various IP's are taken and integrated into the design. A behavioral description of the blocks are written in HDL and the design is analyzed in terms of functionality, performance and other high level issues.

Verification: Once the RTL is ready, it needs to be verified for functional correctness. Various EDA tools like simvision and xcellium (from Cadence), Verdi (from Synopsys) are used for verifying the RTL. Currently, Verification has become an important part of chip designing and is the most time consuming process in the lifecycle. Various methodologies like UVM, OVM have been adopted for better verification of the design and to reduce time to market

To save time and reach functional closure, both the design and verification teams operate in parallel where the designers "release" the RTL version, and the verification team develops the testbench enviroment and test cases to test the functionality of the RTL Version. A UVM

scoreboard is shown in figure 2.5 below.

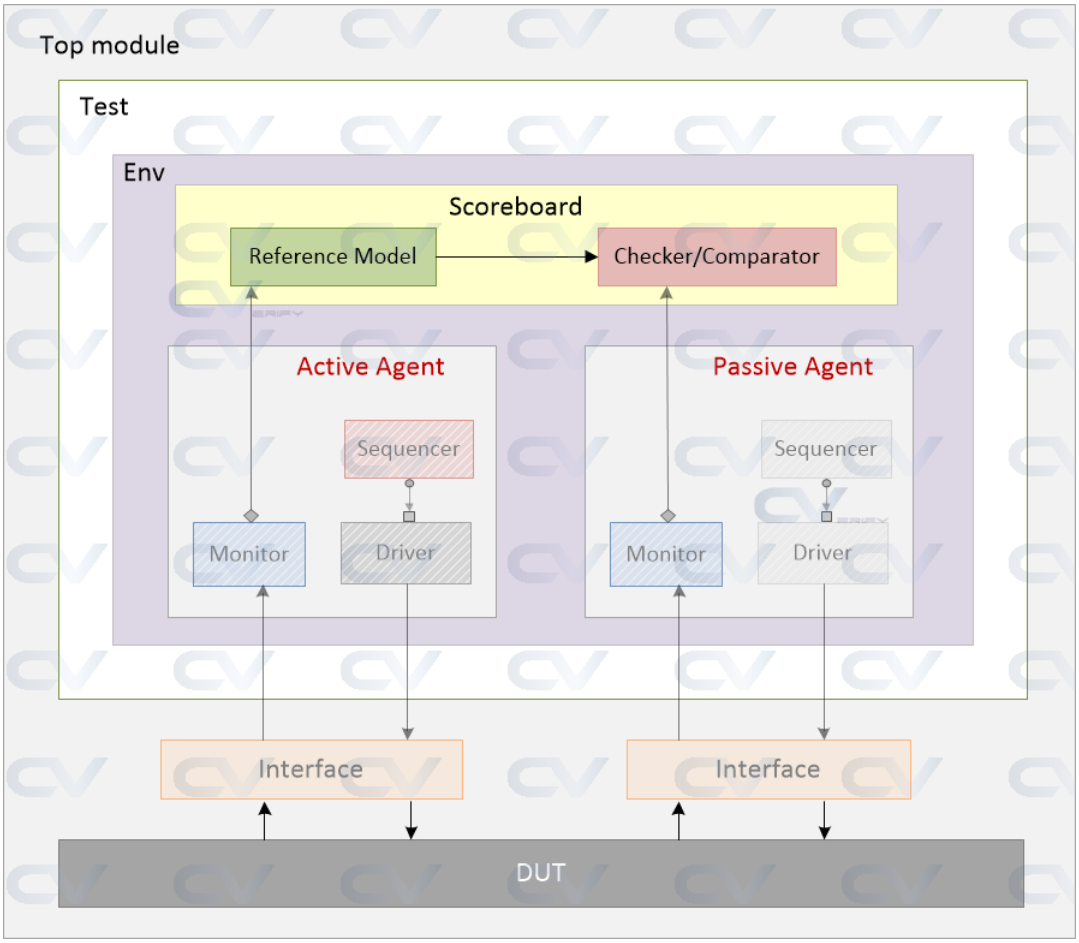

Figure 2.5: UVM Scoreboard

Logic synthesis and Equivalance check: Once the design is ready and verified, we convert it into hardware schematic with real elements like combinational gates, muxed, FlipFLops. Logic synthesis tool like Genus (from Cadence), enable us to convert RTL description into a gate level netlist. Tool ensures the netlist meets the timing, area and performance. We have the library files for each element in the netlist which we get from foundaries as PDKs.

The gate level netlist is checked for logical equivalence with the RTL and GLS (Gate level Simulations) are required to ensure the synthesized design still holds the same functionality. Genus GUI is shown in figure 2.6 below.

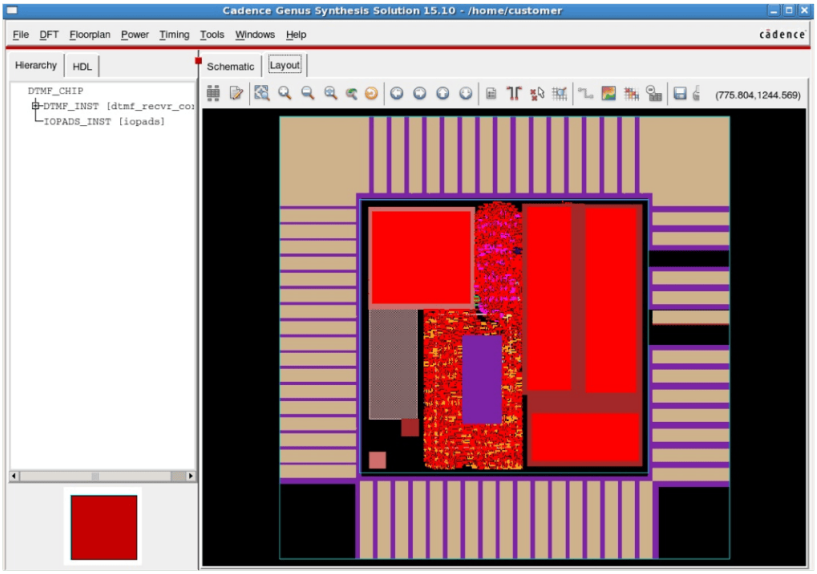

Figure 2.6: Genus Tool for Synthesis

Placement and Routing: The netlist is sourced into the physical design flow, where automatic place and route (APR or PnR) is done with the help of EDA tools, Eg. Cadence Innovus, Encounter and Synopsys IC compiler. This will select and place standard cells into rows, define ball maps for input-output, create different metal layers, and buffers to meet timing. Once this process is done, the layout is send for fabrication. Cadence Encounter GUI is shown in below figure 2.7.

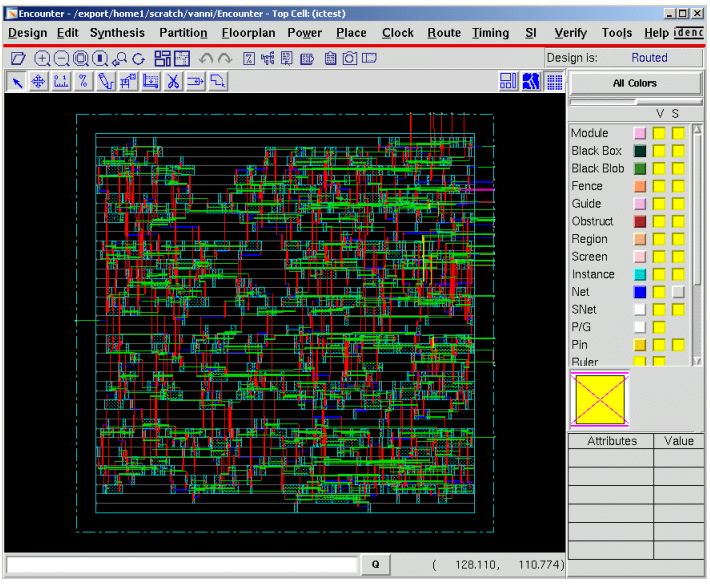

Figure 2.7: Cadence Encounter

2.8 JTAG (IEEE 1149.1)

The Joint Test Action Group (JTAG), formalized as the IEEE 1149.1 standard, provides a standardized serial boundary-scan architecture. It is primarily intended for structural testing, in-system programming, and embedded debugging of digital integrated circuits (ICs). JTAG has evolved into an indispensable interface, especially within modern System-on-Chip (SoC) and FPGA environments, enabling non-intrusive access to internal logic and registers. This capability facilitates design validation, post-silicon verification, board bring-up, and runtime diagnostics, making JTAG a cornerstone in contemporary digital design workflows.

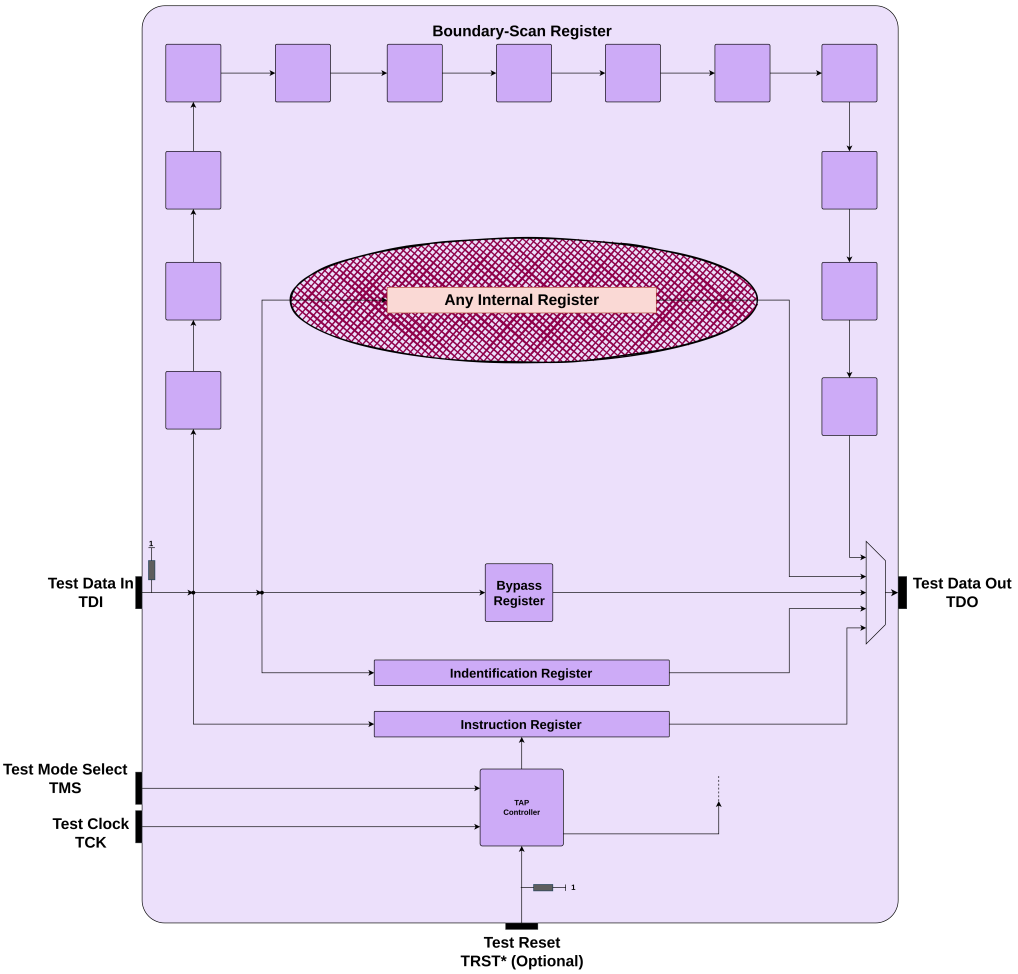

Figure 2.8: IEEE 1149.1 Chip Architecture

2.8.1 JTAG Signal Description

- ***TDI (Test Data In):*** A unidirectional serial input line through which instruction or test data is injected into the target device. In multi-device configurations, it typically connects to the *TDO* of the preceding device.
- ***TDO (Test Data Out):*** The serial output line that transmits the response data from the device. It is chained to the *TDI* of the subsequent device.
- ***TCK (Test Clock):*** A dedicated clock signal for synchronizing all JTAG operations. It is independent of the system clock, enabling asynchronous test access.
- ***TMS (Test Mode Select):*** Governs transitions within the TAP controller's state machine. The state of *TMS* is sampled on the rising edge of *TCK*.
- ***TRST (Test Reset – optional)***: An asynchronous reset signal that forces the TAP controller into the Test-Logic-Reset state without requiring *TCK* activity.

2.8.2 JTAG Timing Diagram

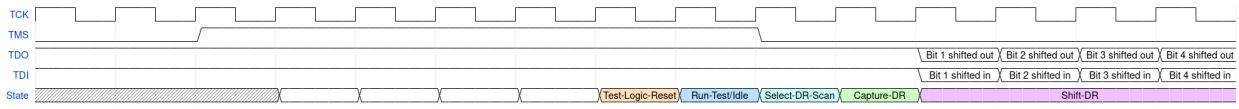

Figure 2.9: JTAG Timing Diagram

JTAG timing diagrams (as illustrated in 2.9) describe the temporal coordination between *TDI*, *TDO*, *TMS*, and *TCK* during scan operations. Key aspects include:

1. ***Setup and Hold Requirements:*** Data inputs *TDI* and *TMS* must remain stable relative to the rising edge of *TCK* to ensure deterministic sampling.
2. ***TDO Validity:*** Output data becomes valid on the falling edge of *TCK* and must be captured externally before the next transition.
3. ***Data and Instruction Shifting:*** Precise clocking ensures successful bit-wise propagation through instruction and data registers.

Robust timing analysis becomes increasingly critical as scan chains grow in length or operate at higher frequencies.

2.8.3 Test Access Port (TAP) Overview

The Test Access Port (TAP) acts as the primary interface between external JTAG tools and the internal infrastructure of the target integrated circuit. It establishes a serial communication path that enables access to the *Instruction Register (IR)*, various *Data Registers (DRs)*, and the TAP Controller's state machine. Through this interface, test patterns can be delivered, internal data can be acquired, and control signals can be propagated—all without the need for additional I/O resources. Owing to its minimal pin requirements and standardized operation, the TAP plays a critical role in facilitating embedded debugging, hardware validation, and boundary scan operations within modern digital systems.

2.8.4 TAP Controller State Machine

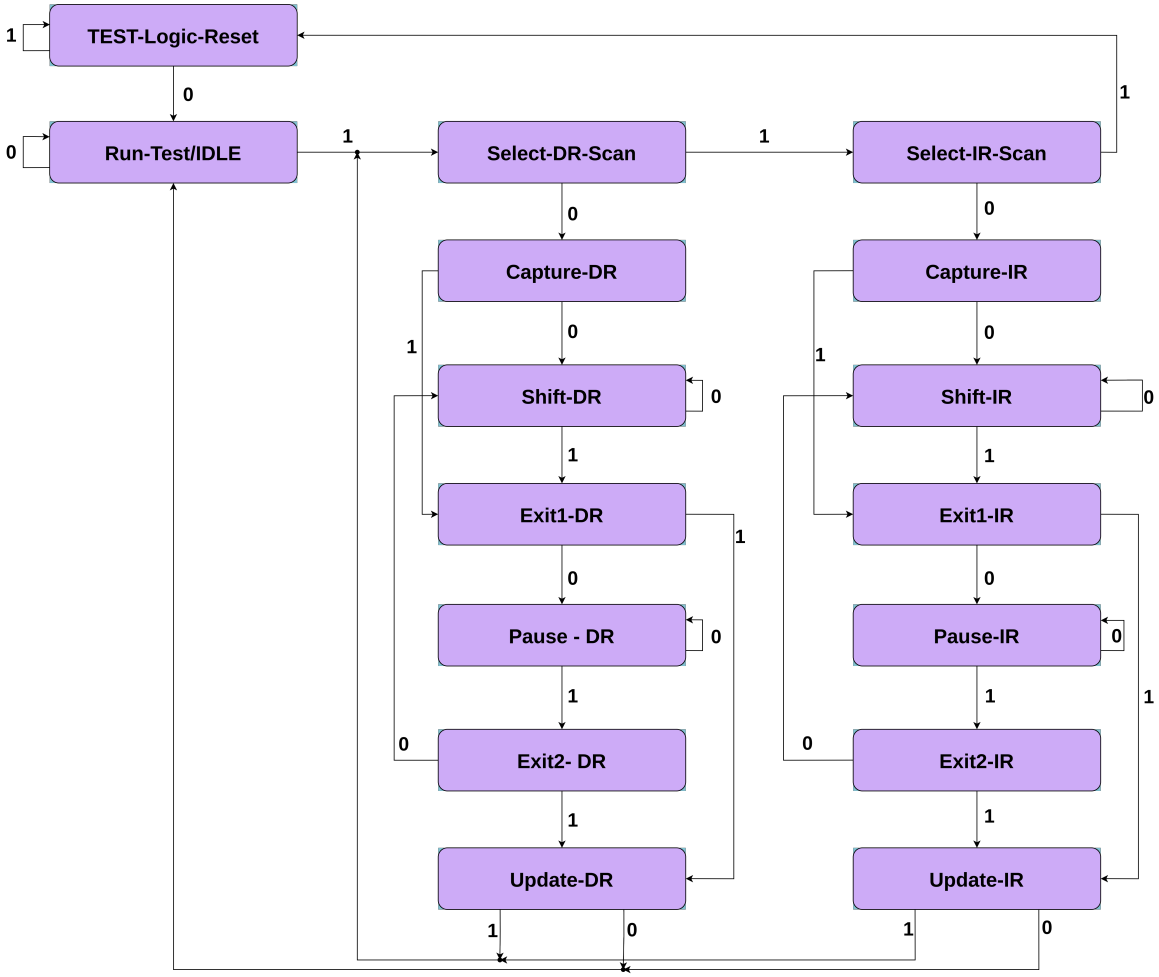

Figure 2.10: TAP Controller State Machine

At the heart of the JTAG interface lies a deterministic 16-state finite state machine known as the Test Access Port (TAP) controller (see Figure 2.10). This controller orchestrates the sequencing and execution of all JTAG operations by controlling access to the internal instruction and data registers of the target device. The TAP controller is central to enabling flexible test and debug functionality over a minimal pin interface. Its primary states are outlined below z:

- ***Test-Logic-Reset:*** This state asynchronously initializes all JTAG logic within the device, ensuring that the system enters a well-defined, inactive configuration. It is commonly used to recover from an unknown or unstable state before initiating a test or debug session.
- ***Run-Test/Idle:*** After reset, the TAP enters this state, which either maintains the system in a passive idle condition or triggers the execution of specific test operations, such as Built-In Self-Test (BIST) or continuous scan testing, depending on the configuration of the active instruction.
- ***Select-DR-Scan / Select-IR-Scan:*** These intermediate states determine the type of register path—*Data Register (DR)* or *Instruction Register (IR)*—that will be accessed next. They serve as branch points for directing the TAP controller's flow toward the desired scan operation.
- ***Capture, Shift, Pause, Update States:*** These sets of states exist for both *DR* and *IR* scan paths and are responsible for the detailed handling of serial data. The Capture state samples current values into the respective shift register; Shift allows for serial movement of bits through the register via *TDI* and *TDO*; Pause temporarily halts shifting to allow external processes or decision-making; and Update writes the shifted data into the active register, committing the operation.

State transitions within the TAP controller are governed by the *Test Mode Select (TMS)* signal, which is evaluated on the rising edge of the *Test Clock (TCK)*. This serialized control scheme enables deterministic navigation through the TAP state machine using a single control line, eliminating the need for parallel signaling or complex external logic. Such a mechanism ensures precise coordination of test operations and register access.

2.8.5 Instruction Register

The *Instruction Register* (*IR*) defines the operational context of the JTAG interface by determining which data register is accessible during a given operation. Typically 5 bits wide, the *IR*

is loaded serially via the Shift-IR and Update-IR states within the TAP controller. Standardized instructions include:

- ***EXTEST:*** Enables external pin-level testing through the boundary scan register.
- ***SAMPLE/PRELOAD:*** Captures the real-time state of selected signals or preloads values into scan chains for subsequent operations.
- ***IDCODE (optional):*** Provides a static device identifier to facilitate device recognition and configuration. If the identification register is not implemented in hardware, the *IDCODE* instruction must be interpreted as a *BYPASS* instruction.
- ***BYPASS:*** Selects a minimal 1-bit data register, allowing the device to be bypassed in a scan chain and enabling faster access to downstream devices.
- ***USER Instructions:*** Reserved opcodes that support vendor-defined or application-specific functionality, enhancing the flexibility of the JTAG interface.

The support for *USER* instructions significantly extends the utility of the *IR* by enabling custom debug features and facilitating proprietary extensions within standardized test workflows.

2.8.6 Data Registers Overview

Data Registers (DRs) in the JTAG interface are used for data exchange between the external debugger and the target device. The active *DR* is determined by the current instruction loaded into the *Instruction Register (IR)*, with different instructions providing access to various *DRs*. These registers allow operations such as reading and writing to internal registers, configuring device state, and performing other test and debug operations.

Data is shifted serially through the *DRs* during scan operations, enabling external debuggers to interact with the processor's internal state. This mechanism supports a range of tasks, including control, inspection, and configuration, all facilitated by the JTAG scan protocol. The ability to select different *DRs* based on the loaded instruction adds flexibility, enabling tailored debug and testing functionality specific to the target device's needs.

2.8.6.1 IDCODE Register

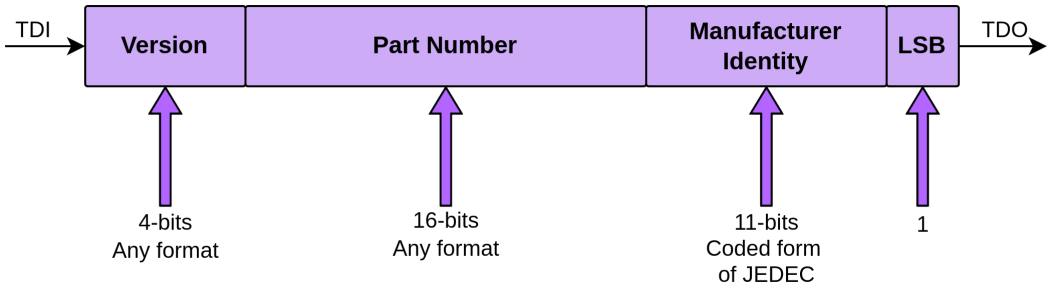

Figure 2.11: Device Identification Code Structure

The *IDCODE* register is a 32-bit read-only register that provides a standardized identifier for the target device (see Figure 2.11). Its structure is as follows:

- 4 bits for Version Number
- 16 bits for Part Number
- 11 bits for Manufacturer ID
- 1-bit Stop Bit (logic high)

The *IDCODE* register plays a critical role in automated toolchains by enabling target detection, configuration validation, and the provisioning of firmware. It facilitates device identification and ensures compatibility between the debugging tools and the target hardware, making it an essential component for efficient debugging and system management.

2.8.6.2 BYPASS Register

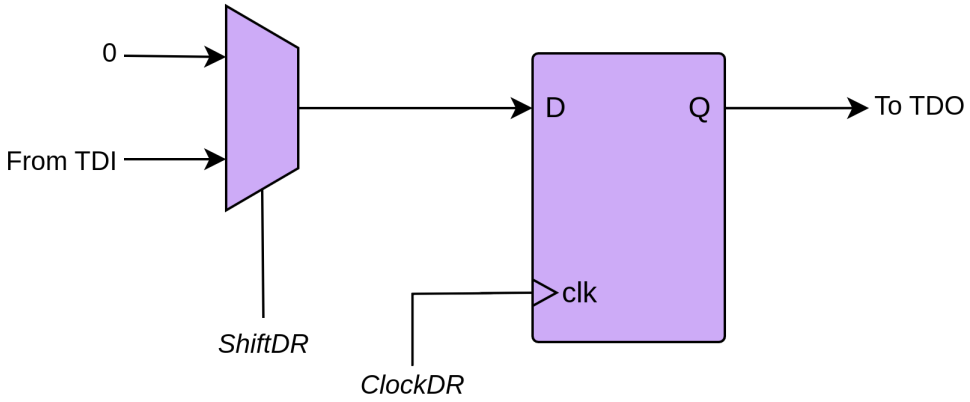

Figure 2.12: Bypass Register Structure

The *BYPASS* register is a one-bit shift register that serves to exclude a device from the scan chain when it is not the target of a JTAG operation. This functionality optimizes the scan chain by reducing latency and simplifying data transfers in multi-device configurations. When the device is in the bypass state, JTAG signals pass through it without interacting with its internal logic, enhancing the efficiency of the test and debug processes (see Figure 2.12).

2.8.6.3 Boundary Scan Register (optional)

The *Boundary Scan Register* (*BSR*) is an optional but highly valuable feature that enables access to the device's digital I/O pins through a shift register architecture. It facilitates a range of critical functions:

- ***Non-intrusive interconnect testing:*** The *EXTEST* instruction utilizes the *BSR* to test the device's interconnects without the need for physical probing, ensuring integrity without disrupting the system.
- ***Real-time pin state capture:*** Through the *SAMPLE* instruction, the *BSR* allows for the real-time capture of I/O pin states, providing a reliable method for observing device behavior during testing.
- ***Controlled input driving:*** The *PRELOAD* instruction enables the controlled driving of input signals, which is essential for performing diagnostics at the board level.

Its inclusion significantly enhances the observability and controllability of physical signal paths without requiring intrusive probing.

2.8.6.4 Implementation-Specific Data Registers

In advanced system designs, custom data registers are often implemented to support specialized functions tailored to the specific needs of the device. These registers may include:

- ***Debug module communication:*** Registers designed to facilitate efficient communication with on-chip debug modules, enabling detailed control and monitoring during the debugging process.
- ***Memory or CSR (Control and Status Register) access:*** Registers that provide direct access to memory regions or critical system control registers, enhancing low-level interaction with the system's internal state.
- ***Performance monitoring:*** Custom registers dedicated to tracking system performance metrics, providing insights into operational efficiency, resource utilization, or bottlenecks.

These user-defined data registers significantly enhance the flexibility and functionality of the JTAG interface, particularly in FPGA or processor-centric systems, where seamless integration with on-chip resources is essential for effective operation and debugging.

2.8.7 Supported JTAG Instructions

JTAG-compliant devices implement a core subset of instructions as defined by the IEEE 1149.1 specification. These instructions govern the configuration of the scan path and determine which data register is active during JTAG operations. Some of the most common instructions include:

- ***BYPASS:*** Activates the 1-bit bypass register.
- ***IDCODE***: Enables the *ID Register* for device identification.
- ***EXTEST:*** Facilitates external connectivity testing.
- ***SAMPLE/PRELOAD:*** Provides mechanisms for input capture or pattern setup.
- ***USER Instructions:*** Enable access to implementation-specific logic.

Instruction decoding and register mapping form the foundation of low-level test and debug access in JTAG-based systems. The flexibility to define and integrate custom instructions is a key feature of the JTAG architecture, ensuring that it can be adapted to meet the requirements of a wide range of applications, from basic device testing to complex, domain-specific debug scenarios.

In summary, the JTAG interface is an essential tool for test, debug, and in-system configuration in complex digital systems. Its well-structured state machine, standardized signaling protocol, and extensible register architecture make it a powerful mechanism for verification, post-silicon debugging, production testing, and ongoing system maintenance.

2.9 RISC-V Compliant Debugger

2.9.1 Overview of RISC-V Debug Specification

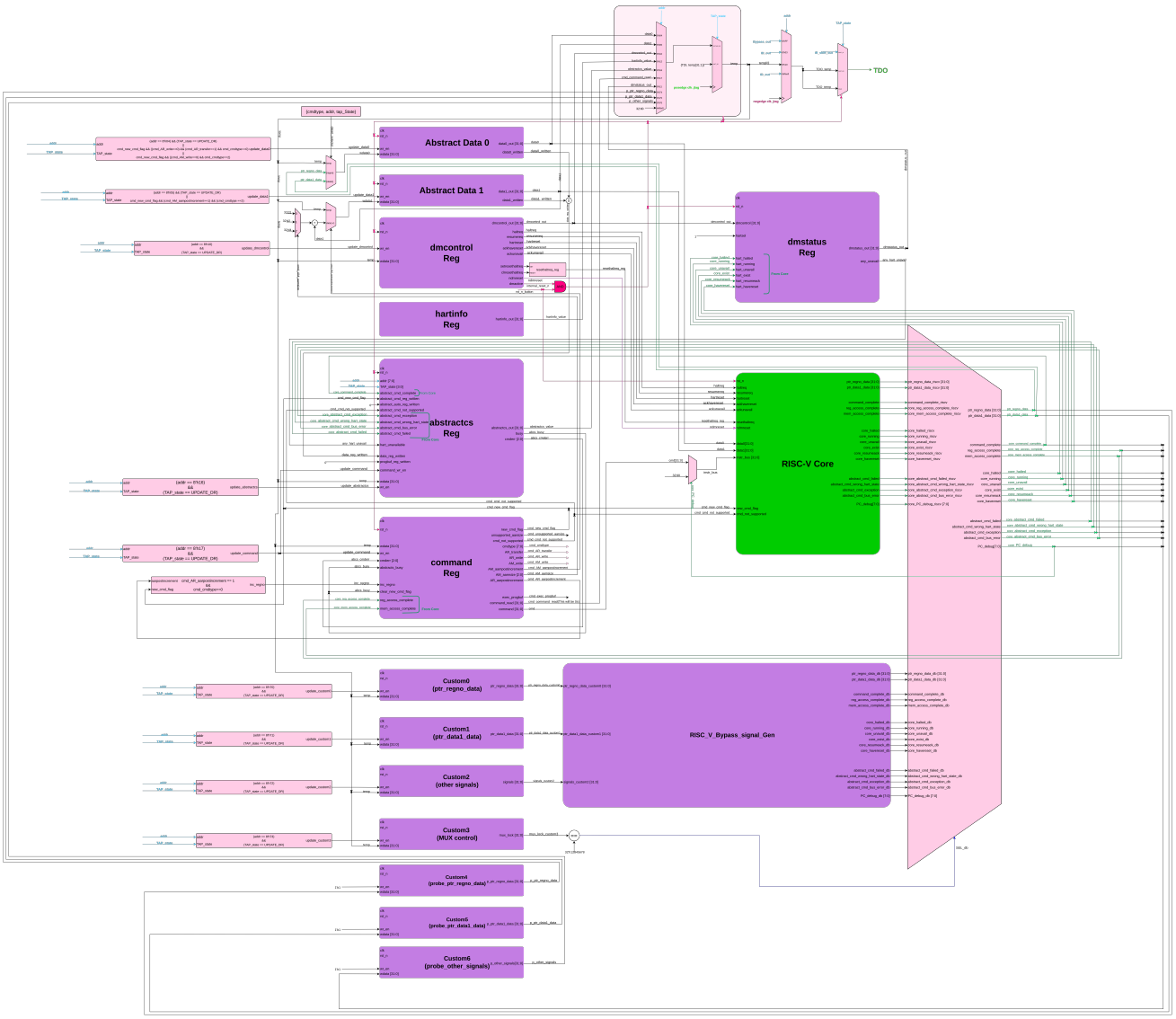

Figure 2.13: RISCV Debug System Overview

The RISC-V Debug Specification defines a modular and transport-agnostic framework for external debuggers to interface with RISC-V processors. Its architecture separates the transport, control, and execution layers to enable scalable and flexible debug support. The primary components include the *Debug Transport Module (DTM)*, the *Debug Module Interface (DMI)*, and the *Debug Module (DM)*—each fulfilling a distinct role in the debug pathway.

Debug communication typically begins with a debugger host (e.g., *GDB*) interfacing through a hardware driver (e.g., *OpenOCD*), which communicates with the *DTM* using a physical transport such as JTAG or UART. The *DTM* forwards these transactions to the *DM* via the *DMI*, which exposes a set of control and status registers that allow register-level interaction with the processor core (hart). This architecture supports both single and multi-hart systems, enabling robust and extensible debugging functionality (see Figure 2.13).

2.9.2 Debug System Components

The RISC-V debug architecture is modular, consisting of three primary components: the *Debug Transport Module (DTM)*, the *Debug Module Interface (DMI)*, and the *Debug Module (DM)*. These components work in conjunction to facilitate interaction between an external debugger and the internal state of one or more RISC-V harts.

- ***Debug Transport Module (DTM):*** Acts as the physical access point for an external debugger. It implements a protocol-specific interface—typically JTAG or Serial Wire Debug (SWD)—and forwards incoming debug transactions to the Debug Module through the *DMI*.
- ***Debug Module Interface (DMI):*** Serves as a memory-mapped interface between the *DTM* and the *Debug Module*. It supports 32-bit read and write transactions to a defined address space, enabling command and status exchange. The interface is intentionally minimalistic to ensure lightweight integration with transport mechanisms.
- ***Debug Module (DM):*** Implements the primary debugging functionality. It controls hart execution (*halt*, *resume*), provides abstract access to memory and registers, and supports programmable instruction execution using a program buffer. A single *DM* typically serves all harts in a system, allowing fine-grained control via internal hart selection mechanisms.

Each hart (hardware thread) is associated with one *DM*. In multi-hart systems, a centralized *DM* facilitates debugging across all harts, simplifying design and reducing overhead.

2.9.3 Debug Module Interface (DMI)

The *DMI* provides the communication pathway between the JTAG interface (via the *DTM*) and the internal *Debug Module*. It exposes a 32-bit addressable register space where debug

operations can be initiated or queried. The interface is streamlined, requiring only address and data lines, enabling efficient read/write access without additional control handshaking. This simplicity supports both hardware and software implementations with minimal complexity.

2.9.4 Debug Module Registers

The Debug Module exposes several memory-mapped registers via *DMI* to configure, control, and monitor debugging operations. Key registers include:

- ***dmcontrol (0x10):*** Enables the debug module, selects the target hart, and issues halt/resume requests.
- ***dmstatus (0x11):*** Reflects the current status of all managed harts (e.g., *halted*, *running*, *unavailable*).
- ***abstractcs (0x16):*** Indicates the availability and configuration of abstract command mechanisms, such as the program buffer.
- ***command (0x17):*** Encodes abstract commands to be executed by the *DM* (e.g., register access).
- ***abstractauto (0x18):*** Automates execution of abstract commands and data access patterns.
- ***data0–dataN (0x04–0x0f):*** Temporary storage registers for abstract command operands or results.
- ***progbuf0–N (0x20–0x2X):*** Holds RISC-V instructions to be executed during debug mode using the program buffer.

Additional registers such as *hartinfo*, *nextdm*, *dmcs2*, and implementation-specific extensions can be included to support more advanced functionality or multicore management.

2.9.5 Abstract Commands and Program Buffer

Abstract commands provide a flexible, implementation-independent method for executing fundamental debug tasks. The *DM* supports the following command types:

- ***Access Register:*** Read or write general-purpose registers (*GPRs*) or control/status registers (*CSRs*).
- ***Quick Access:*** Perform a rapid sequence of halt, instruction execution, and resume for common diagnostics.
- ***Access Memory:*** Read or write system memory from the perspective of the target hart.

For more complex operations, a *Program Buffer* allows execution of short RISC-V instruction sequences. The buffer typically supports at least two instruction slots and must terminate with an *ebreak* (or *c.ebreak*) instruction. This mechanism allows custom register transfers, *CSR* manipulation, and even limited branching within debug mode.

2.9.6 Hart Debug Control & CSRs

Each hart features dedicated control and status registers for managing debug state:

- ***dcsr (0x7b0):*** The *Debug Control and Status Register (dcsr)* includes fields indicating the cause of entry into debug mode, the current privilege level, single-step enable, and other configuration bits.
- ***dpc (0x7b1):*** *Debug Program Counter (dpc)* stores the address at which the hart was halted.
- ***dscratch0, dscratch1 (0x7b2, 0x7b3):*** General-purpose *scratch registers* available for temporary storage by the debugger during execution of abstract commands or program buffer instructions.

These *CSRs* ensure safe and controlled operation of the debugger and allow seamless entry and exit from debug mode.

2.9.7 Debug Mode Operation

A RISC-V hart enters Debug Mode when specific events or conditions occur that indicate external or internal debug intervention. These conditions include:

- ***External Halt Request:*** The debugger sets the *haltreq* bit in the *dmcontrol* register via the *Debug Module Interface (DMI)*. This signal requests that the hart halt at the next instruction boundary.

- ***Trigger Events:*** The hart is configured with hardware breakpoints or watchpoints (via trigger modules). Upon matching a trigger condition—such as a specific instruction fetch, memory access, or address match—the hart automatically transitions to Debug Mode.

- ***Single-Step Execution:*** If the *step* bit is set in the *dcsr (Debug Control and Status Register)*, the hart executes one instruction and then automatically halts and enters Debug Mode. This is useful for instruction-level stepping during software debugging.

Once a hart enters Debug Mode, the following changes occur in its execution environment:

- ***Instruction Fetch Is Disabled:*** The hart's normal instruction execution pipeline is stalled. It does not fetch or retire instructions from the normal program memory, effectively freezing the program state.

- ***Access to Debug Resources:*** While halted, the Debug Module gains exclusive access to the hart's internal state. Abstract commands issued by writing to the *command register* allow the debugger to read or write general-purpose registers (*GPRs*), control and status registers (*CSRs*), or *memory*.

- ***Execution via Program Buffer:*** A small, fixed-size instruction buffer (*progbuf*) inside the Debug Module allows the debugger to inject and execute RISC-V instructions while the hart is halted. The program buffer enables flexible, direct manipulation of the hart state (e.g., loading registers from memory, executing computation, etc.). For correct behavior, program buffer execution must end with an *ebreak* or *c.ebreak* instruction, which causes control to return to the Debug Module.

- ***Preservation of Program State:*** Upon entering Debug Mode, the current value of the *Program Counter (PC)* is saved in the *dpc* register. The cause of entry is recorded in *dcsr.cause*, which can indicate external request, trigger match, single-step, or exception during debug execution. The *dcsr* also reflects the previous privilege level and various control bits such as *step*, *ebreakm*, *ebreaks*, and *ebreaku*.

Exiting Debug Mode is initiated by the debugger through the *dmcontrol* register. The debugger must clear the *haltreq* bit and set the *resumereq* bit. Once the Debug Module confirms that the hart is ready (via *dmstatus*), the hart resumes normal instruction execution from the address stored in *dpc*.

If an invalid instruction is executed in the program buffer, or if an exception occurs during abstract command execution, the hart remains in Debug Mode, and the corresponding error code is updated in the *cmderr* field of the *abstractcs* register. This ensures that faulty debug operations do not unintentionally resume program execution or corrupt state.

This robust mechanism for entering, managing, and exiting Debug Mode makes the RISC-V debug architecture suitable for a wide range of use cases—from deeply embedded systems requiring minimal overhead to complex multi-core SoCs needing advanced debugging and trace capabilities.

2.10 Target Specification

Target specifications formulated for the design are:

- 32-bit, 5 stage Pipelined Processor
- Supports RV32G (RV32IMAFD) instructions
- Integrated Floating Point Unit
- Split I and D (Instruction and Data) caches
- Virtual Memory
- Data and Instruction Translation Lookaside Buffers (DTLB & ITLB)
- Branch Prediction Unit
- System Counters
- Exception and Trap controller
- CLINT Interrupt controller for timer and software interrupts
- ZiCSR extension support
- Debug Support
- PMP support for security applications

2.11 Conclusion

In the above study, we have compared various industrial and academic RISC-V designs. The RISC-V ISA was thoroughly studied to understand the interaction between software and hardware, allowing us to identify areas where hardware can be enhanced to handle certain anomalies—such as hazards and stalls—more efficiently, thereby reducing the compiler's burden and accelerating the software development process. Building on the insights gained from literature and prior designs, we proposed a new RISC-V architecture incorporating several critical design blocks and features. The inclusion of features such as CSR, exception and interrupt handling, debug support, memory protection, TLB-based address translation, and cache management will make this processor suitable for academic research and low-power embedded control applications. Furthermore, the design will progress toward an ASIC implementation, enabling more efficient, compact, and power-optimized deployment in real-world embedded systems.

Chapter 3

Design

This chapter provides a comprehensive overview of the architectural and functional aspects of our RISC-V processor design. It details the structure of the processor's pipeline and explains how various functional units are seamlessly integrated into the core. The flow of instruction execution—from fetching to write-back—is described along with how the pipeline handles different scenarios. The processor is equipped with full interrupt and exception handling, which has been carefully designed and integrated into the pipeline control logic.

A key enhancement in our design is the implementation of a Memory Management Unit that supports the Sv32 virtual memory scheme with a two-level page table structure and dynamic Translation Lookaside Buffer updates. To enforce memory access control, a Memory Protection Unit has been incorporated that allows selective access to predefined physical memory regions, configurable during the boot process.

Additionally, a Core Local Interrupt controller has been integrated to manage software-generated and timer-based interrupts. For debugging, a comprehensive Debug Unit is developed that supports features such as single-stepping, hardware breakpoints, and the ability to inspect core registers and memory. This unit is interfaced with a custom-designed JTAG controller, enabling external debugging tools like GDB to connect and interact with the core during software development and testing.

3.1 Design Overview

Processor Core and ISA Support

- Implements a 32-bit RISC-V processor core compliant with the RV32G ISA, supporting integer (I), multiplication/division (M), atomic (A), single-precision (F), and double-precision (D) instructions.
- Designed with a 5-stage pipelined architecture comprising Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) stages.

Cache Architecture

- Equipped with separate Instruction and Data Caches, each with:
  - 8 KB capacity
  - 2-way set associative mapping
  - 32-byte block size
  - Least Recently Used (LRU) replacement policy
  - Write-back eviction strategy for efficient memory management
  - Software cache flush and Cache invalidation support

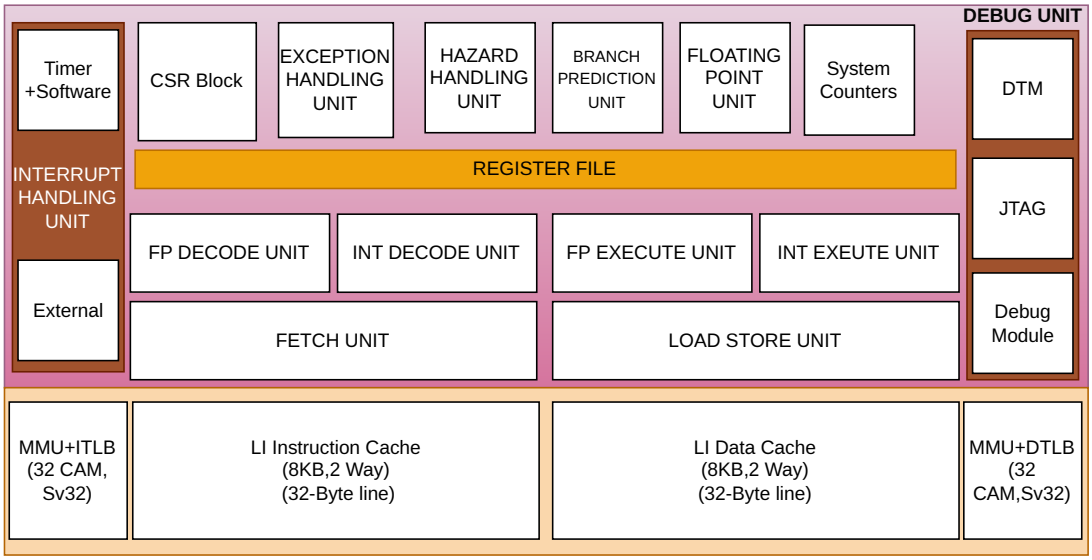

Figure 3.1: RV32G Processor architecture

Virtual Memory and Translation Lookaside Buffers (TLBs)

- Supports Sv32 virtual memory architecture with page table walking hardware.
- Instruction and Data TLBs include:
  - 32 fully-associative CAM entries per TLB
  - Support for two-level page tables
  - Hardware-managed TLB filling for improved performance
  - Virtual address-based replacement policy

Memory Protection

- Incorporates a Memory Protection Unit (MPU) capable of managing up to 16 programmable memory protection regions.
- Enables region-based access control to enhance system security and task isolation.

Branch Prediction Unit

- Features a high-performance Branch Prediction Unit (BPU) including:
  - Branch Target Buffer (BTB) with 512 entries, 4-way set associative
  - Pattern History Table (PHT) with 2048 entries, utilizing 2-bit saturating counters
  - Return Address Stack (RAS) with 32 entries for accurate subroutine return address predictions

Interrupt and Exception Unit

- Provides support for software, timer and external interrupts.
- Machine level CSRs are implemented for internal exception handling
- Supports CLINT based interrupt architecture

Debug Capabilities

- Integrates JTAG controller for debugging using Serial Wire.
- Supprts single stepping, breakpoint and trace debug.
- Allows inspection of register state, CSRs and Caches using debugger

3.2 Core Architecture

The processor is a 32-bit implementation of the RISC-V architecture, fully compliant with the RV32G standard [22]. The "G" extension signifies that the processor supports a comprehensive instruction set including the base integer instructions (I), multiplication and division (M), atomic operations (A), single-precision floating-point (F), and double-precision floating-point (D). This makes the core suitable for both general-purpose computing and applications requiring numerical processing or concurrency control. The processor is designed for high efficiency and real-time capability, making it suitable for embedded systems and operating system-level workloads. It also supports RTOS execution and includes a debug module capable of halting execution, single-stepping through instructions, and executing commands from a program buffer or via abstract command mechanisms. The processor microarchitecture is given in figure 3.2.

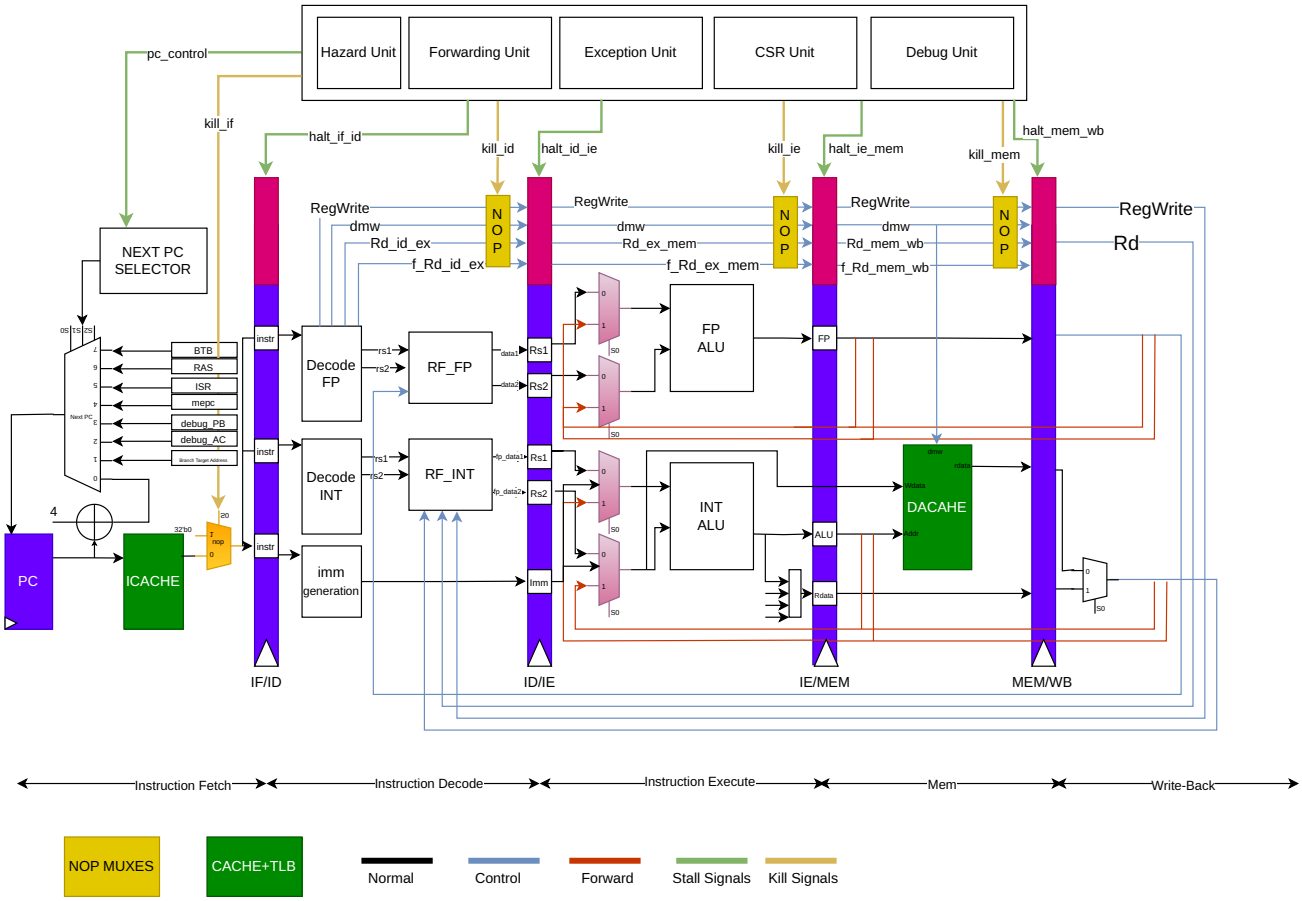

Figure 3.2: RV32G Processor micro-architecture

Key Architectural Components:

1. **Pipeline Architecture**: Implements a 5-stage pipeline comprising Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB), allowing instruction-level parallelism and efficient execution flow.

2. **Branch Prediction Unit**: Improves processor performance by guessing the outcome of branch instructions to reduce pipeline stalls.

3. **Memory Subsystem**: Features instruction and data caches (each 8KB, 2-way set associative), with write-back eviction and LRU replacement. Includes ITLB/DTLB with Sv32 virtual memory support and a memory protection unit (PMA/PMP) supporting up to 64 regions.

4. **Control and Status Registers (CSR)**: Fully compliant with the RISC-V privileged specification, enabling software to interact with exception handling, interrupt control, and processor state.

5. This processor supports external interrupts to be service as per the RISC-V Privileged specification and hardware interrupt handling.

6. **Exception Handling Unit**: Detects and manages synchronous events (e.g., illegal instructions, page faults), redirecting control flow via trap handlers configured through the CSR block.

7. **Debug Support**: Integrated debug module enables full system observability, including features such as single-step execution, halt/resume control, and execution of commands through the program buffer or abstract command interface.

3.3 Pipeline

The RV32G processor adopts a classic 5-stage pipeline architecture as shown in the below figure 3.3 —Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB)—augmented with additional stall, kill, and forwarding logic to manage hazards and ensure correct instruction execution. Each stage is designed for optimal throughput while maintaining support for advanced features like floating-point operations, and memory-mapped I/O.

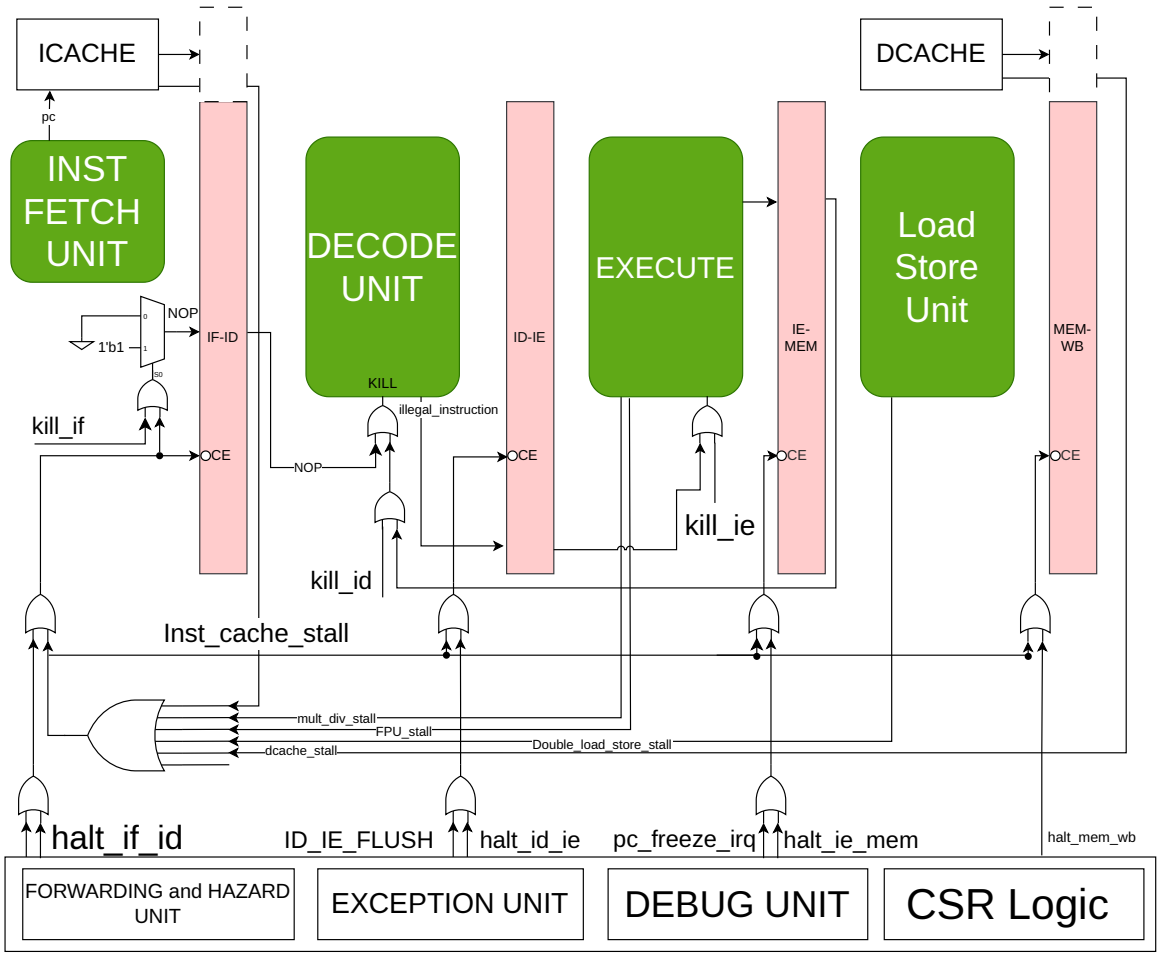

Figure 3.3: Pipeline Architecture

1. **Instruction Fetch (IF)**: The Instruction Fetch stage is responsible for retrieving instructions from memory using a tightly coupled *ICACHE* unit. Instructions are aligned on 32-bit word boundary and **fetch one instruction per cycle**, assuming the instruction-side memory system (e.g., instruction cache) is ready. The *inst_cache_stall* signal may be as-

serted in this stage when the instruction cache cannot immediately provide the requested data, resulting in a stall across the pipeline. Control signals like *kill_if* and *halt_if_id* can be asserted here to invalidate or stall instructions, typically due to branch mispredictions, exceptions, or interactions with the debug unit.

2. **Instruction Decode (ID)**: The Instruction Decode stage interprets the opcode and operands from the fetched instruction and initiates register file reads for source operands. This stage also interacts with the forwarding logic to resolve data hazards by reusing results from later stages when needed. If dependencies cannot be bypassed, stalls such as *halt_id_ie* may be introduced. Control and exception-related operations (e.g., CSR accesses) are decoded here, and *kill_id* can invalidate instructions due to misprediction or pipeline flushes.

3. **Execute (EX)**: The Execute stage performs the actual computation required by the instruction. It houses key functional units including:

   - The Arithmetic Logic Unit (ALU)
   - The Multiplier-Divider Unit (MDU) for integer multiply/divide instructions
   - Branch conditions are evaluated in this stage, and branch decisions are made here. Multi-cycle operations—such as multiplication, division, and certain floating-point operations—will stall the EX stage using control signals like *mult_div_stall* or *FPU_stall* until the operation completes. Data dependencies may also trigger pipeline stalls if forwarding is insufficient, particularly when dealing with memory or floating-point results.

4. **Memory Access (MEM)**: In the Memory Access stage, load and store instructions interact with the data cache or memory-mapped I/O peripherals. Address values are passed from the EX stage, and the memory hierarchy returns or receives data accordingly. This stage handles stalls due to:

   (a) Double load/store hazards (e.g., *double_load_store_stall*)

   (b) Data cache unavailability (*dcache_stall*)

   Integration with the PMA/PMP unit ensures that memory accesses respect protection and region-based access policies. The exception unit is actively involved here in validating memory accesses and raising faults if necessary.

5. **Write-Back (WB)**: The Write-Back stage completes the execution cycle by writing the results of ALU operations, memory loads, multiplication/division outputs, and floating-point computations back into the register file. This stage marks the end of the instruction's lifetime in the pipeline and also supports interaction with the debug module if a single-step or halt condition was in effect. The *halt_mem_wb signal* may pause this stage if there is an unresolved pipeline dependency, debug request, or flush event from exception or trap logic.

6. **Control and Coordination Units**

   - **Forwarding and Hazard Unit**: Ensures data is forwarded between pipeline stages when possible and detects data hazards that require stalls or data forwarding.
   - **Exception Unit**: Detects illegal instructions and synchronous faults, and generates appropriate trap/interrupt control.
   - **CSR Logic**: Manages privileged state and communication with system-level software, integrating closely with the exception unit.
   - **Debug Unit**: Introduces halts, breakpoints, or kill signals to support single-stepping, abstract command execution, and visibility into processor state.

7. **Kill and Halt Control Signals**: To resolve pipeline hazards or invalid instruction flows, the following kill and halt signals are used:

   (a) **Kill Signals**: *kill_if*, *kill_id*, *kill_ie* Used to invalidate instructions in various stages (e.g. after a branch misprediction or exception).

   (b) **Halt Signals:** *halt_if_id*, *halt_id_ie*, *halt_ie_mem*, *halt_mem_wb* temporarily stop progression between stages to prevent incorrect execution or allow time for dependent data.

   (c) *PC_CONTROL_IRQ*, *PC_FREEZE_IRQ*, *ID_IE_FLUSH* signals are asserted by the exception unit to halt the IF stage for fetching instructions from the ISR address and let other stages to complete the current instruction currently in the pipeline. This ensures less interrupt overhead.

The Table 3.1 shows the cycle counter per instructions which are executed in the pipeline.

Instruction Type	Cycles	Description
Integer Computational	1	Integer Computational Instructions are defined in the RISCV-V RV32I Base Integer Instruction Set.
CSR Access	1	CSR Access Instruction are defined in 'Zicsr' of the RISC-V specification.
Load/Store	1	Load/Store is handled in 1 bus transaction using both EX and WB stages for 1 cycle each. Misaligned transfers are not supported and will raise and exception
Multiplication	1 (mul) 4 (mulh, mulhsu, mulhu)	RV32G uses a single-cycle 32-bit x 32-bit multiplier with a 32-bit result. The multiplications with upper-word result take 4 cycles to compute.
Division Remainder	3 - 35 3 - 35	The number of cycles depends on the divider operand value (operand b), i.e. in the number of leading bits at 0. The minimum number of cycles is 3 when the divider has zero leading bits at 0 (e.g., 0x8000000). The maximum number of cycles is 35 when the divider is 0
Jump/ Branches	1 (Branch predicted correctly) 3 (Misprediction)	Jumps and Branches are taken in EX stage and hence 2 stages needs to be flushed
mret/ ECALL/ EBREAK	2	Mret is performed in the ID stage. Upon an mret the IF stage is flushed. The new PC request will appear on the instruction-side memory interface the same cycle the mret instruction is in the ID stage.

Table 3.1: Cycle count per instruction

3.4 System Control Unit (SysCtl)

The SysCtl Unit acts as a central controller for events that affect all other units, like interruptions and exceptions. The Control and Status Registers in RISC-V privileged ISA control the essential core settings. Hence, these CSRs are also managed by SysCtl Unit. Figure 3.4 shows the logical block diagram for System Control (SysCtl) Unit.

The RISC-V architecture, as detailed in the "RISC-V Instruction Set Manual Volume II: Privileged Architecture Document Version 20211203," provides a comprehensive framework for handling exceptions and interrupts. For our RV32G core, the exception unit is a critical component that manages these events according to the specifications outlined in the privileged document

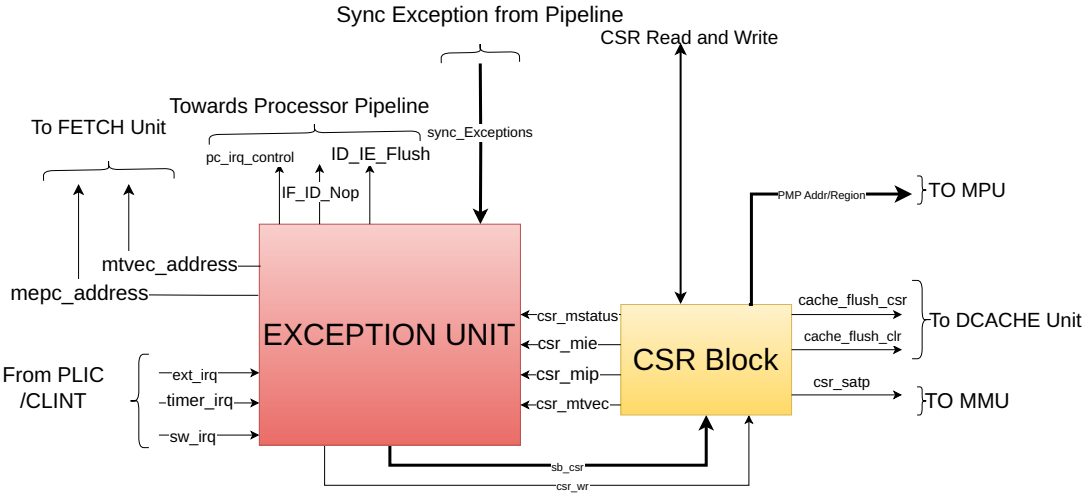

Figure 3.4: System Control Unit

3.5 EXCEPTION UNIT

The exception unit in RV32G core is responsible for managing and handling exceptions and interrupts according to the RISCV Privilaged specifications [23]. It interfaces with various components of the processor, including the pipeline, CSR (Control and Status Register) block, and memory units. The primary functions of the exception unit include:

- Detecting and handling synchronous and asynchronous exceptions.
- Updating relevant CSRs to reflect the state of the processor during an exception or inter-

rupt.

- Ensuring proper handling and resolution of exceptions to maintain system stability and security.

It handles the exception and interrupts for the Core. In the RISC-V architecture, exceptions and interrupts are both considered traps, but they differ fundamentally in their source and behavior. Exceptions are synchronous events that arise due to the execution of specific instructions—such as ***illegal operations, memory access faults, or system calls***—and they occur predictably during the instruction flow. These are typically used to signal errors or specific conditions that require immediate attention by the processor. **Interrupts**, on the other hand, are asynchronous events triggered by external or internal devices such as **timers, I/O peripherals, or software signals**. They occur independently of the current instruction execution and can happen at any point between instructions. While exceptions help the processor respond to internal execution issues, interrupts allow it to handle real-time events and interact with the outside world efficiently. Each of these exceptions have a specific code in the ***mcause*** register in the CSR unit. When any of these occurs, ***mcause*** register is loaded with the value assigned to the corresponding exception, current PC is stored in ***MEPC*** register.

The ***ECALL*** instruction is used to trigger a system call or exception, typically for transitioning control to the operating system or firmware, such as entering a handler in machine mode. Since it causes an exception, all instructions before the ECALL must complete before it is processed. The instruction is recognized during the Decode stage, but it will only proceed if no prior branch is taken—if a branch is taken, the Decode stage is flushed, and the ECALL will not execute. If a branch is not taken, the PC is loaded with the *mtvec* value and the decode and execute unit's are flushed and NOP is inserted.

MRET Instruction is used to return from an interrupt routine. PC is loaded with the value of mepc and normal program execution resumes.

The CSR (Control and Status Register) block in a RISC-V processor plays a pivotal role in exception and interrupt handling by maintaining critical system-level information. One of the key registers is *csr_mstatus*, which monitors the processor's current state, including the privilege level, interrupt enable settings, and context stack, ensuring a smooth transition during trap entry and return. The *csr_mepc* register records the program counter (PC) of the instruction that was executing at the time the exception occurred, allowing the processor to resume execution from the correct point after handling the trap. The *csr_mie* register manages the interrupt enable bits, determining which specific interrupts are allowed to trigger a trap, while the *csr_mip*

register indicates pending interrupts that await service. Finally, the *csr_mtvec* register holds the base address for the trap vector, guiding the processor to the appropriate handler routine during exceptions or interrupts.

Exception Handling

When entering an interrupt/exception handler, the core sets the *mepc* CSR to the current program counter and saves *mstatus.MIE* to *mstatus.MPIE*. All exceptions cause the core to jump to the base address of the vector table in the *mtvec* CSR. Interrupts are handled in either non-vectored mode or vectored mode depending on the value of *mtvec.MODE*. In non-vectored mode the core jumps to the base address of the vector table in the mtvec CSR. In vectored mode the core jumps to the base address plus four times the interrupt ID. Upon executing an mret instruction, the core jumps to the program counter previously saved in the *mepc* CSR and restores *mstatus.MPIE* to *mstatus.MIE*.

3.5.1 Exception Detection

- Synchronous Exceptions: These are detected during the execution of instructions, such as illegal instructions, address misaligned, or access faults.
- Asynchronous Exceptions: These include external interrupts, timer interrupts, and software interrupts. These are detected through external signals like *ext_irq*, *timer_irq*, and *sw_irq*.

Below Table 3.2 shows the current implemented interrupts and exception in the core.

Exceptions	I/E	Name	Description
Asynchronous	I	Machine Software Interrupt	Occurs when core writes 1 in MSIP register
Asynchronous	I	Machine Timer Interrupt	Occurs when core writes 1 in MTIMECMP register
Asynchronous	I	Machine External Interrupt	Occurs when external sources like peripheral, I/Os request for the interrupt

Table 3.2: Interrupts for RV32G

Exceptions	I/E	Name	Description
Synchronous	E	inst_addr_misaligned	When PC is not aligned to 32 bit word boundary
Synchronous	E	inst_access_fault	When PC tries to read a forbidden region in Instruction Memory
Synchronous	E	illegal_instruction	If the current instruction in the decode stage is illegal
Synchronous	E	csr_ro_wr	If we attempt to write a read-only CSR
Synchronous	E	sys[‘IS_EBREAK]	If we encounter EBREAK[1]
Synchronous	E	load_misaligned	If the Load address is not aligned to 32-bit word boundary
Synchronous	E	load_access_fault	If we try to load from not-allowed memory region
Synchronous	E	store_misaligned	If the Store address is not aligned to 32-bit word boundary
Synchronous	E	store_access_fault	If we try to store to a not-allowed memory region
Synchronous	E	sys[‘IS_ECALL]	If we encounter ECALL instruction
Synchronous	E	instruction_page_fault	If we do not have the correct R/X permission for that page
Synchronous	E	load_page_fault	If we do not have the correct R/W/X permission for that page
Synchronous	E	inst_dec_error	If we encounter a wrong instruction(Unspecified opcode)

Table 3.3: Exceptions for RV32G

3.5.2 CSR Updates

The below algorithm 3.1 shows how the CSR registers are updated when the core enters in the trap mode.

Algorithm 3.1 Exception Entry Sequence

/* Synchronous Exception Entering Sequence */

1. mepc <- Current PC
2. mcasue <- Specific cause code for that exception
3. mtval <- Value related to that exception
4. mstatus.mie <- 0 // Disable Global Interrupt Enable
5. mstatus.mip <- 1 // Pending Interrupt flag is raised

PC <- mtvec_address

- **mepc**: When a trap is taken into M-mode, *mepc* is written with the virtual address of the instruction that was interrupted or that encountered the exception.
- **mcause**: When a trap is taken into M-mode, *mcause* is written with a code indicating the event that caused the trap.
- **mtval**: When a trap is taken into M-mode, *mtval* is either set to zero or written with exception-specific information to assist software in handling the trap. Eg.
  - When a misaligned load or store operation results in an access-fault or page-fault exception, the *mtval* register is updated with a nonzero value that holds the virtual address corresponding to the part of the access that triggered the fault.
  - When an instruction access-fault or page-fault exception occurs, then *mtval* will contain the virtual address of the of the instruction that caused the fault, while mepc will point to the beginning of the instruction.
- **mstatus**: The *mstatus* register is updated to reflect the new state. For example, the MPIE bit is set to the current MIE state, and MIE is cleared.
- **mtvec**: The *mtvec* register is used to determine the base address for the trap vector. It has a direct mode and a vectored mode. In direct mode it always points to the base address of the vector table. In vectored mode, the mtvec registers points to the base_address + offset, determined by the mcause code of the interrupt that occured.

3.5.3 Trap Vector Calculation

The *mtvec* register is shown in the figure 3.5 below. The trap vector is calculated based on the *mtvec register* and the *exception cause*. The processor jumps to the appropriate handler address to handle the exception. The figure 3.5 shows the format of mtvec register.

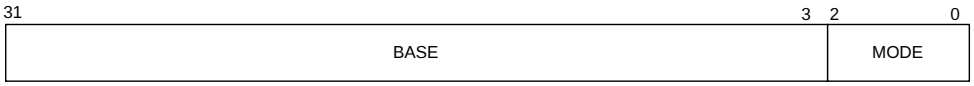

Figure 3.5: Mtvec register

When MODE=Direct, all traps into machine mode cause the PC to be set to the address in the BASE field. When MODE=Vectored, all synchronous exceptions into machine mode cause the PC to be set to the address in the BASE field, whereas interrupts cause the PC to be set to the address in the BASE field plus four times the interrupt cause number.

3.5.4 Mcause codes

The *mcause* register is shown in the figure 3.6 below. In the RISC-V privileged specification [23], the *mcause* CSR (Machine Cause Register) holds the reason for the last trap (which can be an interrupt or an exception). This register helps the system determine how to handle the event appropriately.

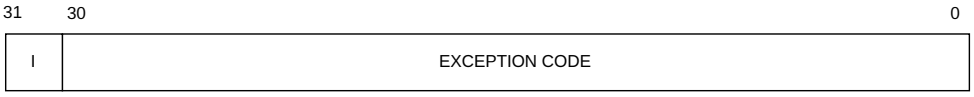

Figure 3.6: Mcause register

The most significant bit of mcause (bit XLEN-1) indicates whether the trap was caused by an interrupt (1) or by a synchronous exception (0). The remaining bits encode the specific cause code, which identifies the exact nature of the trap.

The table 3.5 shows the *mcause* codes corresponding to each exception or interrupt. These codes are essential in designing the trap handler logic, where different actions can be taken based on the type and source of the trap.

Interrupt	Exception Code	Description
1	3	Machine Software Interrupt
1	7	Machine Timer Interrupt
1	11	Machine External Interrupt
0	0	Instruction Address Misaligned
0	1	Instruction Access fault
0	2	Illegal Instruction
0	3	Breakpoint
0	4	Load Address misaligned
0	5	Load Access fault
0	6	Store/AMO Address misaligned
0	7	Store/AMO Access fault
0	11	ECALL from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	24	Instruction Deocde Error

Table 3.5: Mcause exception codes

3.5.5 Pipeline Control

The exception unit may signal the pipeline to flush and ensure that no further instructions are executed until the exception is handled. This is crucial to prevent further exceptions or incorrect state changes.

When a synchronous (e.g., ***msie***, ***mtie***, ***meie***) or asynchronous exception occurs, the ***trap_en*** signal is asserted, stalling the pipeline to halt instruction flow. Relevant CSR registers are updated with exception details, and the *mtvec* register provides the vector table base address, which is loaded into the PC to redirect execution to the exception handler. Upon encountering the MRET instruction in the decode stage, the PC is restored from the saved address, and the core resumes normal execution.

Handling interrupts in hardware offers several key advantages over software-based handling, particularly in real-time and embedded systems where timing is critical. Hardware interrupt handling significantly reduces interrupt latency, which is the time between the occurrence of an interrupt and the start of the corresponding interrupt service routine (ISR). This is crucial in

real-time systems, where deterministic and timely responses are required.

With hardware support, interrupts can be detected and serviced almost immediately, without the overhead of polling or extensive context checks in software. The figure 3.7 shows the integration of exception unit in with the pipeline

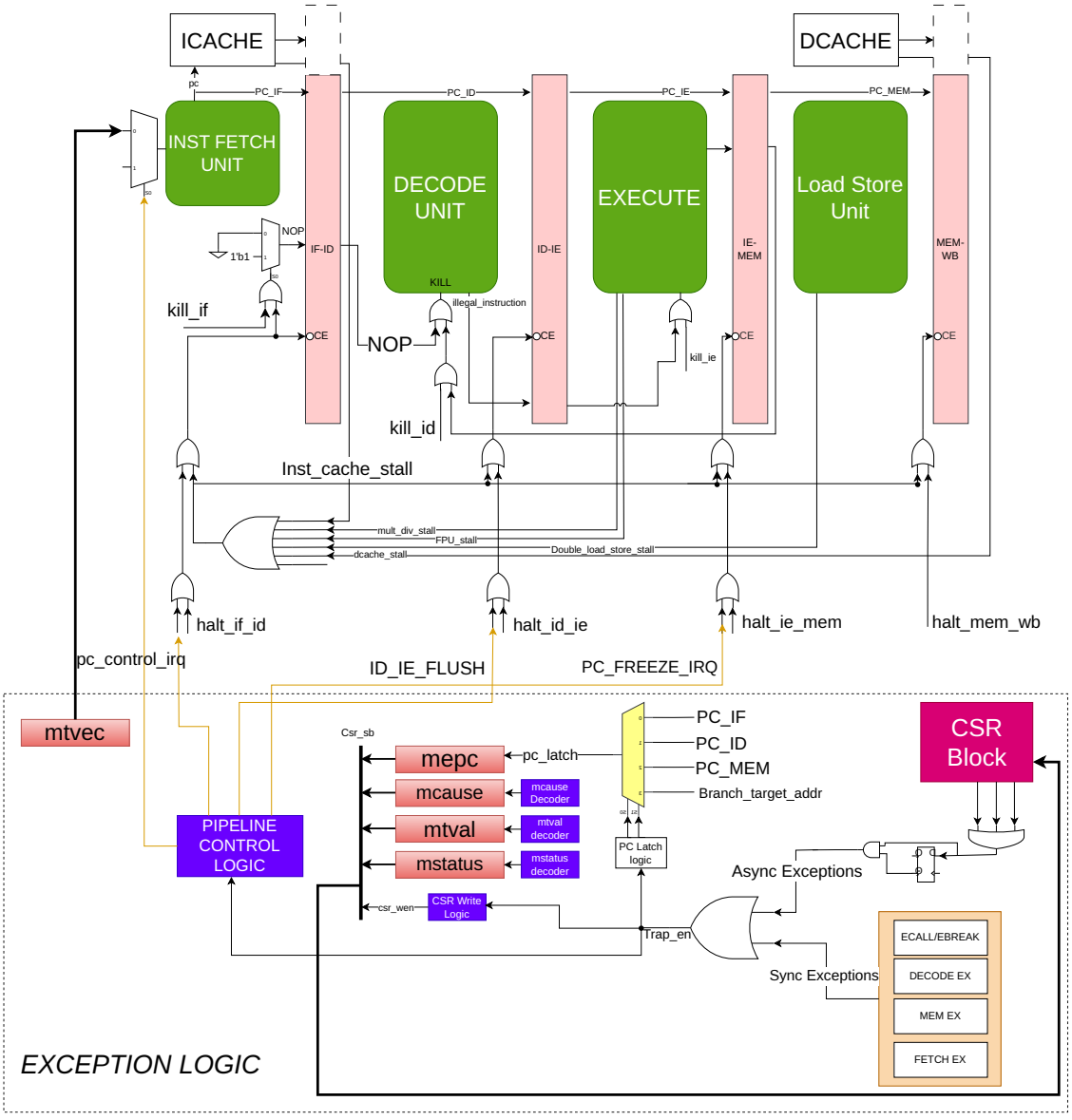

Figure 3.7: Exception Unit pipeline integration

3.5.6 CONTROL AND STATUS REGISTERS

3.5.7 Machine Mode CSR

The Control and Status Register (CSR) architecture in RISC-V, as defined in the RISC-V Privileged Specification[23], incorporates a structured address decoding mechanism to manage access permissions and privilege levels. Specifically, the upper four bits of the 12-bit CSR address field (csr[11:8]) are reserved for this purpose. The top two bits (csr[11:10]) determine the access type: values 00, 01, or 10 indicate that the register is read/write, while 11 designates a read-only register. The subsequent two bits (csr[9:8]) specify the minimum privilege level required to access the CSR—00 for User mode, 01 for Supervisor mode, 10 for Hypervisor mode (if supported), and 11 for Machine mode. This encoding ensures that each CSR is accessed only by the appropriate privilege level and with the correct permissions, thereby enhancing system security and maintaining controlled interaction between hardware and software. Table 3.6 shows the machine mode CSR implemented in our core.

Name	Address	Description
mstatus	0x300	Machine Status (lower 32 bits).
mie	0x304	Machine interrupt enable
mtvec	0x305	Machine Trap Vector address
mepc	0x341	Machine Exception Program Counter
mcause	0x342	Machine Trap Cause
mtval	0x343	Machine Trap Value
mip	0x344	Machine interrupt pending

Table 3.6: Machine mode CSR map

The following CSRs are essential for managing interrupts and exceptions in the RISC-V architecture: *mstatus* (0x300) holds the machine status, including global interrupt enable and privilege mode information; *mie* (0x304) allows enabling of specific interrupts; *mtvec* (0x305) contains the base address for the trap handler; *mepc* (0x341) stores the program counter at the time of an exception; *mcause* (0x342) indicates the cause of the trap; *mtval* (0x343) provides fault-specific data like the address or instruction that caused the exception; and *mip* (0x344) reflects which interrupts are currently pending. These CSRs are initialized by software immediately after the processor comes out of reset to configure the core's behavior in response to traps and interrupts.

3.5.8 Debug CSR

We have implemented the core debug registers (*dcsr*, *dpc*, *dscratch0*, and *dscratch1*) in accordance with the official RISC-V Debug Specification[24]. These registers are accessible only when the core enters Debug Mode and can be used through standard CSR instructions or abstract debug commands. Any attempt to access these registers outside Debug Mode, or on a core where they are not implemented, results in an illegal instruction exception. This ensures compliance with the RISC-V debug architecture and provides essential support for system-level debugging. Table 3.7 shows the address of debug CSRs.

Name	Address	Description
dcsr	0x7b0	Machine Status (lower 32 bits).
dpc	0x7b1	Machine interrupt enable
dscratch0	0x7b2	Machine Trap Vector address
dscratch1	0x7b3	Machine Exception Program Counter

Table 3.7: Debug CSRs

3.5.8.1 Debug Control and Status (dcsr, at 0x7b0)

Upon entry into Debug Mode, *v* and *prv* are updated with the privilege level the hart was previously in, and *cause* is updated with the reason for Debug Mode entry. Other than these fields and *nmip*, the other fields of *dcsr* are only writable by the external debugger. Since we only support the machine mode, *v* and *prv* will behardwired to 0. The figure 3.8 shows the format of DCSR register

Figure 3.8: DCSR CSR

During the debug mode entry, the cause of entry is written in the *dcsr[cause]* bit. The specification describes the priority between different debug entry sources.

3.5.8.2 Debug PC (dpc, at 0x7b1):

Upon entry to debug mode, *dpc* is updated with the virtual address of the next instruction to be executed. The writability of *dpc* follows the same rules as *mepc* as defined in the Privileged Spec. When we resume, the heart's PC is restored with the value in *dpc*. The figure 3.8 shows which address needs to be stored in DPC, while entering the debug mode.

Cause	dpc value
ebreak	Address of the ebreak instruction
single step	Address of the instruction that would be executed next if no debugging was going on. Ie. pc + 4 for 32-bit instructions that don't change program flow, the destination PC on taken jumps/branches, etc.
halt request	Address of the next instruction to be executed at the time that debug mode was entered

Table 3.8: Virtual address in DPC upon Debug Mode Entry

3.5.8.3 Debug Scratch Register 0/1 (dscratch0/1, at 0x7b2/0x7b3):

The *dscratch0* and *dscratch1* registers serve as temporary general-purpose registers that can be used by debugging tools or software during debug operations. Since the debugger may need to perform various operations without interfering with the normal state of the core, these scratch registers offer safe storage for intermediate values, calculations, or context data. They are especially useful for saving and restoring register values when single-stepping or inspecting memory and allow for flexible debug routines without corrupting the core's execution state.

3.5.9 PMP Configuration CSRs

In our design, the physical memory can be divided into a maximum of 16 configurable regions, allowing fine-grained control over memory access. Each region is defined using a PMPADDRx register, which stores the base or top address depending on the selected addressing mode. To specify access permissions and region attributes, each memory region is associated with an 8-bit configuration field that includes read, write, and execute permissions, the addressing mode,

and a lock bit to prevent further modification. Since each *pmpcfg* register holds 8 bits per region, a single register can manage configuration for up to 4 regions. Therefore, a total of 4 PMPCFG registers are implemented in our core to support all 16 regions. This setup enables flexible and secure memory partitioning aligned with the RISC-V privilege specification. The table 3.9 shows address map of PMPCFGx and PMPADDRy CSRs in our design.

Name	Address	Description
PMPCFG0	0x3A0	PMP configuration 1
PMPCFG1	0x3A1	PMP configuration 2
PMPCFG2	0x3A2	PMP configuration 3
PMPCFG3	0x3A3	PMP configuration 4
PMPADDR0	0x3B0	PMP address region 1
PMPADDR1	0x3B1	PMP address region 2
PMPADDR2	0x3B2	PMP address region 3
PMPADDR3	0x3B3	PMP address region 4
PMPADDR4	0x3B4	PMP address region 5
PMPADDR5	0x3B5	PMP address region 6
PMPADDR6	0x3B6	PMP address region 7
PMPADDR7	0x3B7	PMP address region 8
PMPADDR8	0x3B8	PMP address region 9
PMPADDR9	0x3B9	PMP address region 10
PMPADDR10	0x3BA	PMP address region 11
PMPADDR11	0x3BB	PMP address region 12
PMPADDR12	0x3BC	PMP address region 13
PMPADDR13	0x3BD	PMP address region 14
PMPADDR14	0x3BE	PMP address region 15
PMPADDR15	0x3BF	PMP address region 16

Table 3.9: PMP Configuration CSR

3.5.10 Custom CSR

We support cache flush feature in dcache. To flush the cache, write 1'b1 on this register. The table 3.10 shows the address of Cache_flush CSR in our design.

Name	Address	Description
cahce_flush	0x400	Cache flush

Table 3.10: Custom CSRs

3.6 Interrupt Subsystem

This core utilizes a CLINT-based interrupt architecture, where the irq_i[31:0] interrupt signals are managed through the *mstatus*, *mie*, and *mip* CSRs. The core designates the upper 16 bits (irq_i[31:16]) of the *mie* and *mip* registers for custom interrupt handling, reflecting an intentional extension to the standard RISC-V CLINT model. The figure 3.9 shows the interrupt platform integrated with the core.

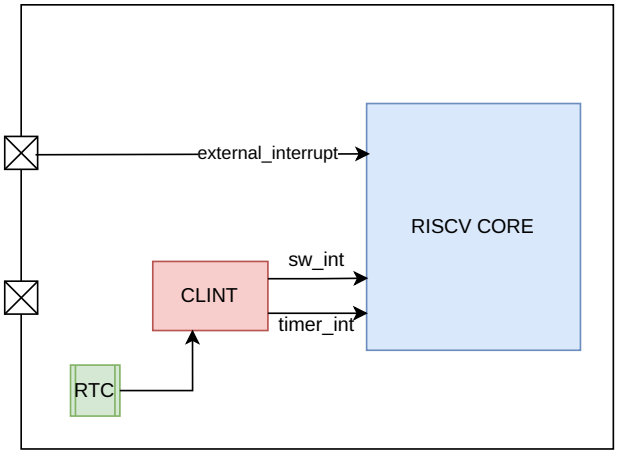

Figure 3.9: Interrupt Platform

For any *irq_i[31:0]* interrupt to become active, both the global interrupt enable (MIE) bit in mstatus and the corresponding individual enable bit in the mie register must be set.

The interrupts needs to be enabled first during the boot configuration. If the corresponding bit in *MIE* is 1, then if that interrupt occurs, it will make the *MIP* bit as 1 indicating that interrupt is pending and needs to be serviced. External interrupt bit is cleared once it is serviced, but software and timer bits needs to be cleared by writing zero in *MSIP* or *MTIMECMP* register. Figure 3.10 shows the machine interrupt enable (*MIE*) and machine interrupt pending (*MIP*) register.

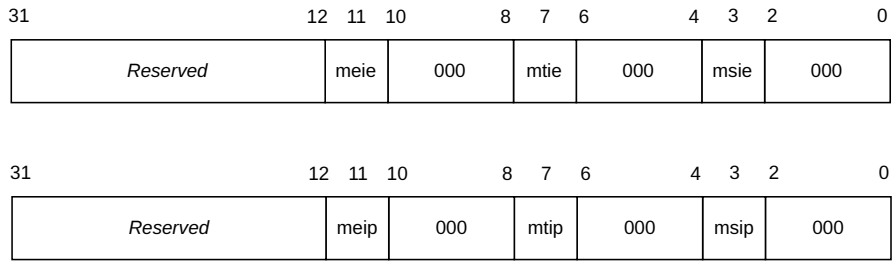

Figure 3.10: mie and mip CSR register

Table 3.11 describes the interrupt interface used for the CLINT mode interrupt architecture.

Signals	Direction	Description
irq_i[31:16]	Input	Active high, level sensistive interrupt inputs. Custom extension.
irq_i[15:12]	Input	Reserved. Tie to 0.
irq_i[11]	Input	Active high, level sensistive interrupt input. Referred to as Machine External Interrupt (MEI)
irq_i[10:8]	Input	Reserved. Tie to 0.
irq_i[7]	Input	Active high, level sensistive interrupt input. Referred to as Machine Timer Interrupt (MTI)
irq_i[6:4]	Input	Reserved. Tie to 0.
irq_i[3]	Input	Active high, level sensistive interrupt input. Referred to as Machine Software Interrupt (MSI)
irq_i[2:0]	Input	Reserved. Tie to 0.

Table 3.11: CLINT mode architecture interface signals

When one of the assigned interrupts occurs, the corresponding bit in the *mip* (Machine Interrupt Pending) register is set to indicate that the interrupt is pending. If the same bit is also set in the *mie* (Machine Interrupt Enable) register, it means that the interrupt is locally enabled. Only when both the *mip* and *mie* bits for a specific interrupt are set does the processor proceed to service that interrupt.

3.6.1 CLINT Controller

Core Local interrupt controller is responsible for handling the timer interrupts and software interrupts. There are three register which are memory mapped in the CLINT module. MSIP, MTIME and MTIMECMP. MSIP is of 32bit, MTIME and MTIMECMP register are of 64-bit each. These are asynchronous interrupts, as they are generated outside the core module. CLINT is a memory mapped device and needs to be written by the processor. Base address of these registers are mentioned in table 3.12.

CLINT Register	Address	Size
MSIP_BASE_ADDRESS	0x5F00_0000	32-bit
MTIME_BASE_ADDRESS	0x5F00_0004	64-bit
MTIMECMP_BASE_ADDRESS	0x5F00_000C	64-bit

Table 3.12: CLINT Base address

These register are configured by the software in the following manner:

Algorithm 3.2 Enable External Interrupt

```
/* Enable External interrupts */
1. MTVEC = <trap_handler_base_address>
2. MSTATUS = 0x0000_0008
3. MIE = 0x0000_0888
4. MTIME = 0x0001_8000
5. MTIMECMP = 0x0000_0888
6. MTIME = <Timer Interrupt>
```

MTIME stores the number of RTC clock values. If the RTC module is working on 32.768 Khz, then in 1sec it will store 32768 in *mtime* register. If we want an interrupt at every 3 sec, we need to initialise MTIMECMP with 3 left shifted by 15 (3*32768). This timer or external interrupt is converted into pulse and is given to the EXCEPTION UNIT. When these asynchronous expcetions occur, current PC is stored in the *mepc* and next PC is loaded as *mtvec + 4*offset*. This offset is fixed for machine Software, timer and external interrupts.

3.7 Memory Management Unit

The Sv32 scheme is the standard virtual memory translation mechanism for 32-bit RISC-V systems as shown in figure 3.11. It defines how virtual addresses (VAs) are translated to physical addresses (PAs) using a two-level page table system. This mechanism enables isolation and memory protection between different processes in a system. In Sv32, a 32-bit virtual address is divided into three parts:

- **VPN[1]** (bits 31–22): First level (root) page table index
- **VPN[0]** (bits 21–12): Second level page table index
- **Offset** (bits 11–0): Byte offset within the 4 KB page

Each page table contains 1024 (2^{10}) entries, and each Page Table Entry (PTE) is 32 bits. The page size is fixed at 4 KB.

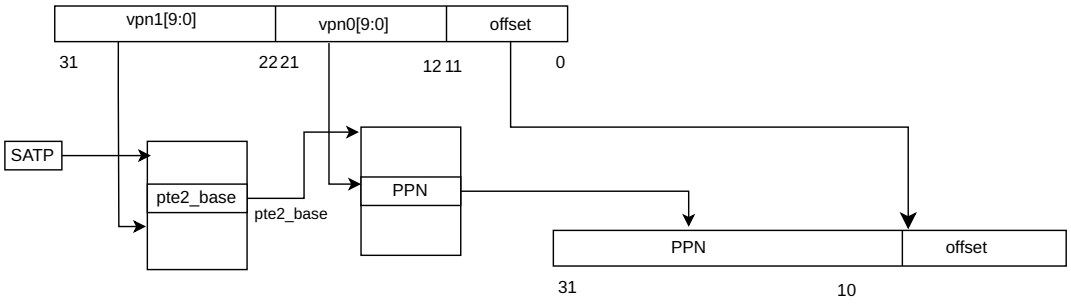

Figure 3.11: Sv32 memory translation

These translation is supported on both the side Instruction and Data. Virtual address is given by the core along with the translation request. If we get a TLB miss, then the page needs to be fetched from memory if it has valid PMP access.

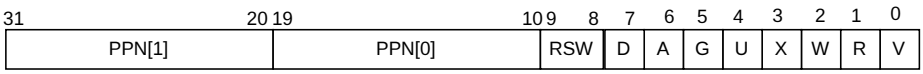

Figure 3.12: Sv32 page table entry

In the Sv32 virtual memory system, each Page Table Entry (PTE) follows the format shown in Figure 3.12 . The V bit (Valid) determines whether the PTE is considered active—if this

X	W	R	Meaning
0	0	0	Pointer to next Page Table
0	0	1	Read-Only page
0	1	0	*Reserved for future use*
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Write-execute page
1	1	1	Read-write-execute page

Table 3.13: Encoding of PTE R/W/X fields

bit is 0, the rest of the bits in the PTE are ignored by the hardware and can be repurposed by software. The R, W, and X bits represent access permissions, indicating whether the page is readable, writable, and executable, respectively. If all three permission bits are cleared (0), the PTE is treated as a pointer to a lower-level page table rather than a direct mapping to a physical page, identifying it as a non-leaf entry. Conversely, if any of the R, W, or X bits are set, the PTE is classified as a leaf entry and directly maps a virtual page to a physical page. The encoding of RWX bits is shown in figure 3.13.

Attempting to fetch an instruction from a page that does not have execute permissions raises a fetch **page-fault exception**. Attempting to execute a load or load-reserved instruction whose effective address lies within a page without read permissions raises a **load page-fault exception**. Attempting to execute a store, store-conditional, or AMO instruction whose effective address lies within a page without write permissions raises a **store page-fault exception.**

3.7.1 Page Table Walker

The two-level page table walker is a hardware mechanism that automates the translation process. When we get VPN (virtual page number) and translation request from the processor, corresponding translation is looked at the CAM (Content addresable memory), if there exsist a previous matching then the PPN is given out along with the **tlb_hit** signal. If we encounter the miss, then the mapping needs to looked in the page table located in the memory. Hence we need to do the page table walk. Page table walker is shown in the figure 3.13.

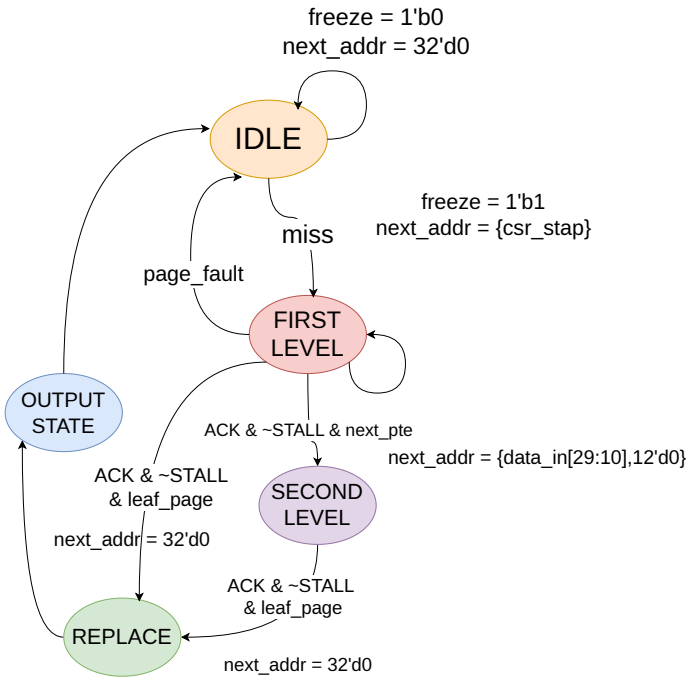

Figure 3.13: Page Table Walker

The translation process involves two stages, each corresponding to a level in the page table hierarchy.

Level 1 Translation

- Base Address Calculation: The base address of the level 1 page table is obtained from the satp register.
  - *satp.PPN* provides the base address of the level 1 page table.
  - The base address is shifted left by 12 bits to align with the page table entry size (4 KiB).
- Index Calculation: The index into the level 1 page table is derived from *vpn1*
  - vpn1[9:0] is used to index into the level 1 page table.
  - The address of the level 1 page table entry (PTE1) is calculated as: **pte1_address** = satp.PPN $<<$ 12 | vpn1[9:0] $<<$ 2
- PTE1 Access: The level 1 page table entry (PTE1) is accessed at the calculated address.

- PTE1 contains the base address of the corresponding level 2 page table.

Level 2 Translation

- Base Address Calculation: The base address of the level 2 page table is obtained from PTE1 if that is not the leaf entry.
  - The base address is shifted left by 12 bits to align with the page table entry size (4 KiB).
- Index Calculation: The index into the level 2 page table is derived from *vpn0*.
  - vpn0[9:0] is used to index into the level 2 page table.
  - The address of the level 2 page table entry (PTE2) is calculated as: **pte2_address** = pte1.PPN $<<$ 12 | vpn0[9:0] $<<$ 2
- PTE2 Access: The level 2 page table entry (PTE2) is accessed at the calculated address.
  - PTE2 contains the physical page number (PPN) and other metadata (e.g., permissions, accessed bit, dirty bit).

Final Physical Address Calculation

- PPN Extraction: The PPN is extracted from PTE2.
  - The PPN is combined with the offset to form the final physical address.
  - The final physical address is calculated as: **physical_address** = pte2.PPN $<<$ 12 | offset

If either PTE1 or PTE2 is invalid (i.e., the V bit is 0), a page fault exception is raised. The exception handler can then take appropriate action, such as loading the missing page table entry or handling the fault. The permissions and access control are checked during the translation process:

3.7.2 VIPT cache

Since translation process requires fetching of page from the main memory and sometimes from DRAM (in case of OS), it can slow down the process of memory access. Hence to avoid that, we are using VIPT caches both for instruction and data memory. In VIPT caches, the index

and block offset bits (12-bits in our case) are used to fetch cache line and parallely virtual_tag is given to the TLB for physical mapping (or physical tag). Once we get this *physical_tag*, it is compared against the *tag_out* from the cache memory to check if it's a hit or miss. The figure 3.14 shows how the VIPT caches are instantiated.

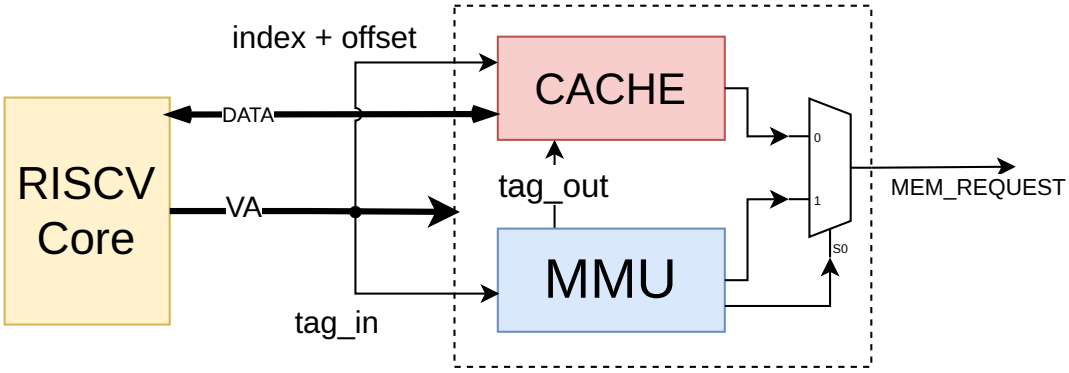

Figure 3.14: VIPT cache design

VIPT (Virtually Indexed, Physically Tagged) caches are a type of cache memory organization that combines the benefits of both virtual and physical caching strategies. In a VIPT cache, the cache is indexed using the virtual address, which allows for faster cache access since the translation from virtual to physical address is not required at cache lookup time. This can significantly reduce the latency associated with cache accesses, especially in systems where address translation is complex or involves multiple stages, such as in virtualized environments or systems with large page tables.

The cache lines in a VIPT cache are tagged with the physical address, which ensures that the cache coherence and consistency are maintained across different processes and address spaces. This tagging mechanism helps in avoiding conflicts and false sharing between different virtual addresses that may map to the same physical address, thereby improving the overall cache performance and reducing cache thrashing.

3.7.3 ITLB

The page table walker helps in the translation from virtual to physical address which then will be used for accessing the cache. The diagram 3.15 shows the complete ITLB design.

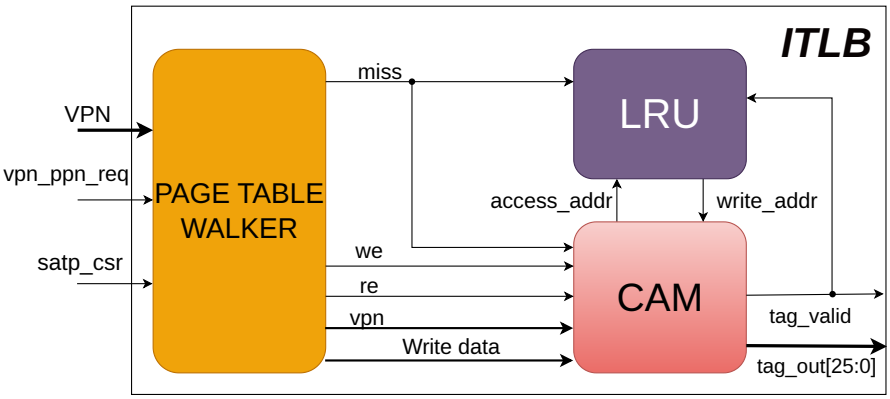

Figure 3.15: ITLB

Here if there is a miss from the CAM, the translation process starts and the processor remains stalled for that duration. Parallely $<index+blk\_offset>$ are used to access the cache line. When the translation completes we have the correponding physical address, and the mapping of VPN to PPN is written in the CAM which is also known as Translation Lookaside buffer (TLB). ***tag_out*** and ***tag_valid*** are given to the ICACHE which signifies the corresponding physical tag is valid.

There are total of 32 entries in the TLB, each entry of 52 bit (20 bits of VPN and 32 bit of PTE).

If all the entries of TLB are filled then the entry which is least recently accessed gets evicted with the help of LRU block.

3.7.4 DTLB

The structure of DTLB is similar to that of ITLB except that there will be 2 ports for accessing the data memory. One port is used by the LSU (Load store unit) for conventional Load/Store operation and another port is used by the decode stage for ATOMIC operation. In atomic operations, memory need to be read for flags or lock bit and it needs to be stored in the register before executing any other instructions. The figure 3.16 shows how the DTLB is integrated with the page table walker for fetching the virtual to physical mappings.

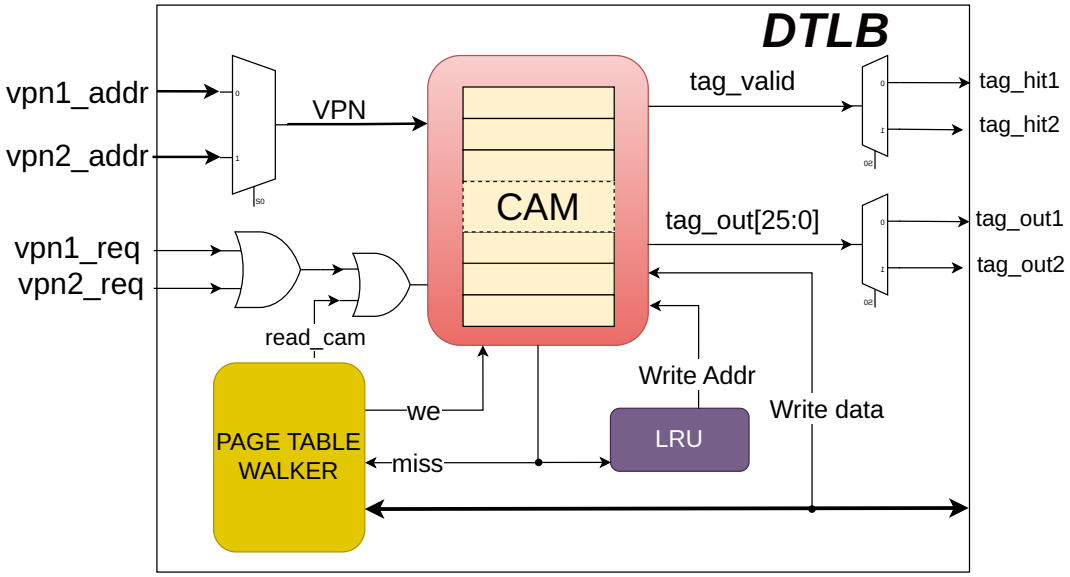

Figure 3.16: DTLB

3.8 ICACHE Memory Subsystem

The Instruction memory subsystem consists of **Instruction Cache module** (I-Cache and Instruction TLB (ITLB)) and the **Instruction Cache controller** as shown in the figure 3.17. Virtual Address is used for accessing the ICACHE. In our design we have Virtually indexed and Physically tagged cache i.e Virtual Index and Offset (PC_ADDR[11:0]) is used for accessing the index and the corresponding word in that Block. Parallely Virtual Tags are used to access the TLB that translates this Virtual Tag into Physical Tag. This Tag coming from the TLB is then comapared with the TAG coming from the CACHE Array to check whether it's a Cache hit or miss. If the Tag matches then it's a cache hit and corresponding Instruction is given to the processor pipeline.

The I-cache controller manages cache read and write. If it is a cache hit, the requested instruction is given to the fetch stage. If it is a cache miss, a write bus transaction is initiated on the memory bus unit. The page offset from the virtual address and the physical tag bits from the TLB form the address of the instruction line to be fetched and given to the memory interface. The figure 3.17 shows the Instruction cache subsystem.

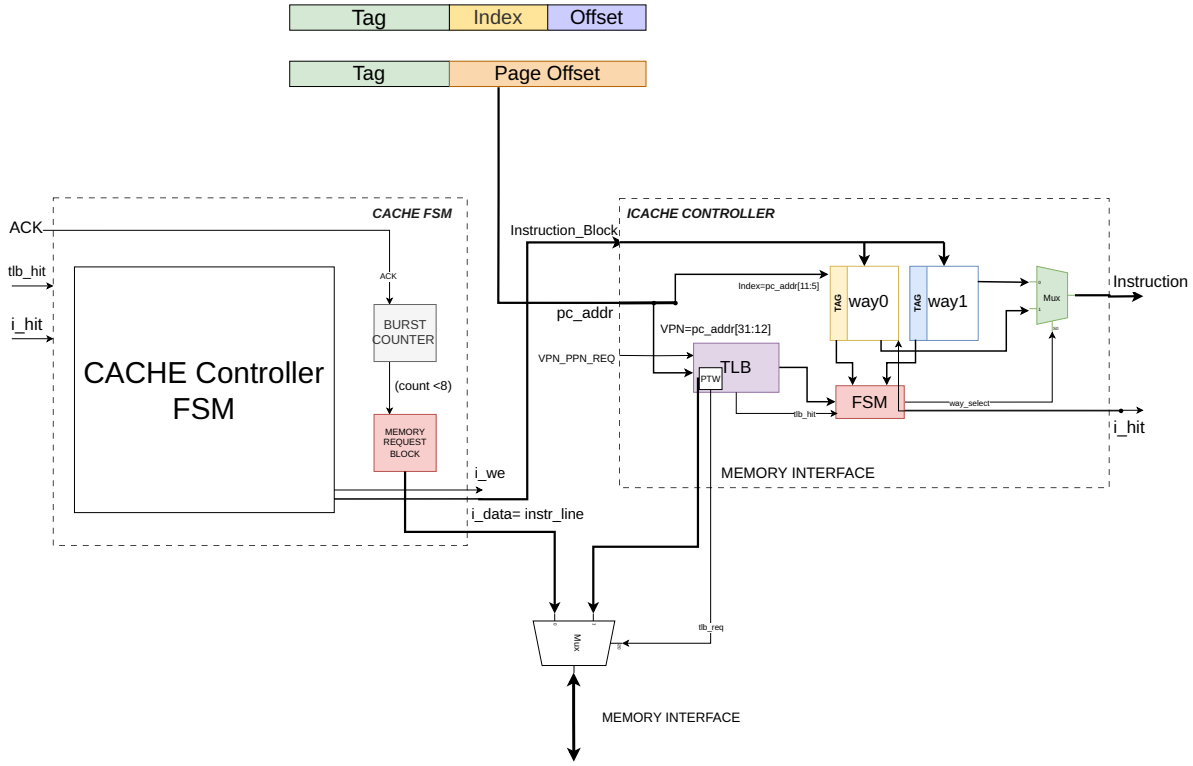

Figure 3.17: Instruction Cache Subsystem

The I-Cache generates instruction access fault and instruction page fault exceptions. Access fault exception is generated when the PMP and PMA checks on the physical address fail. Instruction page fault is generated by the TLB during page table walk when an invalid page table entry is encountered, or the page does not have execute permission. The exceptions generated by the I-Cache are carried along the pipeline.

3.8.1 Instruction Cache module

This module consists of ICACHE SRAM macros, ITLB and FSM, both of them work in parallel fashion as shown in the figure below.

The ICACHE is 8KB, 2-Way Set associative with block size of 32Bytes (8-Instruction). The I-TLB is a content Addressable memory with 32 entries.

There are 128 sets in the cache array, each of which has a 32Bytes of Block. Out of 32-bit of Virtual Address (PC_ADDR[31:0]), last 12 bits are used for accessing the Cache array of which upper 7 bits (PC_ADDR[11:5]) are used to address into cache array using Sets and last 5 bits (PC_ADDR[4:0]) are required to fetch the corresponsding byte in the Block. Since the

instructions are aligned on 32-bit word boundary, last two bits of Block offset (PC_ADDR[1:0]) has to be 0. Configuration of ICACHE is given in table 3.14 and figure 3.18.

ICACHE Parameter	Value	Description
Total size	8KB	Total icache size is of 8KB
No of ways	2	Each way is of 4KB
Line size	32 Bytes	8 words per cache line
No of index	128	

Table 3.14: Icache organisation

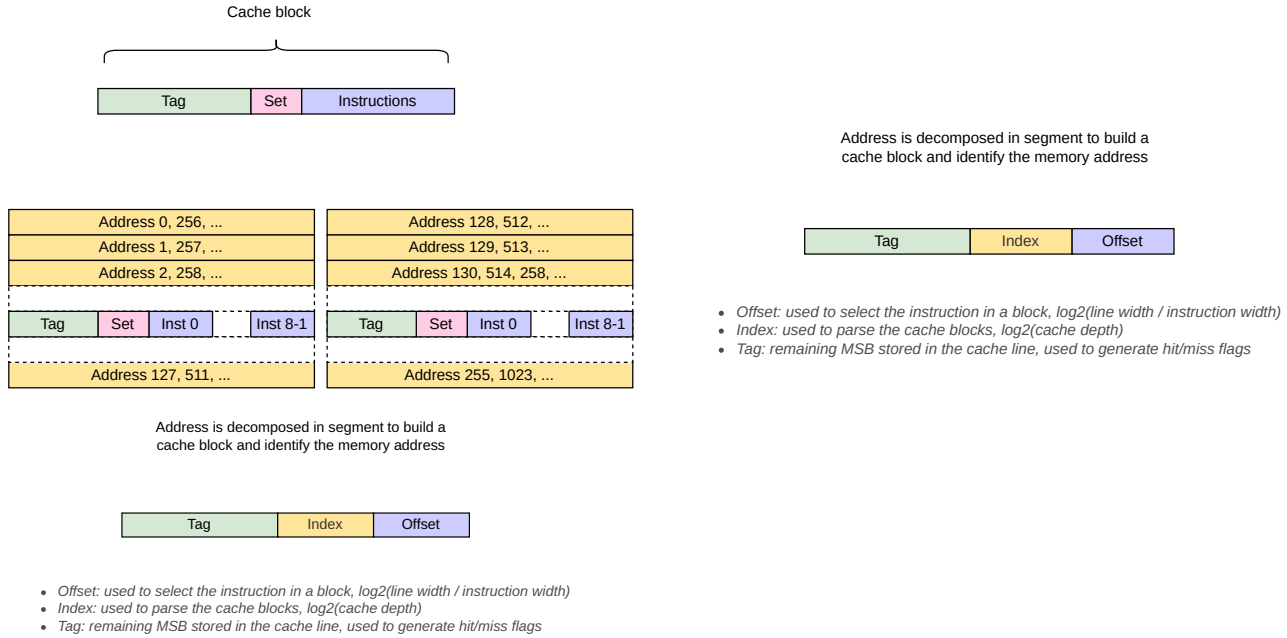

Figure 3.18: Icache Organisation

Cache line consists of 20-bits of Tag and 128-bits of Block. Tag array and Block array are designed separately, each of which is accessed by the Virtual index. When there's a TLB hit, in the next cycle Physical TAG from TLB is compared against the TAG coming out from the array. If they matches then we have the cache hit otherwise miss. If the cache hits then the corresponsing instruction is selected based on the block offset bits. If the cache misses, then corresponsding Instruction needs to be fetched from the Main memory. This process is taken care of by the *Cache controller* block. The figure 3.19 shows the structure of Instruction cache module.

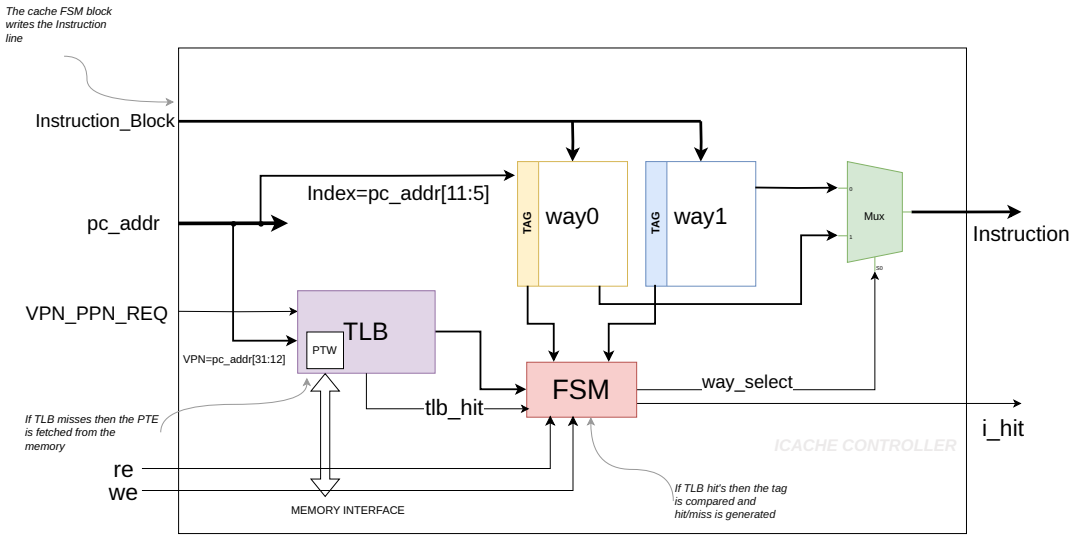

Figure 3.19: Instruction Cache module

3.8.1.1 Icache FSM

This FSM controls the writing of the instruction cache line in any one of they way and corresponding tag in the TAG macro during the cache filling stage.

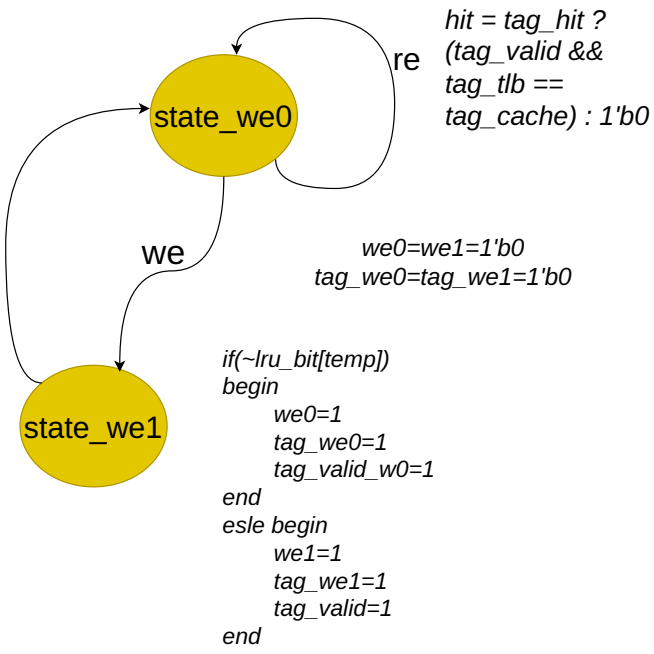

Figure 3.20: Icache FSM

Initially we are in the state_we0 state, when the re signals comes from the cache controller indicating we have the read request from the processor. If we encounter tlb_hit, the tag is compared with the tag from the TAG array. If it matches it is a hit and the corresponding instruction is given out. If we encounter a miss, then we need to wait for the cache controller to service the miss. When the complete cache line (32Bytes) is received, we get the we signal and we move on to state_we1 state in next cycle. Then we decide on which way we need to write this instruction and tag line based on the LRU bit. Once we write the cache, it again goes to state_we0 in next cycle. Hit goes to 1 and the instructions are supplied back to the processor.

Instruction_miss_latency = 8cycle/instruction, Instruction_hit_latency = 1cycle/instruction

3.8.2 Cache FSM Module

When the TLB hit's and after comparision with Tag from the Tag Array, cache_hit signal is generated. If the cache_hit is 1'b1, then the corresponding Instruction is given out to the Pipeline Stage (IF). The below figure 3.21 shows the design of Icache controller module.

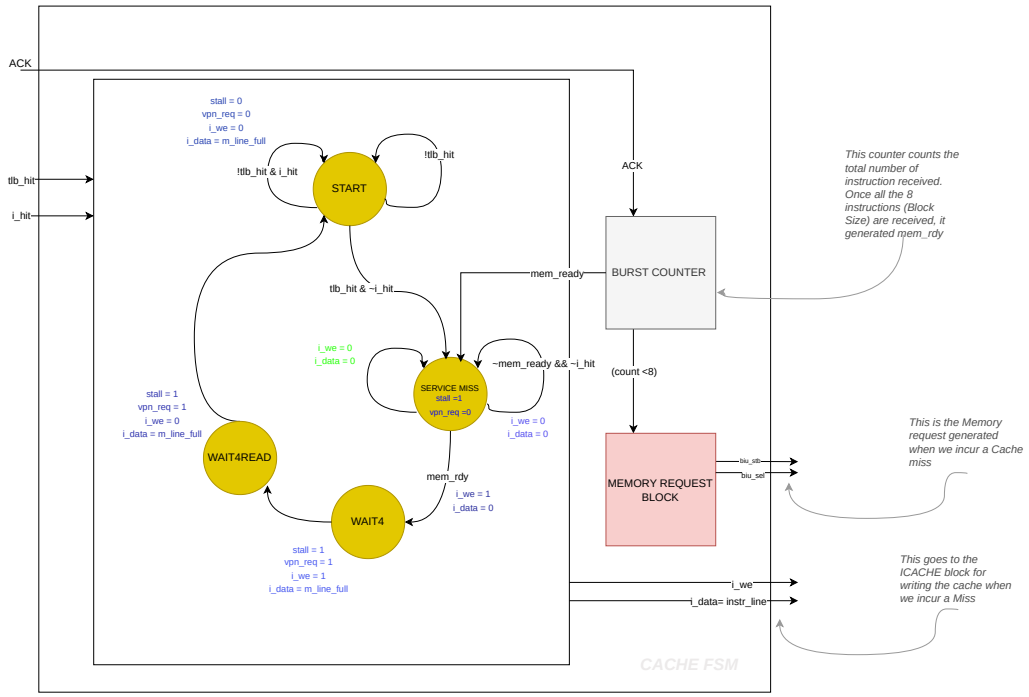

Figure 3.21: Icache controller module

The I-cache controller manages cache read and write. If it is a cache hit, the whole instruction line read is given to the fetch stage. If it is a cache miss, a write bus transaction is initiated

based on memory request unit. If we have a cache miss, then we service the miss by fetching 8 consecutive instructions from the memory through burst and in next cycle we write the whole line in the cache array.

In WAIT4 and WAIT4READ state, *we* is high and Icache FSM writes the cache line in the corresponding way. In the next cycle we get a cache hit and the instruction is supplied back to the processor.

3.8.3 Icache protocol

Icache timing protocol is shown in figure 3.22 . The process is shown after the TLB hits and we have the PPN out from the TLB

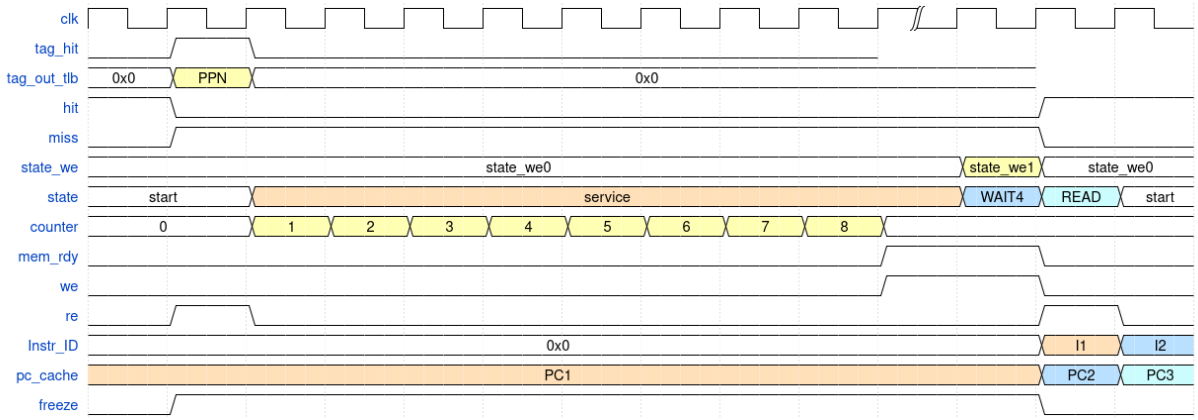

Figure 3.22: Icache miss protocol

The processor boots from the address 32'h0000_0000, which is the base address of the instruction memory (BOOT ROM) in our case.

When the first time processor sends the BASE_ADDRESS, there is a icache miss. Then the virtual address is given to the TLB block and physical address is given out with a tag_valid signal. In our case, since we have disabled the translation for ease of use, we'll have physical address = virtual address. When the *tag_hit* is 1'b1 and *hit_out* is 1'b0, then the state machine inside the **cachefsm** block goes to the service miss state(2'b01).

In the state SERVICE_MISS, the physical address is given to the instruction memory for fetching the instruction in burst. We have a cache line of 256 bits, that is 8 words (32-bits).

When the whole cache line is fetched, the cachefsm signals *we* to the icache controller for writing the cache line in to the memory. Physical tag along with the *tag_valid* is also written to

the tag array memory. When *we* comes from the cachefsm, the FSM inside the icache controller state *state_we* goes to 2 (2'b10). In state 2, it cache line and tag is written on to the ICACE SRAM, and in next cycle it again comes to state 1.

Parallely, states of cachefsm are also moving. When the state of cachefsm is 2'b10, it raises the *vpn_to_ppn_req_3* signal, which goes to the icache controller and the state goes to 0.

This *vpn_to_ppn_req_3*, is used to read the tag again from the TLB and compare it with the tag coming from the TAG SRAM ARRAY. It it matches, then it is a hit otherwise miss.

When the cache hits, it sends the *instruction[31:0]* to the pipeline unit using block select counter.

Now then the PC goes to 32'h0000_0020, the index increments and we encounter a cache miss, since the tag from the TLB(physical) doesnt matches with that from the memory. So signal i_hit goes down. Now the state in cachefsm again goes to SERVICE_MISS. At this moment, the tag_valid[VA] for the next upcoming way is written 1'b1 and cache again fetched the next line from the memory.

The figure 3.23 shows the timing waveform of the Icache module when it first encounters the cache miss.

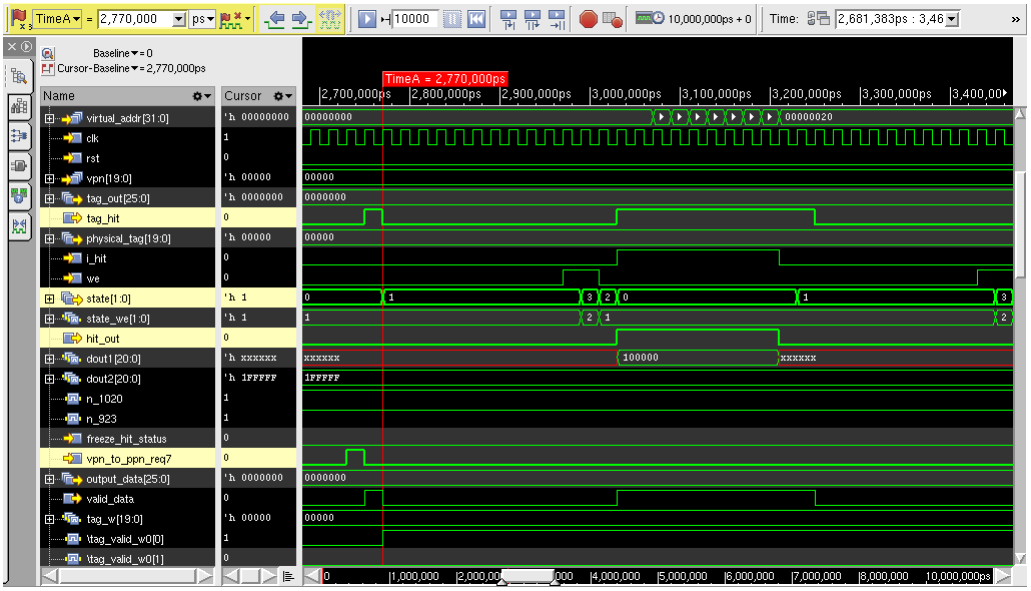

Figure 3.23: Initial icache miss

The valid bit for each cache line is designed as a register since the SRAMs are initialised to unknown state, and initial tag comparision results in an unknown state of a processor. Hence tag valid bits are designed as flop memories and tag array is the TSMC SRAM array.

3.9 DATA CACHE

3.9.1 Data Cache pipeline integration

The figure 3.24 shows the integration of DCACHE module with the Processor pipeline.

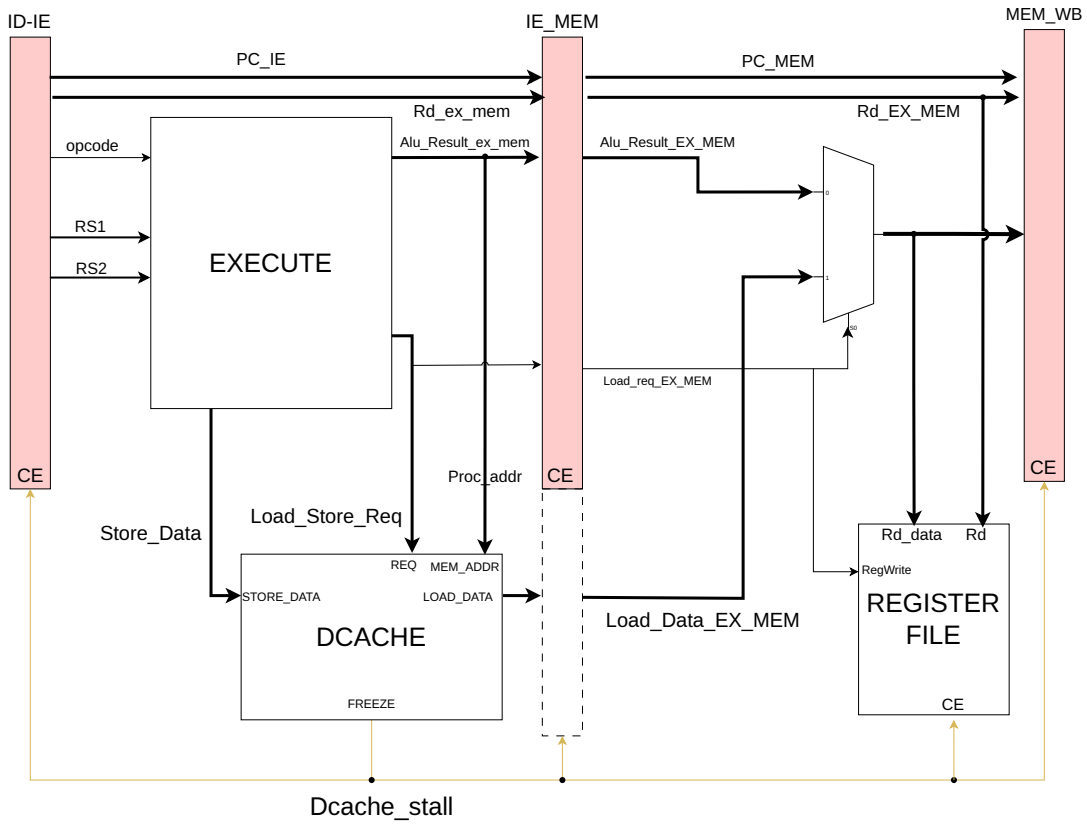

Figure 3.24: Dcache pipeline integration

When the load or store instruction enters the execute block, it generates the processor address and *Load_Store_req*. The request is directly given to the dcache so as to get the memory data in next cycle (if hit) and pipeline can work without any stall. If there is a cache miss in next cycle, *dcache_stall* goes high in same cycle as miss and the pipeline will be stalled and the corresponding memory instruction enters the MEM stage. Miss will be serviced by fetching eight words from the Main memory and cache line is written when the whole line is available. On servicing the miss, the require data is supplied first (Criticle word first) to the processor. When the miss completes, the *dcache_stall* is deasserted and normal pipeline operations resumes. The data is written in the register file in the write back stage.

DTLB is also integrated inside the DCACHE module and works in same fashion as ITLB. If we encounter a TLB_MISS, then the required mapping is fetched from the main memory with the help of 2 level page table walker. Once the TLB_HIT is 1, corresponding cache hit/miss is calculated based on tag comparision and valid bit.

3.9.2 Dcache organisation

The data memory subsystem consists of two modules, ***dcache_fsm*** and ***dcache_dpram.*** *Dcache_fsm* is responsible for generating the memory request if we encounter a cache miss and deliver the Load data to the processor if we get a Load hit. Address must be aligned to 32-bit word bounday, hence Load/Store misaligned exception is raised if we get a misaligned address. DCACHE and DTLB works in parallel fashion (VIPT), where virtual index is used for indexing the memory and Physical tag from the TLB is used for checking the hit condition. The Dcache design is shows in figure 3.25

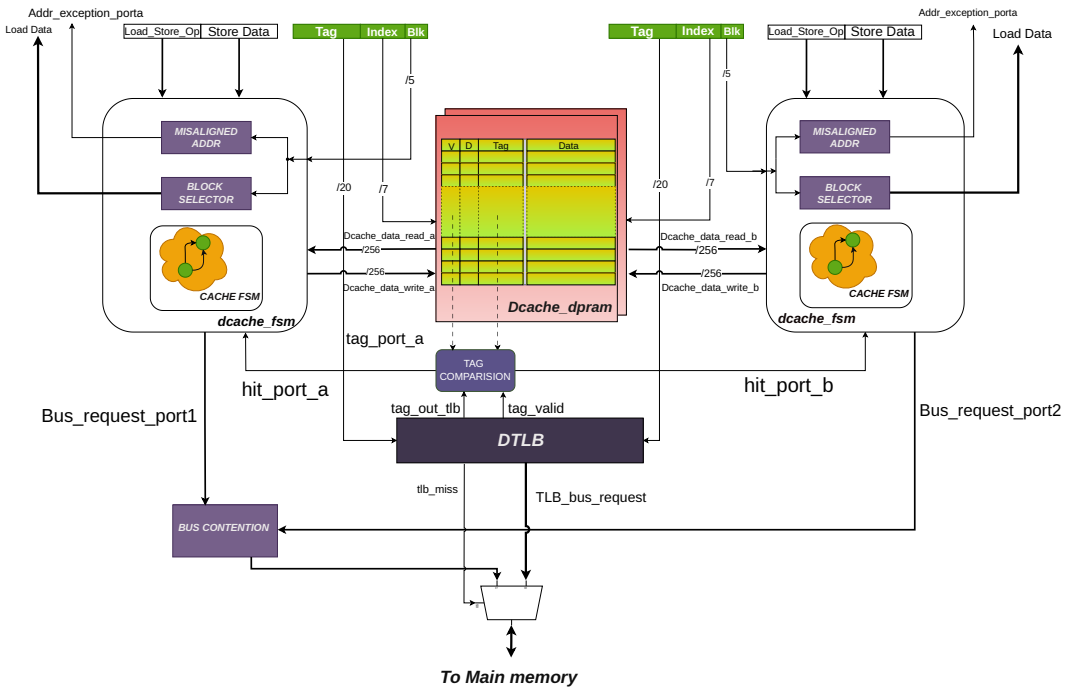

Figure 3.25: Dcache design

The D-Cache is 8KB, 2 Way set-associative cache with block size of 32-bytes. The D-TLB is a Content Addressable Memory with 32-entries. It supports simultaneous read and write requests and has two separate ports, one port is controlled by the Load-Store unit for conventional Load/Store or peripheral accesses and another port is for accessing the memory during

AMO operations. Since both the request cannot occur simultaneously, we'll not experience a Structural hazard.

The D-Cache follows the write-back policy, i.e., the data is written only to the cache and not to the memory during stores. Only when this block needs to be replaced by a new block, the data is written back to memory. The D-cache supports word, half-word, and byte data type with aligned memory access. Data write is byte-level enabled. In the case of data read, the cache line of the cache way which is hit passes through the Block selector, and the selected bytes are given at output. In the case of data write, the input bytes are stored in the Write buffer and are written to the cache way which is hit.

The **dcache_fsm** handles the Load/Store/AMO request coming from the processor pipeline. It first performs the PMA checks in DTLB, whether correct R/W/X permissions are there. If the processor tries to write on a Read-Only page region or if we encounter a page fault, *Page fault exception* is raised. The physical address generated in the next cycle of request, along with the index and block is checked for the PMP permissions. If the *dmem_disallow* goes high, then the corresponding exception is raised. For example if we want to attempt a store in Read-only region, store fault is raised.

This Dcache supports the **cache flush** feature. Since it has write back caches, the store wont be visible to the main memory. Hence caches can be flushed using the custom CSR (Software). All the cache line with dirty bit gets written to the main memory.

TSMC Cut name	Instance Name	Size(words * bits)	Description
TSDN65LPLLA128X128M4F	ram_w0_1	128 X 128	Way0 Cache array-bank1
	ram_w0_2	128 X 128	Way0 Cache array-bank2
	ram_w1_1	128 X 128	Way1 Cache array-bank1
	ram_w1_2	128 X 128	Way1 Cache array-bank2
TSDN65LPLLA128X32M8F	tag_w0, tag_w1	128 X 32	Way0 tag Array ,Way1 tag Array

Table 3.15: TSMC Memory cuts

For the ASIC design, we need to replace all the BRAMs with the memory cuts from the foundary for that particular technology node. We are working on 65nm. The table 3.15 shows the memory cuts we are using in this design.

3.9.3 Data Cache FSM

Dcache controller FSM design is shown in the figure 3.26 .

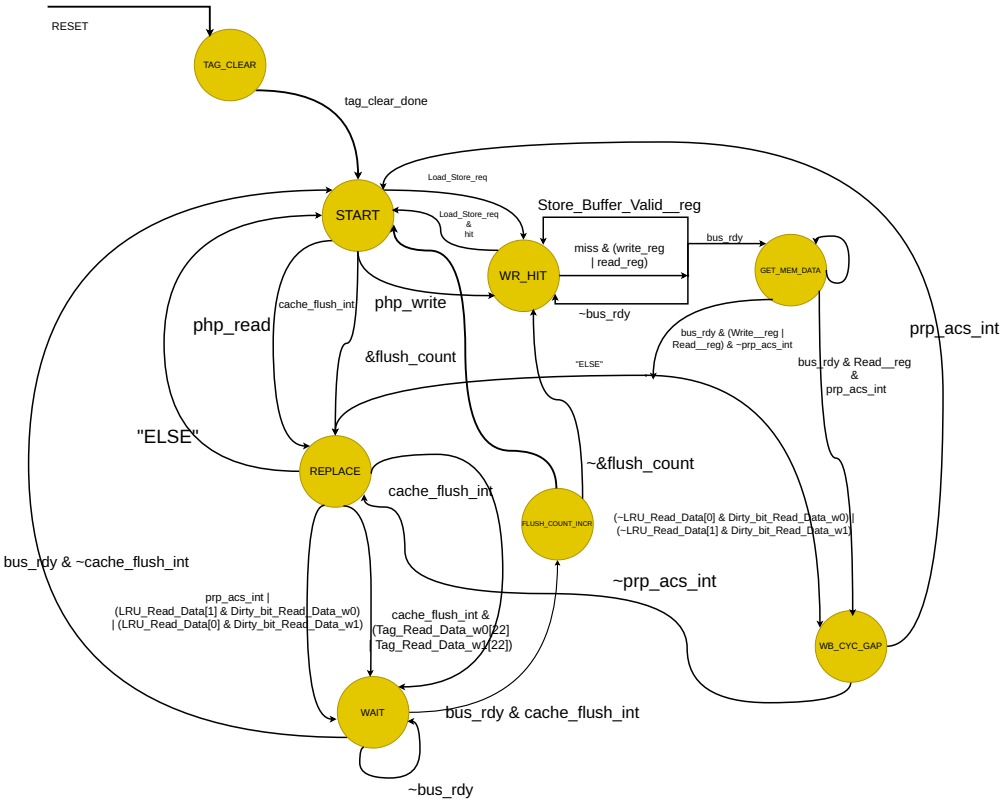

Figure 3.26: Dcache FSM

On reset, the initial state entered is **TAG_CLEAR** when the system is powered on or reset. In this state, all the cache line is invalidated before begining any cache operation. This is needed specifically in ASIC designs, where the intial state of memory is not defined (X). If we reset (Invalidate) all the cache line, then we can directly start the cache process.

1. **START**: The START state serves as the idle and decision-making state where the FSM waits for incoming memory access requests. If a ***Load_Store_req*** is detected, the controller goes to the WT_HIT state to complete the operation quickly. If *cache_flush_int* is asserted, then it goes to the **REPLACE** state.

2. **WT_HIT**: In the WT_HIT state, the cache controller checks whether there is a hit or a miss. If it is a read hit, the load data is written back to the processor, if it is a store operation it updates the appropriate cache line with the new data. The data may also be buffered using a store buffer if **Store_Buffer_Valid__reg** is set. This state allows for fast,

low-latency completion of memory write operations without accessing main memory. If it is a cache miss, the state goes to the **GET_MEM_DATA**.

3. **GET_MEM_DATA**: The GET_MEM_DATA state is responsible for handling cache misses by initiating memory access to retrieve the requested data. The controller requests data from the memory subsystem and waits for the *bus_rdy* signal to indicate that the bus is ready to complete the transaction. If the bus is not ready, the FSM remains in **GET_MEM_DATA** state. Otherwise, it proceeds to **REPLACE** state to replace a cache line. If the current cache line is dirty, or if we have read request then it goes to **WB_CYC_GAP** to accomade that one cycle delay.

4. **REPLACE**: The REPLACE state is entered when a cache miss occurs and the existing cache line that will be replaced is dirty and needs to be written back. If there are no dirty lines, the cache line fetched is directly written on to the cache and goes to **START** state. The controller identifies the LRU line and evaluates its dirty status before replacement. If it is dirty, it goes to **WAIT** state for evicting the cache line back to the main memory. If *cache_flush_int* is asserted, it goes to state **WAIT** and writes the dirty line back to memory before bringing in the new data. This state ensures that no modified data is lost during the replacement process.

5. **WAIT**: The WAIT state acts as a holding state when the memory bus is not ready (˜bus_rdy). In this state the cache line is written back to the main memory. This can happen in 2 scenarios, if the current cache line is dirty and it needs to be evicted, or if encounter *cache_flush_int,* then the dirty line has to written back to the memory. This ensures that the FSM does not proceed with memory operations until it can safely complete them. Once *bus_rdy* goes high, the FSM transitions back to **START** state to mark the end of cache process, or to **FLUSH_COUNT_INCR** if a cache flush is underway. This state helps coordinate timing between the cache controller and the memory system for reliable data transfers.

6. **WB_CYC_GAP:** The WB_CYC_GAP (Write-Back Cycle Gap) state introduces timing separation between memory write-back cycles. It allows the FSM to manage multiple pending write-back operations in a coordinated manner. This state can be used to align cache operations with bus cycles or to satisfy bus arbitration requirements. Once the write-back operation is appropriately spaced or completed, the FSM continues to the **REPLACE** state.

7. **FLUSH_COUNT_INCR**: The FLUSH_COUNT_INCR state is used during a cache flush

operation to increment a counter that tracks how many cache lines have been flushed. This is necessary when a full cache flush is requested via Cache flush CSR. The controller loops through cache lines one by one, ensuring that dirty lines are written back and valid bits are cleared. Once all lines have been processed, the FSM transitions back to START.

8. **TAG_CLEAR**: The TAG_CLEAR state handles invalidation of cache tags, typically as part of a full cache flush. In this state, the controller clears the valid bits or tag fields to ensure all cached entries are marked as invalid. This is crucial for maintaining coherency and ensuring that stale data is not used in future operations. If all tag entries are not yet cleared, the FSM moves to FLUSH_COUNT_INCR; otherwise, it returns to the START state when tag_clear_done is asserted.

3.9.4 Dcache miss protocol

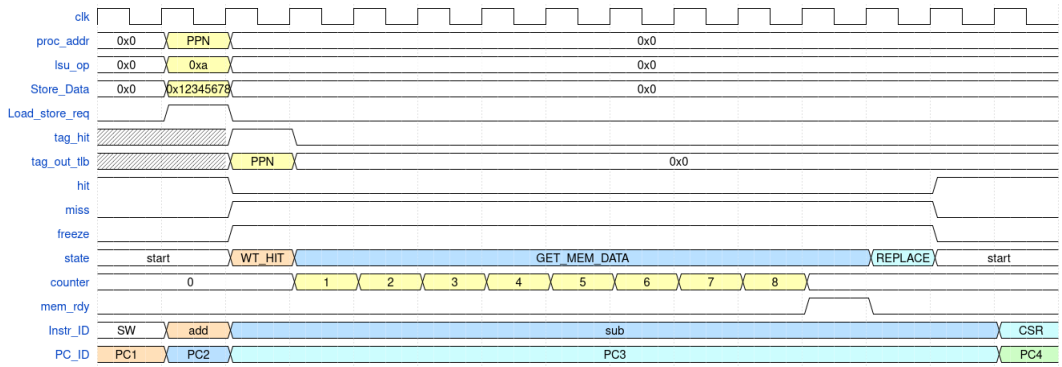

Figure 3.27: Dcache miss protocol

The figure 3.27 illustrates the behavior of the data cache (dcache) controller when servicing a cache miss. During the EXECUTE stage, the processor places the physical address (proc_addr), the store operation code (lsu_op), and the data to be written (Store_Data) onto the appropriate lines. In the subsequent cycle, the Physical Page Number (PPN) is available from the TLB, and a tag comparison is initiated. The FSM transitions into the WR_HIT state to perform a tag lookup. However, since there is no match (*tag_hit* is low), the controller detects a miss, and the pipeline is immediately stalled (freeze is asserted).

To service the miss, the FSM transitions to the GET_MEM_DATA state, where it begins retrieving the missing cache line from main memory. A counter increments with each word fetched, as shown in the waveform. Once the entire line is retrieved and *mem_rdy* goes high again, the

FSM enters the REPLACE state to write the fetched cache line into the cache. After this, the FSM returns to the START state, allowing the pipeline to resume execution.

In the case of a store hit, the write operation is directed to the store buffer, allowing the processor to proceed without stalling. However, in a store miss, the pipeline stalls, and the FSM follows the same miss-handling sequence described above. Notably, the critical word (the one initially requested) is forwarded to the processor as soon as it is fetched from memory, rather than waiting for the entire line, improving perceived memory latency.

3.9.5 Dcache hit protocol

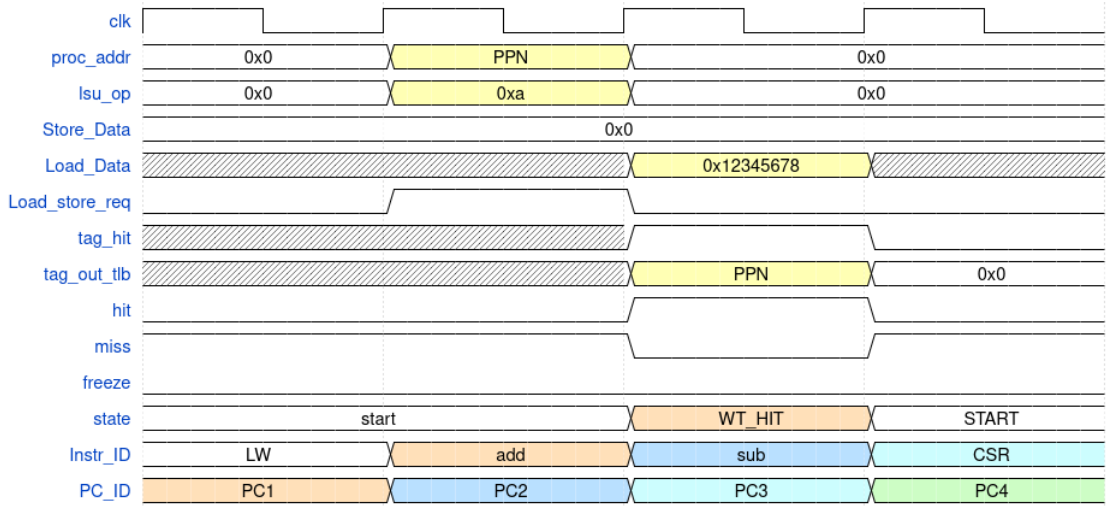

Figure 3.28: Dcache hit protocol

The waveform 3.28 depicts the behavior of the data cache (dcache) controller during a read hit. At the beginning of the EXECUTE stage, the processor asserts the physical address (*proc_addr*), load/store operation code (*lsu_op*), and initiates a memory access through the *Load_store_req* signal. In the following cycle, the Physical Page Number (PPN) is available via the TLB (*tag_out_tlb*), and a tag comparison is performed using the tag from the tag array.

Since the tag matches, a hit is registered (*tag_hit* is high and *miss* is low), and the FSM transitions from the start state to WR_HIT. During this state, the requested data is directly fetched from the cache and placed on the Load_Data line (e.g., 0x12345678), enabling fast, single-cycle data retrieval. The pipeline proceeds without any stall (freeze remains low), and the FSM returns to the START state in the next cycle to handle subsequent memory requests.

This efficient hit-handling mechanism enables the load instruction (LW) to retire quickly while

allowing the pipeline to continue executing subsequent instructions (add, sub, CSR) without delay, thereby maximizing instruction throughput.

3.9.6 Cache flush mechanism

The layout of DCACHE as shown in the figure 3.29 . LRU and Dirty bit memory is separate from the DCACHE and TAG array. Actual tag is of 20 bits, along with Valid bit (22) and Write bit (21).

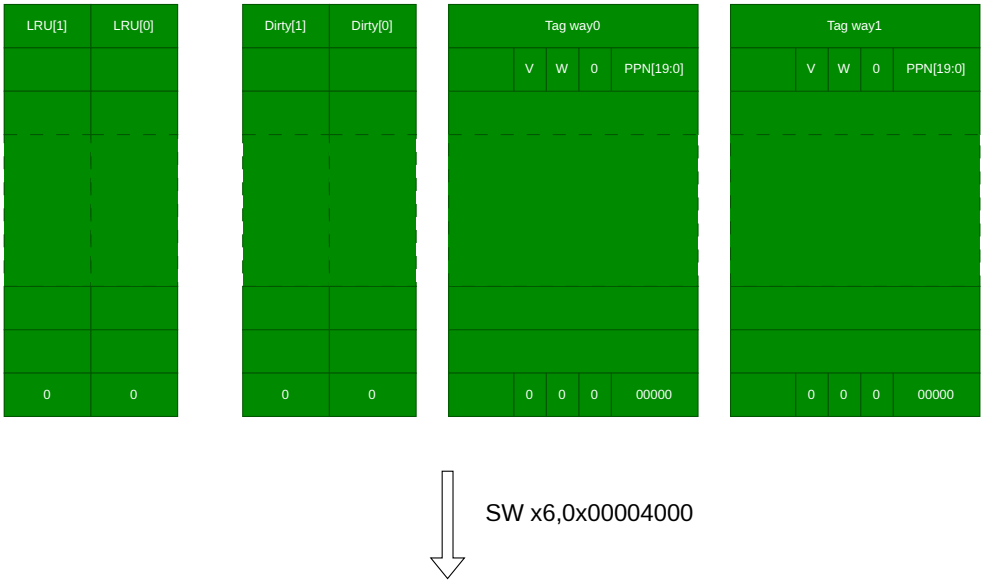

Figure 3.29: Dcache layout

LRU bit 0 means that cache line is next to be filled it misses on that index. Since it is a write-back cache, dirty bit goes high whenever the cache line is different from the main memory and it signifies that before evicting it, we need to store that line back to the main memory. Valid bit signifies that the corresponding cache line is valid, and write bit signifies that some store operation is done on that line.

Cache line will be flushed automatically to the main memory when some new cache line needs to be placed at the same location if it is dirty. But if the software wants to flush a cache line without waiting on it for the next line, the cache flush feature can be used. This is controlled by the software using the CSR register (*Cache Flush*). It can be used by the OS or software to if the writes needs to be visible on to the main memory even before accessing new line.

Cache flush CSR is located at 0x400 address of CSR memory map. LSB bit of that CSR

indicates whether cache flush has to take place or not.

To understand this lets take a scenario as shown in the figure 3.30 . Lets say the processor executes following store instruction sw x6, 0x0000_4000. Hence it needs to write the data at x6 to the location 0x0000_4000. Since this cache line is currently not there, there's a miss and the whole cache line will be brought to the dcache array and correspinsing tag (0x00004), along with the valid bit and write bit will be stored in the TAG array of the selected way. Since LRU[0] is 0, the way0 is selected and it is written 1. The dirty bit is also written to 1 as shown in the figure 3.30 .

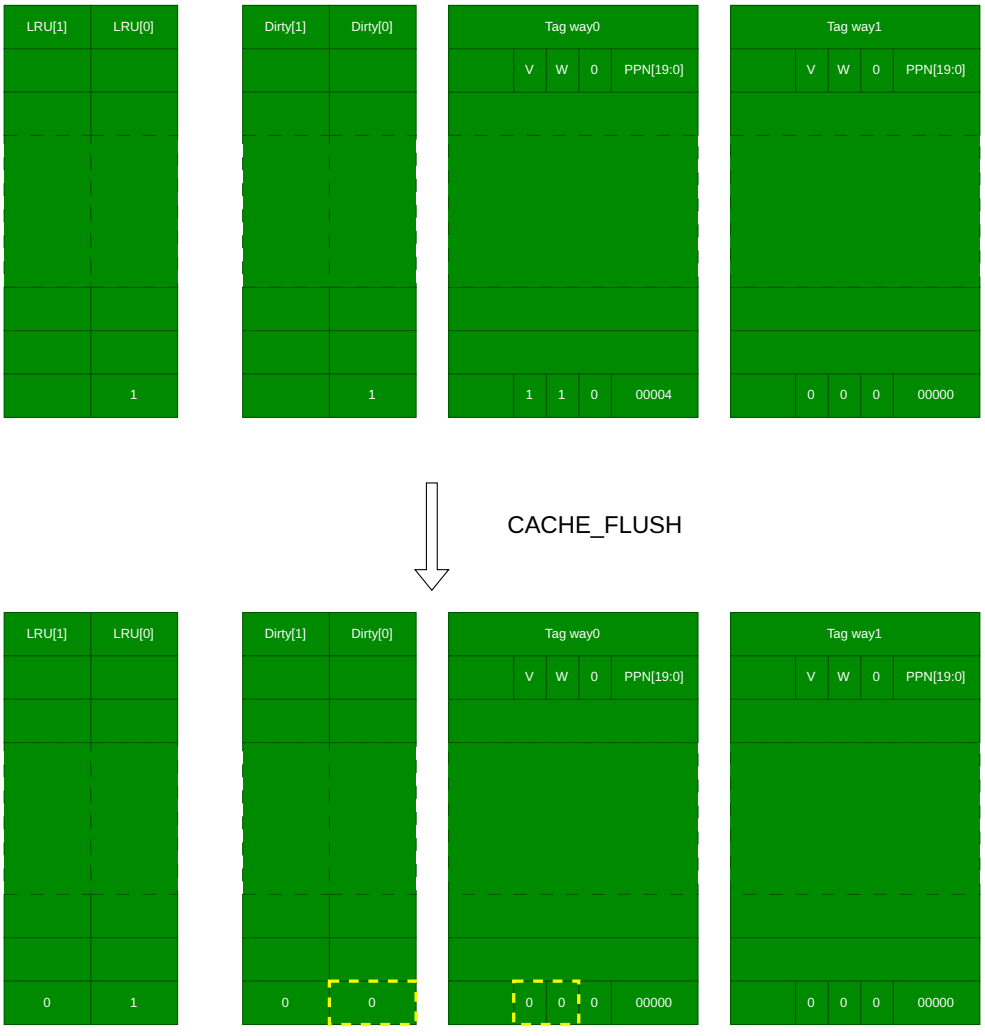

Figure 3.30: Cache flush mechanism

When the cache flushes, following thing occurs:

1. Reset the tag line (Write valid =0, write =0)
2. Reset the dirty bit
3. Reset LRU bit since that line is flushed.

The whole cache line is written on to the memory.

When switching between processes or threads, the cache may hold data relevant to the previous context. Flushing ensures that the new process doesn't see or interfere with stale cache data from the previous one — this is important for security and correctness. Also in multi-core RISC systems, each core has its own cache. To ensure coherency across cores, one core may need to flush (write back) or invalidate cache lines so others see the updated data.

3.10 Memory Protection Unit

3.10.1 MPU pipeline integration

The figure 3.31 shows the integration of MPU with the processor pipeline.

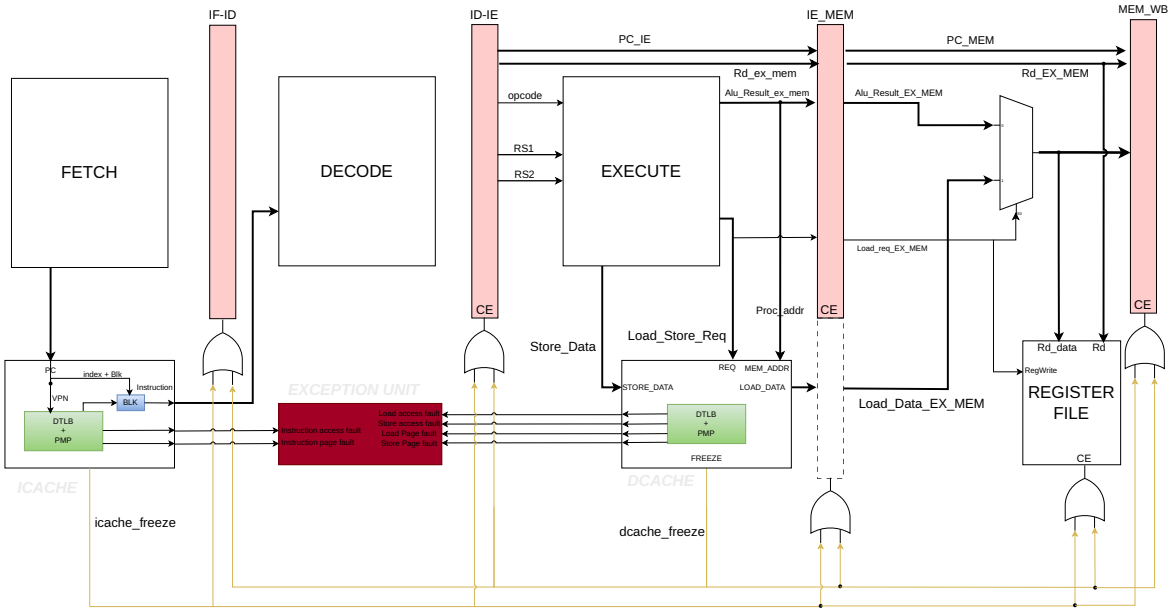

Figure 3.31: MPU pipeline integration

The PMP (Physical Memory Protection) block plays a critical role in enforcing memory access permissions in a RISC-V processor pipeline by integrating tightly with both instruction and data

access paths. As shown in the figure 3.31, the PMP is connected in parallel with the DTLB (Data Translation Lookaside Buffer) and ITLB (Instruction TLB) and performs permission checks immediately after address translation. When a virtual address is translated to a physical address, the PMP verifies whether the access type (instruction fetch, load, or store) is allowed for that physical address range based on the currently active PMP configuration.

In the instruction fetch path, the PMP works alongside the ITLB. If the instruction access does not meet the PMP permissions, an instruction access fault or instruction page fault is raised and directed to the exception unit. Similarly, on the data access path, after address translation by the DTLB, the PMP checks whether a load or store is permitted. Violations trigger exceptions like load/store access faults or page faults. These exceptions are routed to the exception unit, which stalls the pipeline by asserting control signals like *icache_freeze* or *dcache_freeze* to prevent further instruction or data processing until the fault is resolved.

The PMP regions are configured during booting and can be fixed for a specific application. By default, the boot region is assigned read and execute permissions to allow execution of boot code, after which additional permissions can be configured for other memory regions. Since this processor design does not support USER and SUPERVISOR modes, all PMP permissions are applied in machine mode, ensuring that protection policies are enforced even at the highest privilege level.

Suppose if we are at the user privilege level, then some of the memory regions would not be allowed for the user program to access. Then a user can switch to machine mode privilege level and can change the configuration of these PMP CSRs. Accessing these CSRs from user or supervisiory level will result in illegal instruction exception.

3.10.2 MPU block diagram

The figure 3.32 shows the Memory Protection Unit design.

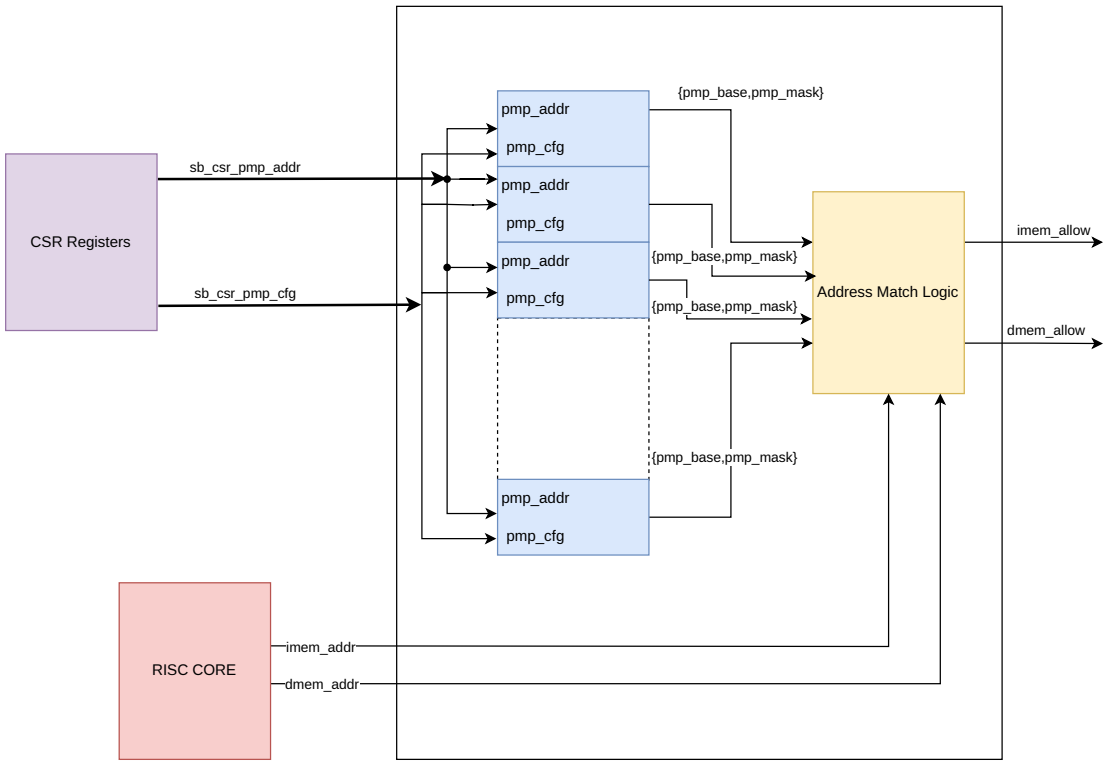

Figure 3.32: MPU RTL design

The internal block diagram of the MPU (Memory Protection Unit), based on the RISC-V Privileged Specification[23] , demonstrates how the system enforces memory access control through programmable physical memory protection (PMP) regions. At the heart of the MPU are the PMP address and configuration registers, which are configured via the CSR (Control and Status Register) interface. These registers store the base addresses (*pmp_base*) and corresponding masks (*pmp_mask*) that define memory regions, along with permission attributes such as read, write, and execute, encoded in pmp_cfg.

The MPU receives addresses from both instruction memory (*imem_addr*) and data memory (*dmem_addr*) paths. These addresses are passed to address match logic blocks, where they are compared against each defined PMP region using the configured base and mask values. The result of this matching determines whether access is permitted based on the region's permissions. For instruction accesses, the result is reflected in *imem_allow*, and for data accesses, in *dmem_allow*.

The PMP configuration and address registers (*sb_csr_pmp_cfg*, *sb_csr_pmp_addr*) can be updated by machine-mode software, allowing dynamic configuration of memory regions. Since only machine mode is supported in this implementation (no user or supervisor mode), all access checks apply uniformly in this highest privilege level. The MPU thus acts as a gatekeeper that filters memory operations before they reach memory, ensuring only authorized instruction fetches and data accesses are allowed, as mandated by the RISC-V privilege architecture.

3.10.3 PMP CSR registers

The figure 3.33 shows the CSR registers used for configuring the memory regions.

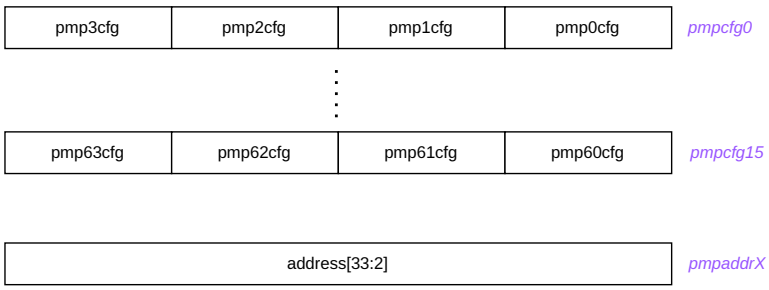

Figure 3.33: PMP CSR registers

The Physical Memory Protection (PMP) feature in RISC-V is configured through a set of dedicated CSR (Control and Status Register) registers, primarily consisting of *pmpcfg* and *pmpaddr* registers. The pmpcfg registers define the access permissions and address matching modes for each PMP entry. Each pmpcfg register (e.g., *pmpcfg0*, *pmpcfg1*, ..., up to *pmpcfg15* in RV32) is 8 bits per PMP entry and packs four entries per 32-bit register. Each 8-bit configuration field includes flags for read (R), write (W), and execute (X) permissions, as well as an address-matching mode (A) and a lock bit (L) that can prevent modification until the next reset.

Corresponding to each *pmpcfg* entry is a pmpaddr register (e.g., *pmpaddr0*, *pmpaddr1*, etc.), which defines the base address of the protected region. In RV32 systems, these registers use bits [33:2] of the address space, allowing memory regions to be defined in granularity of 4 bytes. The combination of these registers enables fine-grained control over memory access, making it possible to define secure execution regions, restrict access to critical peripherals, and isolate tasks. These registers are only writable in machine mode and are essential for enforcing the memory protection model defined in the RISC-V privileged architecture.

3.10.4 Address match logic

The figure 3.34 shows the address match mechanism used for allowing access to the valid region of physical memory for that particular thread.

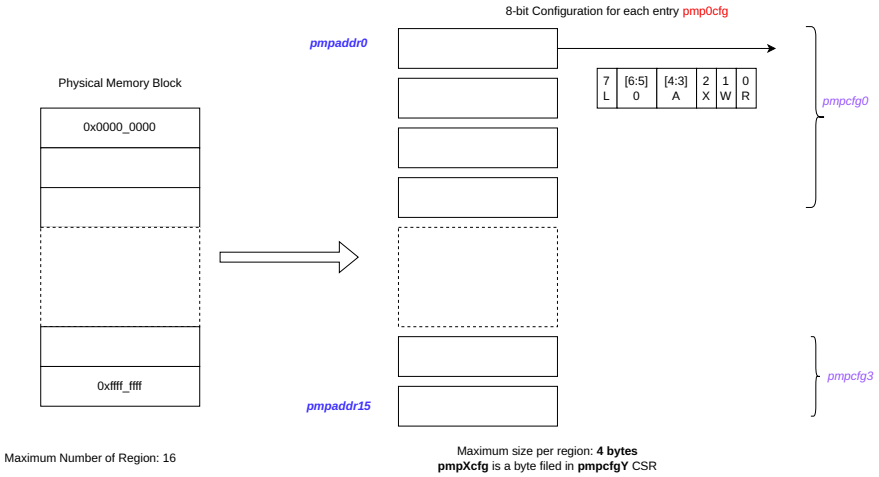

Figure 3.34: Address Match mechanism

In the RISC-V PMP (Physical Memory Protection) mechanism, memory regions are identified and access permissions are enforced using the *pmpaddrX* and *pmpcfgY* CSR registers. In our specific implementation, the physical memory is divided into 16 distinct regions, allowing fine-grained access control. Each region is associated with a corresponding *pmpaddrX* register, which defines the region's address boundaries. Permissions for each region are specified using 8-bit entries within *pmpcfgY* registers, where each pmpcfg can accommodate up to four 8-bit configuration fields, thus covering a total of 16 regions across *pmpcfg0* to *pmpcfg3*.

Each 8-bit configuration field includes bits for Read (R), Write (W), Execute (X) permissions, an Address matching mode (A), and a Lock bit (L). The A field plays a crucial role in defining the address range matching scheme. When A = 00, the region is disabled—no access is allowed. When A = 01, the Top of Range (TOR) mode is enabled, where the region starts at the previous PMP address and ends at the current pmpaddrX. This is typically used to define the first region. When A = 10, the region covers a fixed 4-byte range at the address specified by pmpaddrX. Finally, when A = 11, the NAPOT (Naturally Aligned Power-Of-Two) mode is used, allowing flexible region sizes that are aligned and sized to powers of two (e.g., 8, 16, 32 bytes, etc.), offering scalable and efficient region mapping.

This setup allows machine-mode software to configure and lock down access to different parts

of memory at boot time, enforcing strict isolation and security guarantees in systems without user or supervisor modes.

3.10.5 Sample PMP configuration

To configure a region using the Physical Memory Protection (PMP) unit, both the base address and the size of the region must be specified. RISC-V supports two primary addressing schemes for PMP entries: TOR (Top of Range) and NAPOT (Naturally Aligned Power of Two). In the TOR scheme, the address specified in a PMP entry defines the upper bound of the protected region, while the lower bound is defined by the previous PMP entry. This means that access is permitted from the address in the previous entry up to, but not including, the current entry's address. For the very first region, if TOR is used, the start address is implicitly taken as 0x0000_0000. For configuring intermediate or more complex memory regions, the NAPOT scheme is preferred. NAPOT allows specifying memory regions that are naturally aligned and sized as a power of two, using a special encoding format in the address field to represent both the base address and the region size simultaneously. This encoding simplifies memory protection for commonly aligned and sized regions, and enables more compact representation of memory ranges. The encoding is given in the table 3.16.

Base address	**Region size**	**pmpaddrX value**
addr[31:0]	8B	addr[33:2] | 1'b0
addr[31:0]	32B	addr[33:2] | 3'b011
addr[31:0]	4KB	addr[33:2] | 10'b01_1111_1111
addr[31:0]	16KB	addr[33:2] | 12'b0111_1111_1111

Table 3.16: Address and size encoding for NAPOT scheme

The position of 0 (LSZB) determines the size for the particular region. We can use following formula for the same:

$$RegionSize = 2^{(LSZB+3)}$$

Hence if we have region of 16KB, then LSZB has to be 11. It has to be ored with the base address right sifted by 2.

Below section gives few of the memory region configurations for this core, and if further section needs to be added, it can be added in the same way.

3.10.6 Boot ROM configuration

The figure 3.35 shows the Boot ROM region.

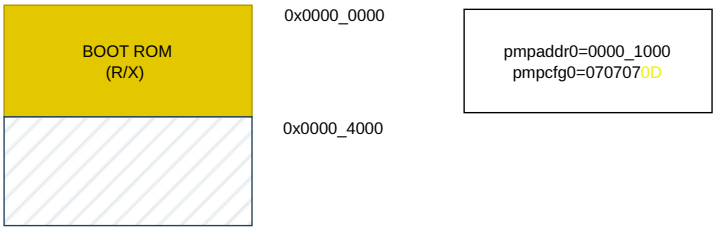

Figure 3.35: Boot ROM region

Since it is the very first region, we may use the TOR (Top of Range) scheme. Hence we right shift the top address (0x0000_1000) and store it in PMPADDR0 region. Since it has the RX attributes, the value of pmpcfg0 is 0x0D.

3.10.7 SRAM Configuration

The figure 3.36 shows the SRAM region.

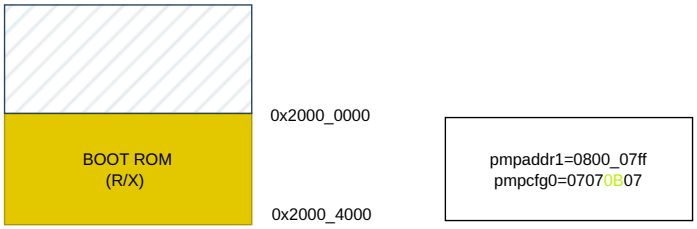

Figure 3.36: SRAM region

We will be using NAPOT (Natually aligned power of 2) addressing scheme for this region. So we right shift the base address by 2 (0x0800_0000). Since the size of this region is 16KB, hence the value of LSZB is 11.

Hence the final value of PMPADDR1 is 0x0800_07FF. Since it has the attribute of R/W, the value of pmpcfg register will be 0x0707**0B**07. Following assembly command 3.3 is used to write both the configuration.

Algorithm 3.3 PMP configuration for SRAM region

```
.equiv PMP_REGION0, 0x00004000
.equiv PMP_REGION1, 0x20000000
li t0,PMPADDR0
srli t0, t0, 2
csrw pmpaddr0, t0
li t0, PMPADDR1
srli t0, t0, 2
csrw pmpaddr1, t0
li t0, 0x07070B0D
csrw pmpcfg0, t0
```

3.11 Memory Map

The memory map of a processor as shown in table 3.17 defines how different regions of memory are organized and accessed within the system's address space. It serves as a blueprint that assigns specific address ranges to various functional units such as RAM, ROM, peripheral devices, memory-mapped I/O, and system control registers. In systems with virtual memory support, the memory map also distinguishes between user and privileged address spaces, allowing for isolation and protection. The memory map ensures that software knows where to load programs, where to store data, and how to interact with hardware. It also plays a critical role in defining areas used for page tables, interrupt vectors, and boot code.

Address Range	Name	R/W/X	SIZE
0x0000_0000 - 0x0000_4000	BOOT ROM	RX	16KB
0x2000_0000 - 0x2000_4000	SRAM	RW	16KB
0x5F00_0000 - 0x5F00_4000	PERIPHERAL	RW	16KB
0x5100_0000 - 0x5100_4000	DEBUG	RX	16KB
0xF000_0000 - 0xF000_4000	PAGE TABLE	RWX	16KB
0xFF00_0000 - 0xFF00_4000	INTERRUPT HANDLER	RWX	16KB

Table 3.17: Memory Map

A well-structured memory map enables efficient access, simplifies system software design like operating systems and drivers, and supports features like memory protection, caching control,

and device-specific optimizations. Visual representation of the physical memory region is given in figure 3.37.

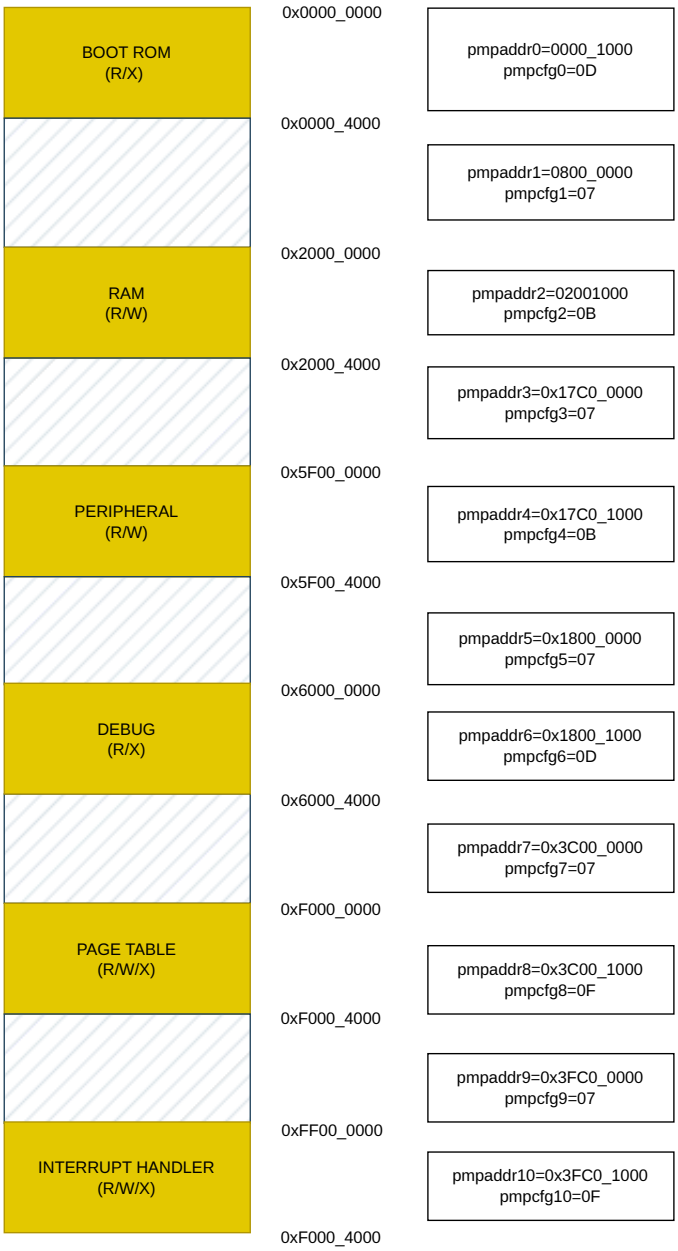

Figure 3.37: Memory map and PMP configuration

3.12 Debugger Design

3.12.1 Debugger Software

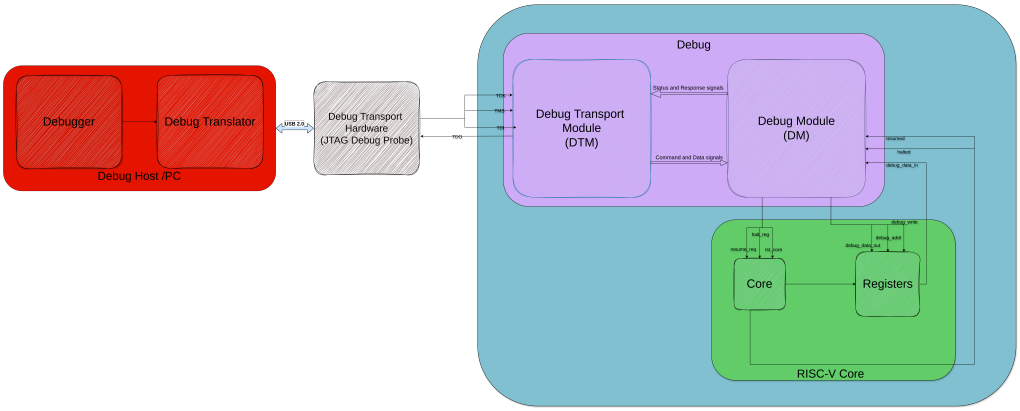

Figure 3.38: Low level JTAG signal Control

The debugger software designed for this work is a custom-built tool developed in Python, specifically tailored to interface with the debug infrastructure of a RISC-V processor through the JTAG interface. Its primary objective is to provide a lightweight yet powerful control interface for hardware-level debugging, offering both interactive control and automation capabilities.

The debugger supports a comprehensive suite of features essential for low-level processor control and visibility. These include core reset functionality, retrieval of core identification and configuration information, and the ability to halt and resume processor execution on demand. Crucially, it enables direct access to architectural state, allowing the user to read from or write to any general-purpose register (*GPR*), control and status register (*CSR*), or memory location within the system.

A key component of the debugger is its fine-grained control over the JTAG data transfer process. Unlike high-level tools that abstract away protocol details, this debugger allows for explicit manipulation of JTAG states and data registers, making it highly suitable for testing, validation, and bring-up of the custom debug interface.

To improve usability during halted states, the debugger includes a "quick glance" feature that provides a real-time snapshot of all *GPR* and *CSR* values. This functionality enables rapid inspection of the processor state, facilitating efficient debugging sessions and reducing the time to identify incorrect execution paths or corrupted state.

In essence, the debugger functions as both a development and diagnostic tool, bridging the gap between software-driven inspection and hardware-level control in the context of a custom RISC-V debug system.The overall interaction between the debugger, debug transport translator, DTM, DM, and the target core is illustrated in figure 3.38, which depicts the hierarchical flow of commands and data from the host software down to the processor.

3.12.2 Debug Translator

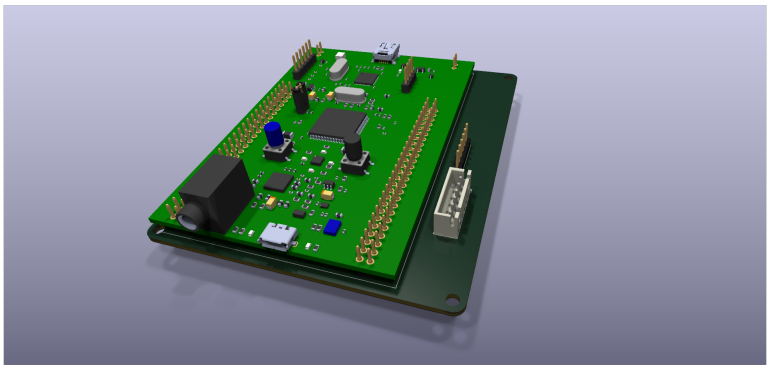

Figure 3.39: Debug Translator

A custom debug translator was developed using the STM32F407 microcontroller (shown in 3.39) to serve as a USB-to-JTAG interface between the host-side debugger and the target RISC-V processor. The translator facilitates reliable communication with transfer rates up to 1 MHz, enabling responsive and efficient debug sessions. The host system interfaces with the STM32-based translator via USB 2.0 Full-Speed using a Virtual COM Port (*VCP*) interface, which provides a simple and widely supported mechanism for serial communication over USB.

The firmware was written using the STM32 Hardware Abstraction Layer (HAL), ensuring portability across a broad range of STM32 boards. This modular and hardware-agnostic design supports reuse on other compatible STM32 platforms with minimal changes, promoting scalability and adaptability for different deployment scenarios.

Internally, the translator manages the full JTAG state machine, including *TMS* - driven state transitions and *TDI/TDO* data shifting. This low-level control enables direct and deterministic communication with the processor's Test Access Port (*TAP*) and Debug Transport Module (*DTM*), which is essential for the operation of the Debug Module Interface (*DMI*).

Overall, the Debug Translator forms a crucial component in the hardware debug infrastructure, offering a compact, low-cost, and reusable solution for interfacing standard USB hosts with

custom RISC-V debug hardware.

3.12.3 JTAG Design

The Joint Test Action Group (JTAG) interface serves as the primary external debug access mechanism for the processor. The implemented JTAG architecture adheres to the IEEE 1149.1 standard, enabling TAP (Test Access Port)-based communication between the host and the debug infrastructure. In this design, the boundary scan registers are intentionally excluded, as the focus is solely on enabling debug and control capabilities rather than full scan-chain testability.

The architecture includes a standard TAP controller, driven by the *TCK* and *TMS* control signals. The TAP controller is implemented as a finite state machine, with state transitions occurring on the rising edge of *TCK*, and output signals changing on the falling edge. The controller supports standard transitions for instruction and data scan sequences, with *TMS* defaulting to logic high (1). Resetting the TAP controller is achieved by applying five consecutive logic high (1) values on *TMS*, which asynchronously brings the TAP into the Test-Logic-Reset state.

TAP Controller Implementation

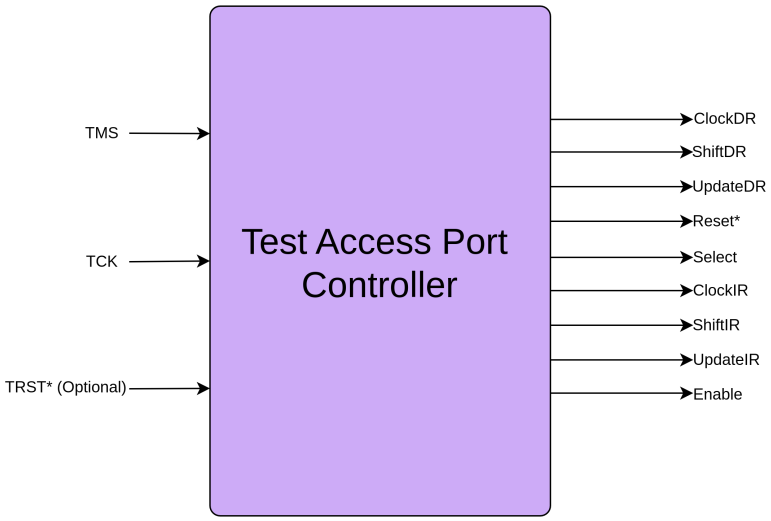

Figure 3.40: TAP Controller

The implemented TAP controller (shown in figure 3.40) follows the 16-state FSM model defined in the IEEE 1149.1 standard. Key states include *Test-Logic-Reset, Run-Test/Idle, Select-DR-Scan, Capture-DR, Shift-DR, Exit1-DR, Pause-DR, Exit2-DR,* and *Update-DR* along with

their instruction register (*IR*) counterparts. This design ensures deterministic navigation through both instruction and data scan paths using only *TCK* and *TMS*.

The state machine transitions synchronously on the positive edge of *TCK*, while the internal output control signals such as *ShiftIR*, *UpdateIR*, *ShiftDR*, *UpdateDR*, etc., toggle on the negative edge of *TCK* to align with standard scan timing protocols. This ensures timing compatibility with standard-compliant external JTAG masters.

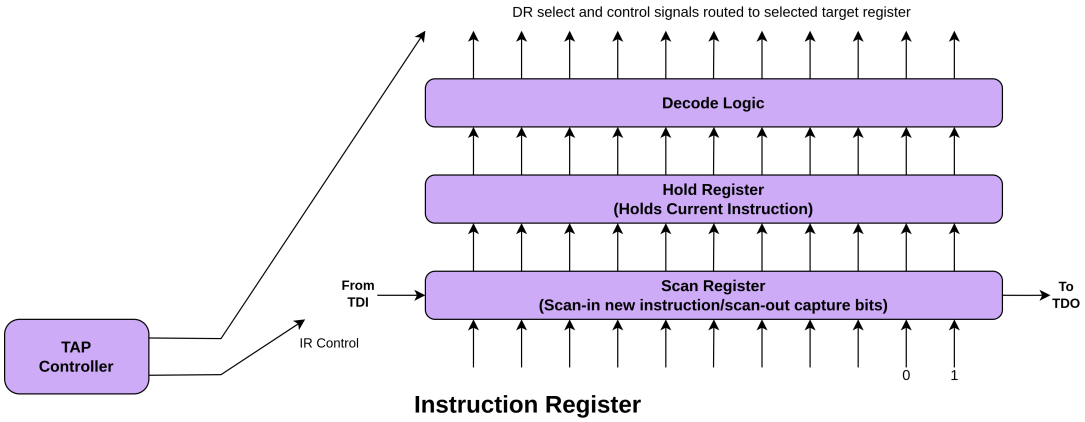

Figure 3.41: Instruction Register

The Instruction Register (*IR*) as shown in figure 3.41 is composed of:

- A *scan register*, which captures new instructions during the *Shift-IR* phase
- A *hold register*, which retains the selected instruction during execution
- Decode logic, which interprets the instruction and selects the appropriate data register (*DR*) or internal control path

The *IR* supports the following core instructions:

- *BYPASS*: Routes the scan path through a single-bit bypass register
- *IDCODE*: Selects a read-only 32-bit device identification register (if present)
- *ACCESS*: Enables access to internal debug data registers

During the *Capture-IR* state, the *IR* hardwires the two least significant bits to 01 for compliance checking. The *IDCODE* instruction is loaded by default on reset. If the device does not implement an identification register, the *IDCODE* behaves as a *BYPASS* instruction.

The TAP controller emits a set of output control signals:

- *ClockDR, ShiftDR, UpdateDR*
- *ClockIR, ShiftIR, UpdateIR*
- *Reset, Enable, Select*

These signals coordinate the activation of scan paths and synchronize internal operations with external scan sequences. The minimalist implementation avoids unnecessary complexity, ensuring a compact and efficient debug interface.

3.12.4 Overall Debug Flow Architecture

The RISC-V debugger architecture is designed to offer precise control and inspection capabilities over the processor core while maintaining modularity, extensibility, and adherence to the RISC-V Debug Specification (v1.0 Stable). The implemented design incorporates a minimal but complete debugging system, supporting critical features such as halting and resuming execution, register and memory access, single-step operation, and core status monitoring. Additionally, several custom debug features have been integrated to enable standalone testing of *DM* without being dependent on a working processor core.

Debugger Initialization

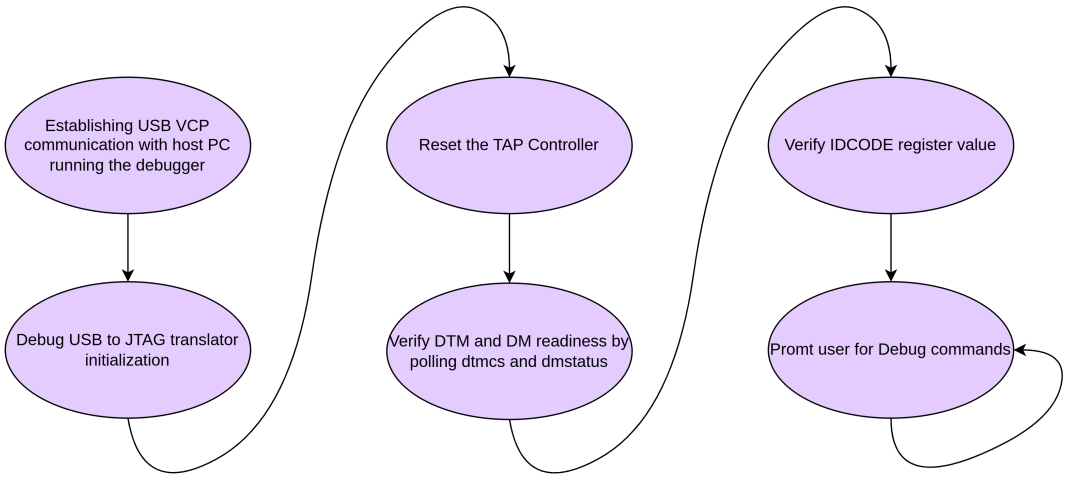

Figure 3.42: Debugger Initialization Steps

At system startup, the debugger initiates a structured initialization sequence to establish communication and prepare the target for debug operations. The process begins by setting up

a USB 2.0 Full-Speed virtual COM port (*VCP*) interface between the host machine and the STM32F407-based USB-to-JTAG translator. This link serves as the communication backbone for transmitting JTAG signals and debug commands.

Once the USB connection is successfully established, the STM32F407 configures its GPIO pins for JTAG signal output *(TCK, TMS, TDI, TDO)* and performs a hardware reset of the Test Access Port (*TAP*) controller. The TAP is transitioned from an unknown state to the *Test-Logic-Reset* state by issuing a sequence of five consecutive logic '1' bits on the *TMS* line, as per the IEEE 1149.1 JTAG standard. Subsequently, the TAP enters the *Run-Test/Idle* state, ready for further JTAG operations.

Following TAP initialization, the debugger verifies the readiness of the *Debug Transport Module (DTM)* and the *Debug Module Interface (DMI)* by polling the *dtmcs* (DTM control and status) and *dmstatus* (Debug Module status) registers, respectively. These checks ensure that the debug infrastructure is responsive and functional. The *IDCODE* is then read and validated to confirm the identity of the target device.

Once all checks pass, the system is ready to receive debug commands from the user. This entire process is depicted in the system initialization flow diagram shown in figure 3.42.

Instruction Execution Control

Execution control is a fundamental capability of the debugger. It allows halting the processor, inspecting or modifying state, and resuming execution. These features are implemented through write operations to the *dmcontrol* register. Halting the core is achieved by asserting the *haltreq* bit, while resumption is triggered by setting the *resumereq* bit. Single-stepping is supported by setting the *step* bit in the *dcsr* register, which instructs the core to execute exactly one instruction after being resumed. These operations rely on a clean handshake protocol between the Debug Module and the core's debug interface.

State Monitoring

To monitor the processor state, the debugger reads internal registers via *abstract* commands. These include:

- *GPRs* and *CSRs*, accessed using Access Register commands
- Memory locations, accessed using Access Memory commands

The results of such operations are routed through the *data0* and *data1* registers, which temporarily store the data during DMI transactions. Additionally, core status is obtained from the

dmstatus register, which indicates whether the core is halted, running, or reset. The *abstractcs* register provides status information about the abstract command engine, including error flags (*cmderr*) and whether the engine is currently busy.

Debug Interaction Flow

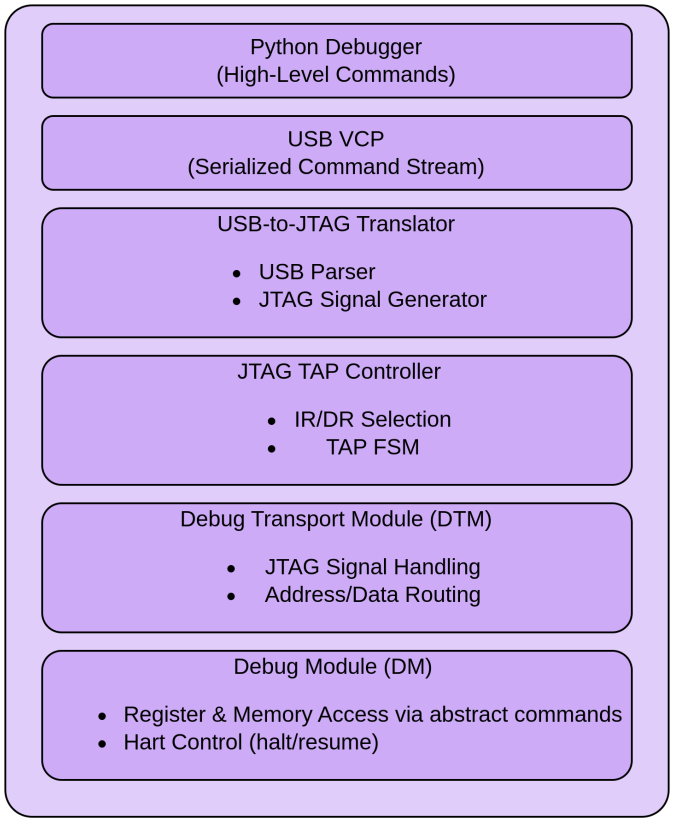

Figure 3.43: Debug Hierarchical Structure

The RISC-V debug system as shown in figure 3.43 follows a hierarchical and modular architecture, enabling clean abstraction between user-level debugging commands and low-level hardware signaling. This layered structure improves scalability, portability, and maintainability, while simplifying system validation and integration.

1. *Debugger Layer (Host Interface):* A custom Python-based debugger forms the topmost layer of the stack, serving as the primary user interface. It allows the user to issue both high-level debug commands (such as halting the core, reading/writing registers, and accessing memory) and low-level JTAG operations. High-level operations are translated into sequences of register accesses to the Debug Module (*DM*), enabling protocol-agnostic debugging through abstract commands or program buffer execution.

2. *USB Communication Layer (Virtual COM Port):* Communication between the host debugger and the embedded translator is established using a USB 2.0 Full-Speed connection, utilizing the Virtual COM Port (*VCP*) mode of the STM32F407. This layer is responsible for reliably transmitting encoded command and response packets across the USB link, abstracting the physical transport from the logical operation being performed.

3. *Translator Layer (STM32F407 Microcontroller):* The STM32F407-based translator serves as a USB-to-JTAG bridge. It decodes the received USB commands and generates the corresponding JTAG waveforms *(TCK, TMS, TDI)* via its GPIO pins. It also monitors *TDO* responses for return to the host. The translator manages TAP state transitions, instruction selection, and data register access to interface correctly with the RISC-V TAP and Debug Transport Module (DTM).

4. *JTAG TAP Controller Layer:* The Test Access Port (TAP) controller, implemented according to the IEEE 1149.1 JTAG standard, interprets the JTAG signal stream and handles TAP state machine transitions. It enables access to various scan chains, such as the instruction register (*IR*) and data register (*DR*), which are used to issue commands to the Debug Transport Module (*DTM*) within the target chip.

5. *Debug Transport Module (DTM):* The *DTM* acts as an interface between the JTAG scan chain and the memory-mapped debug infrastructure. It receives sequences of bits from the JTAG *DR* and decodes them as register read/write transactions targeting the *Debug Module*. While sometimes referred to as "DMI transactions", these are not protocol-level messages but rather structured accesses to fixed registers within the *DM*. The DTM also handles readiness and error signaling to ensure reliable communication.

6. *Debug Module (DM):* The Debug Module is the functional core of the debug system. It implements the actual debug features, such as halting or resuming harts, abstract register/memory access, and execution of instructions via the *Program Buffer*. It provides memory-mapped registers (*dmcontrol*, *dmstatus*, *command*, *dataN*, *progbufN*, etc.) accessible through the *DTM*. Responses from the *DM* (e.g., register values or status flags) are returned via the same path, completing the interaction loop.

This layered interaction model offers a robust and flexible framework for RISC-V debugging. By decoupling user-level debug logic from hardware-specific signaling, the design ensures portability across platforms, simplifies development and verification, and enables seamless integration of additional features such as scripting, logging, or GUI-based frontends in the future. Moreover, the modular design facilitates future enhancements—such as support for alternative

transport layers or multi-core debug capabilities—without requiring significant changes to the core architecture.

Data Transfer Pathways

The primary mechanism for transferring data between the external debugger and the processor is the DMI interface. Each DMI transaction contains:

- A 8-bit address for the target DM register
- A 32-bit data field
- A 2-bit opcode (e.g., read, write, nop)

These transactions are serialized over JTAG and interpreted by the DTM. Inside the Debug Module, data flows to and from core-facing components via multiplexers and latches. Abstract commands read from or write to *CSRs*, memory, or *GPRs*, depending on the contents of the command register. The results are staged into *data0*/*data1* for read-back or supplied from these registers for write operations.

Error Handling

Robust error handling is built into both the standard and custom components of the debug flow. Abstract command errors are flagged via the *cmderr* field in *abstractcs*, which may indicate unsupported access types or illegal operations. The *dmstatus* register includes bits such as anynonexistent and anyunavailable to report unavailable harts or improper configurations. The debugger software detects protocol-level issues, such as failed TAP sequences or invalid data patterns, and responds by reinitializing the JTAG chain or issuing resets to recover communication.

Implemented Debug Registers

The Debug Module implements a set of essential registers that play critical roles in controlling and querying the debug system's state. These registers manage data transfers, control operations, and provide status feedback for both the debug module and the RISC-V target. For a comprehensive overview of these registers, including their addresses and detailed descriptions, please refer to table 3.18.

Register	Address	Description
data0	0x04	Primary data register used for reading/writing data between the debugger and the target.
data1	0x05	Used as the address register in abstract memory operations, holding the address for memory accesses.
dmcontrol	0x10	Control interface for halt/resume commands and enabling/disabling the debug module.
dmstatus	0x11	Provides status information about the hart's state (halted, running) and the debug module.
abstractcs	0x16	Status register for the abstract command engine, indicating the availability and status of abstract commands.
command	0x17	Encodes abstract command operations, such as register or memory access requests.
hartinfo	0x12	Contains metadata about the connected hart(s), including available resources and configuration.

Table 3.18: Implemented Debug Registers

Custom Debug Registers

In addition to the standard debug registers, several custom registers were implemented to allow for comprehensive testing of the debugger's functionality. These custom registers, detailed in table 3.19, provide mechanisms to verify the debugger's operations independently of a valid processor core, supporting fault injection, signal probing, and real-time data monitoring for enhanced debugging validation.

Custom Reg	Address	Function
custom0	0x04	Overrides regno_data used in register read operation
custom1	0x05	Overrides data1_data used in memory read operations
custom2	0x10	Overrides other debug signals from core to debugger
custom3	0x11	Lock register: Enables core bypass when a specific hex key is written
custom4	0x16	Probes real-time values of regno_data
custom5	0x17	Probes real-time values of data1_data
custom6	0x12	Probes other signal lines into the Debug Module

Table 3.19: Custom Debug Registers

These custom registers enable the debugger to be tested and validated independently of the core, ensuring its functionality through various debugging operations. The ability to simulate conditions, inject faults, and monitor real-time data greatly improves the reliability and robustness of the debugger during development.

3.12.5 Debug Module

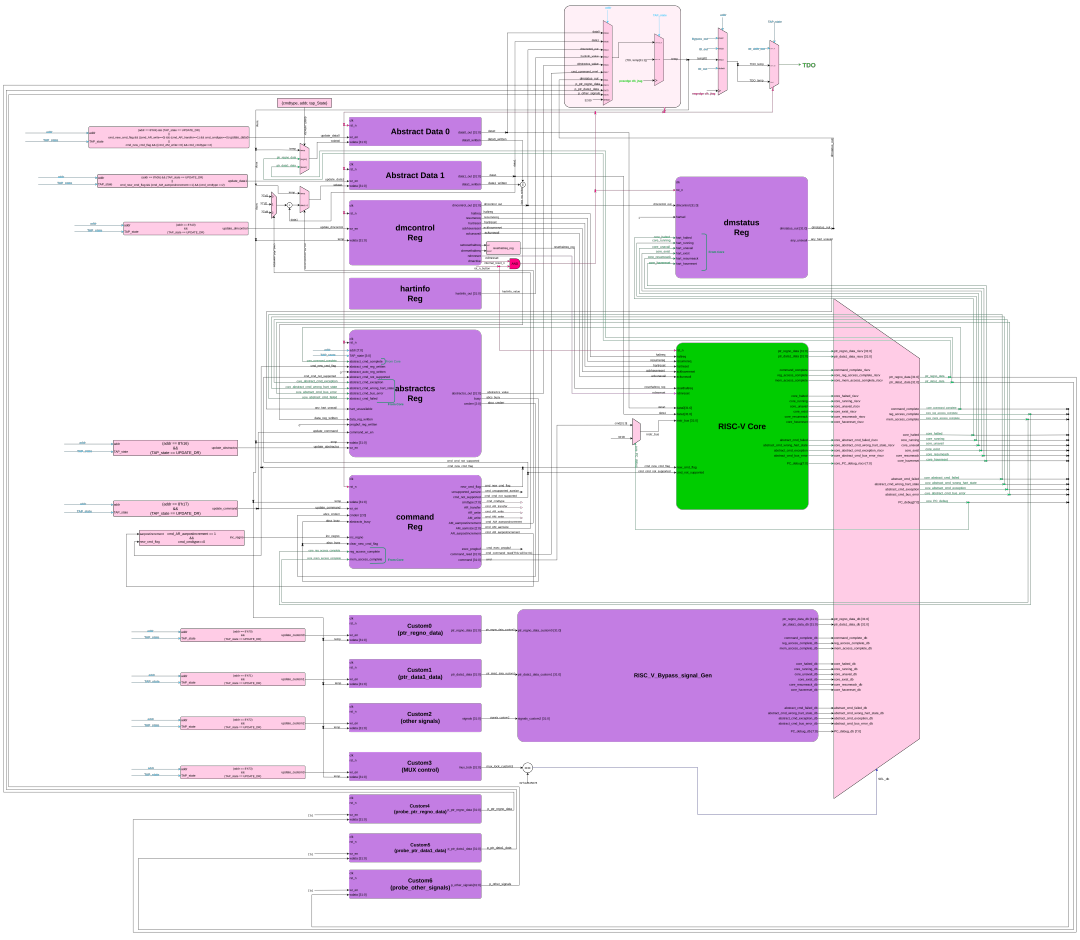

Figure 3.44: Debuger Module Architecture

The Debug Module (DM) is the principal logic block responsible for managing external debugging interactions with the RISC-V processor core. It serves as the terminal endpoint for Debug Module Interface (DMI) transactions and mediates all commands from the host debugger to the internal debug infrastructure. This includes operations such as halting and resuming the core, reading/writing general-purpose and control/status registers (*GPRs* and *CSRs*), memory inspection and modification, and runtime monitoring of processor state. The DM design

adheres to the RISC-V Debug Specification v1.0 with multiple custom extensions to support self-verification and deep observability for validation purposes, as shown in the figure 3.44.

3.12.5.1 Architecture and Role in the Debug Flow

At a high level, the Debug Module (*DM*) functions as an intermediary between the Debug Transport Module (*DTM*) and the RISC-V core and memory subsystem. The *DTM* receives serialized commands from the external debugger via the JTAG interface and translates these into simple register read/write operations. The *DMI* (Debug Module Interface) facilitates communication between the *DTM* and the *DM* registers, enabling the transfer of data. Based on the contents of the command register, the *DM* performs abstract operations such as register or memory access. These operations often involve the *data0* and *data1* registers, which temporarily hold operand values and results for these operations, allowing for seamless execution of debugging tasks.

3.12.5.2 Implemented Debug Module Registers

The following registers form the core of the debug module:

- ***dmcontrol (0x10):*** A 32-bit register for managing the DM and harts.
  - *dmactive:* Master enable for the DM. Must be set to 1 for other fields to be active.
  - *ndmreset:* Optional field to reset the core or other non-debug elements.
  - *haltreq:* Request to halt the selected hart.
  - *resumereq:* Request to resume execution.
  - *hartsel:* Selects which hart is targeted for debug operations. Only a single-hart system is used in this implementation.
- ***dmstatus (0x11):*** Reports the status of the DM and selected hart.
  - *allhalted*, *anyhalted:* Indicate the halt status of harts.
  - *allrunning*, *anyrunning:* Indicate running status.
  - *authenticated:* Always set to 1 as authentication is not implemented.
  - *version:* Set to 2, as per RISC-V Debug Spec v1.0.
- ***abstractcs (0x16):*** Reports the state of the abstract command engine.

  - *busy:* Indicates whether an abstract command is currently in progress.
  - *cmderr:* 3-bit error code; cleared on write of 1.
  - *datacount:* Set to 2, representing availability of *data0* and *data1*.
  - *progbufsize:* Set to 0, as no *program buffer* is implemented.
  - *support:* Reflects that only *Access Register* (0) and *Access Memory* (2) are supported.

- ***command (0x17):*** Writing to this register initiates an abstract command.
  - Supported commands:
    * *Access Register:* Read/write any *CSR* or *GPR*
    * *Access Memory*: Read/write to main memory (via address and size fields)

- ***data0, data1 (0x04, 0x05):*** Used as source/destination for operands and results during abstract command execution.

- ***hartinfo (0x12):***
  - *dataaccess:* Set to 1
  - *datasize:* 2 (32-bit)
  - *dataaddr:* Address offset to *data0*. This register helps debuggers understand how to interact with the data registers.

3.12.5.3 Custom Debug Registers

To enable self-validation, system observability, and finer control, several custom debug registers have been implemented:

- ***custom0 (ptr_regno_data):*** Contains a debug-controlled override value for *regno_data*, used during testing when bypassing the RISC-V core.

- ***custom1 (ptr_data1_data):*** Debug-controlled override for *data1_data* input to the DM.

- ***custom2 (control mux signal override):*** Encapsulates all *multi-bit signals* going from the core to the DM, including *CSR write enables*, *GPR access signals*, and more. Used to simulate internal events without actual core activity.

- ***custom3 (lock register):*** Contains a hardcoded unlock code (0xA5A5A5A5). Only when this code is written can the DM bypass the processor's signals and allow *custom0*, *custom1*, and *custom2* values to take effect. This ensures debug security and prevents unintended bypass activation.
- ***custom4, custom5, custom6:*** These are monitor-only registers
  - *custom4:* Observes *regno_data* from the debugger
  - *custom5:* Observes *data1_data* from the debugger
  - *custom6:* Captures and shows all other control signals being driven into the DM

Together, these registers form a complete observability and override framework allowing full control and debug capability, even without processor cooperation.

3.12.5.4 Abstract Command Pipeline

The debug module's abstract command handler is implemented as a small state machine that:

1. Parses the command (bits [31:24] of *command*).
2. Validates its legality (e.g., only Access Register or Access Memory).
3. Performs the operation:
   - For register access:
     - Extracts the register number and access type from command bits.
     - Routes the request through internal GPR/CSR muxes or to the custom override.
   - For memory access:
     - Uses the address in *data0* and size specifier in *command*.
     - Performs the read/write over a bus-connected memory interface.

If any invalid command or out-of-bounds register access occurs, the *cmderr* field is updated accordingly. The command is considered complete when *busy* is de-asserted.

3.12.5.5 Integration and Control Signals

The DM interfaces with the core via a debug bus that includes:

- *GPR* index/data buses
- *CSR* index/data buses
- Control lines: *halt*, *resume*, *reset*
- Status indicators: *halted*, *running*

All control signals follow handshake semantics to ensure synchronized behavior across clock domains. This is critical since JTAG-driven inputs are asynchronous relative to the processor clock domain.

The halt/resume logic within the core is implemented using edge-triggered detection of *haltreq* and *resumereq* bits in *dmcontrol*, gated by *dmactive*. Once halted, the processor enters Debug Mode and begins executing a small debug handler, if configured. Since no program buffer is present, all debug control flows through abstract commands.

3.12.5.6 Debug Flow Summary

The overall debug operation follows this flow:

1. Debugger sends DMI write to *dmcontrol* $\rightarrow$ Sets *haltreq*.
2. Core transitions to halt state $\rightarrow$ DM updates *dmstatus*.
3. Debugger issues abstract command (via *command* + *data0*).
4. DM executes the command, potentially reading/writing memory/CSRs/GPRs.
5. Debugger clears *haltreq* and sets *resumereq* to continue execution.

3.12.5.7 Additional Features and Considerations

- No program buffer is implemented. All debug activity is performed via data registers and abstract commands.

- Single-step functionality is supported using the *dcsr.step* field and custom halt/resume logic.
- No authentication or access protection mechanisms are currently implemented.
- The system supports a single hart only (*hartsel*=0).

3.12.5.8 Additional Features and Considerations

The implemented Debug Module supports all required functionalities defined by the RISC-V Debug Specification and extends into several optional capabilities to enhance flexibility and control. Specifically, the module:

1. Exposes implementation details such as supported abstract commands, data register sizes, and hart configuration through *hartinfo*, *abstractcs*, and related control/status registers.
2. Supports halting and resuming of the processor through the *haltreq* and *resumereq* signals in *dmcontrol*.
3. Reports core status via fields in *dmstatus*, indicating whether the hart is halted or running.
4. Enables abstract register access to all general-purpose registers (*GPRs*) of the core via the *command*, *data0*, and *data1* registers.
5. Provides access to a core reset signal, enabling debug initialization immediately after system reset.
6. Implements early debug entry, allowing the debugger to connect and halt the hart as it exits reset, regardless of the reset cause.
7. Extends abstract access to non-GPR registers, including *CSRs*, via properly encoded command fields and internal decode logic.
8. Supports memory-mapped debug access, allowing read and write operations to the processor's memory address space from the debugger's perspective.

These features ensure that the debugger can not only inspect and manipulate core state, but also validate behavior from power-up through full execution, including support for advanced use cases like bare-metal debugging and post-reset diagnostics.

3.12.6 Reset Control and Debug Entry Coordination

Reset control is a critical feature in the debug architecture, enabling the debugger to reliably manage the processor's execution lifecycle and gain visibility from the very first instruction after reset. The implemented debug system supports two complementary reset mechanisms—one initiated via the Debug Module and the other through an external hardware switch—each integrated to ensure flexibility, safety, and effective debug entry.

The Debug Module-based reset is triggered by the external debugger through the *dmcontrol* register. This software-controlled reset signal is internally synchronized and routed to the processor core and associated debug logic. Upon release of the *reset*, the Debug Module can assert a *haltreq* signal, allowing the processor to enter debug mode immediately after reset. This supports the RISC-V Debug Specification's requirement for "*debug-after-reset*" behavior. The mechanism ensures that the debugger can observe and intervene before the processor executes the first instruction post-reset, enabling use cases such as boot-time diagnostics, breakpoint setup, or register probing.

Parallel to this, the system includes a manual reset mechanism via a physical push-button switch that triggers a global hardware reset. This reset line synchronously resets all major subsystems, including the core, memory interface, and debug infrastructure. While this method ensures a clean and full system reinitialization, it requires the debugger to re-establish the debug connection and reconfigure debug state (e.g., reasserting halt requests or restoring context). To distinguish between these reset causes, the debug logic inspects reset signals and can maintain internal state only when reset is initiated via the debug interface.

In both scenarios, reset coordination is handled carefully to avoid metastability or inconsistent states. All resets are synchronized with system clocks, and transitions between reset and active states are controlled via explicit state machines. The implementation ensures that TAP controller integrity is maintained across reset cycles, and all required Debug Module registers are reinitialized or retained appropriately depending on the reset source.

This dual reset mechanism, combining controlled debugger-triggered reset with full-system hardware reset, provides comprehensive control for development, testing, and fault recovery, making the debug system robust and responsive to diverse use cases.

3.12.7 Run Control

Run control is the core mechanism through which the external debugger can orchestrate and manage the execution state of the processor. It encompasses halting and resuming the core, initiating single-step execution, and coordinating transitions between normal and debug modes. In compliance with the RISC-V Debug Specification, and tailored to a single-hart implementation, the debug module provides fine-grained control over the processor's execution state through a combination of hardware signals and abstract commands.

The primary mechanism for halting the processor is the assertion of the haltreq bit within the *dmcontrol* register. When this bit is set by the external debugger, the Debug Module asserts an internal halt request signal to the core. The core responds by entering debug mode at a well-defined point in execution and sets the *allhalted* bit in the *dmstatus* register, which serves as confirmation to the debugger that the halt was successful. The halted state allows the debugger to read or write general-purpose registers (*GPRs*), control and status registers (*CSRs*), and initiate memory transactions through the abstract command interface.

Resuming execution is initiated by setting the *resumereq* bit in the *dmcontrol* register. Upon detecting this request, the Debug Module clears the *haltreq* and generates a *resume* signal to the core. The core then exits debug mode and resumes normal execution from the address stored in the *dpc (Debug Program Counter)* register. The *allrunning* bit in *dmstatus* is used to acknowledge that the core has resumed successfully.

In addition to full halt/resume control, the system supports hardware single-step execution. When the *step* bit in the *dcsr (Debug Control and Status Register)* is set, and the processor is resumed from debug mode, it executes exactly one instruction and automatically re-enters debug mode. This behavior allows for precise instruction-level debugging without external breakpoints or complex instrumentation. The debugger can read the updated *dpc* after each step to trace program flow in a fine-grained manner.

The core's entry into debug mode can also be triggered via exceptions or external interrupts, depending on the configuration in the *dcsr*. However, in this implementation, debug entry is typically coordinated via explicit requests from the debugger to ensure deterministic behavior and full external control.

All control signals between the Debug Module and the core are synchronized and safely latched to avoid glitches or metastable conditions. These include *haltreq*, *resumereq*, *ack_halted*, *ack_running*, and *single_step*. Custom logic ensures that only one control path dominates at any time, preventing conflicts in state transitions.

This robust run-control infrastructure ensures reliable interaction between the debugger and the processor, enabling essential debug capabilities such as program inspection, register probing, and step-by-step execution control.

3.12.8 Abstract Command Execution and Data Register Handling

The RISC-V Debug Module implements abstract commands to allow an external debugger to perform basic operations such as reading/writing general-purpose or control/status registers, and accessing memory. These commands are encoded in the *command* register and executed when the core is in debug halt mode.

The *command* register is a 32-bit register that encodes the command type and associated control fields. Two types of abstract commands have been implemented in this design: Access Register (*cmdtype* = 0x00) and Access Memory (*cmdtype* = 0x02). Execution of a command may involve one or both of the Data Registers, *data0* and *data1*, to pass arguments and results.

3.12.8.1 Access Register (cmdtype = 0x00)

This command enables access to the core's general-purpose or CSR registers, following the format detailed in table 3.20.

Bits	Field Name	Description
0-15	regno	Register number to be accessed
16	write	If set, writes data0 to the register; if clear, reads from register into data0
17	transfer	If set, initiates the actual register transfer
18	postexec	If set, the program buffer (not implemented here) would execute after transfer
19	aarpostincrement	If set, regno is incremented after each access
20-22	aarsize	Indicates register size (e.g., 2 for 32-bit)
23	reserved	Must be zero
24-31	cmdtype	Should be 0x00 for Access Register

Table 3.20: Bit fields of the command register for Access Register (cmdtype = 0x00)

Operational Flow:

- When *transfer* is set, the DM interprets the *regno* field, and performs the register access.
- If *write* is set, *data0* is written to the target register.
- If *write* is clear, the value from the register is loaded into *data0*.
- *aarpostincrement*, when set, increments *regno* after the transfer, enabling batch operations.
- The *aarsize* field should match the register width of the target (typically 2 for 32-bit).

3.12.8.2 Access Memory (cmdtype = 0x02)

This command provides abstract memory access from the hart's perspective, allowing loads and stores to memory-mapped locations while the core is halted, as outlined in table 3.21.

Bits	Field Name	Description
0-14	reserved	Must be zero
14-15	target-specific (reserved)	Reserved for future use
16	write	If set, writes value from data0 to memory; else reads into data0
17-18	reserved	Must be zero
19	aampostincrement	If set, memory address (in data1) is incremented after each access
20-22	aamsize	Indicates access size (0 = 8-bit, 1 = 16-bit, 2 = 32-bit, etc.)
23	aamvirtual	Set to 0; only physical addresses supported
24-31	cmdtype	Should be 0x02 for Access Memory

Table 3.21: Bit fields of the command register for Access Memory (cmdtype = 0x02).

Operational Flow:

- The base memory address is supplied in *data1*.
- If *write* is set, the content in *data0* is written to the memory location.
- If *write* is clear, the value at the memory location is loaded into *data0*.
- *aampostincrement*, if set, increments the memory address in *data1* post-access.

- The *aamsize* field determines the access granularity (byte/half-word/word).
- Only physical addresses are supported (*aamvirtual* = 0).

3.12.8.3 Data Registers

Two data registers are implemented:

- *data0:* The primary register for passing and receiving data during abstract command execution.
- *data1:* Used primarily for memory commands, representing the memory address (or secondary data in multi-operand instructions).

Data registers serve as operand channels between the debugger and the debug module. Their contents are valid only while the core is halted. Care is taken to ensure all abstract command execution completes before resuming the core to avoid data corruption.

3.12.9 Debug Module Registers

The Debug Module (DM) implements a set of memory-mapped registers through which an external debugger can manage and monitor core activity. These registers conform to the RISC-V Debug Specification and are accessed via the Debug Module Interface (DMI). The implemented registers allow core control, status reporting, abstract command execution, and customized debugging workflows. All registers are addressable via the DMI address space, with specific control and status bits determining the flow of debug interactions.

3.12.9.1 dmcontrol (Address:0x10):

The *dmcontrol* register is the principal control mechanism for managing core-level debug operations via the Debug Module Interface (DMI). This 32-bit register enables halt/resume requests, hart selection, and system/hart-level resets. All interactions between the debugger and the core are mediated through this register, making it critical to the functionality of the external debug interface, as detailed in table 3.22.

Bits	Field Name	Description
0	dmactive	Enables the Debug Module. Must be set to 1 before any debug operation. Clearing this bit resets the DM to its initial state.
1	ndmreset	Initiates a non-debug module reset of the target system. This is typically used to reset peripherals or the system while keeping the debug module active.
2	clrresethaltreq	Clears any previously set halt request that was configured to trigger upon reset.
3	setresethaltreq	When set, ensures that the selected hart will enter halt state immediately upon reset. Useful for debugging from the first instruction after a system restart.
4	clrkeepalive	Clears the keep-alive request, which prevents the debugger from timing out due to inactivity.
5	setkeepalive	Enables the keep-alive mechanism to maintain communication with the debugger even during long operations.
6-15	hartselhi	Upper bits of the hart selection field. Used in combination with hartsello to identify specific harts in multi-core systems.
16-25	hartsello	Lower bits of the hart selection field. Together with hartselhi, forms the full hart selection signal.
26	hasel	Hart array select enable. When set, enables hart selection through hartselhi and hartsello.
27	ackunavail	When set, acknowledges that the selected hart is currently unavailable (e.g., powered down or inaccessible).
28	ackhavereset	Acknowledges that the hart has recently undergone a reset. This bit is typically set by the hardware and cleared by the debugger.
29	hartreset	Initiates a direct reset of the selected hart. This is distinct from ndmreset, which targets the system.
30	resumereq	Requests that the selected hart resume execution from the halted state.
31	haltreq	Requests that the selected hart enter the halted state. This is the primary means by which the debugger takes control of a running core.

Table 3.22: Bit fields and description of dmcontrol register

Design Considerations and Implementation Notes:

Activation: All debug control activity is gated by the *dmactive* bit. This must be set before any requests such as *halt*, *resume*, or *reset* take effect.

- **Reset Handling:** Two reset pathways are supported: one via *ndmreset* (for full system reset without affecting the debug module), and another via *hartreset* (for selective hart-level reset). Both are implemented in coordination with an external physical reset switch for complete board-level reinitialization.
- **Halt on Reset:** The combination of *setresethaltreq* and *clrresethaltreq* enables or disables the ability to force a core to halt immediately after a reset—critical for debugging from the first instruction.
- **Single-Hart Operation:** Although this implementation targets a single-hart processor, the hart selection fields (*hartselhi*, *hartsello*, *hasel*) are retained for compliance with the RISC-V Debug Specification and future scalability.
- **No Keepalive Fields:** The *setkeepalive* and *clrkeepalive* fields described in the specification are not implemented, as no timeout or heartbeat mechanism is currently required in the system.

3.12.9.2 dmstatus (Address: 0x11):

The *dmstatus* register is a 32-bit read-only status register that reflects the real-time operational state of the Debug Module and the associated hart(s). It provides essential insight into the availability, running/halted state, and reset status of the core, as well as metadata about debug infrastructure implementation. This register is queried regularly by the external debugger to synchronize with the core's debug state, as summarized in table 3.23.

Bits	Field Name	Description
0-3	version	Indicates the version of the debug specification supported. A value of 3 indicates compliance with Debug Spec v1.0
4	confstrptrvalid	Always 0 in this implementation. Indicates whether a valid configuration string pointer is available. Not implemented.
5	hasresethaltreq	Set if the Debug Module supports halting the hart immediately after a reset. Implemented and functional.
6	authbusy	Indicates that authentication is in progress. Not implemented in this design (hardwired to 0).
7	authenticated	Indicates whether the debugger is authenticated. Not implemented (hardwired to 1).
8	anyhalted	Set if any hart is currently in a halted state. Always reflects the state of the single hart in this design.
9	allhalted	Set if all selected harts are halted. For single-hart systems, same as anyhalted.
10	anyrunning	Set if any hart is running.
11	allrunning	Set if all selected harts are running.
12	anyunavail	Set if any selected hart is unavailable (e.g., powered down). Not expected to be set in this system.
13	allunavail	Set if all selected harts are unavailable.
14	anynonexistent	Set if any selected hart index does not correspond to an implemented hart. Useful in multi-hart configurations; here, unused.
5	allnonexistent	Set if all selected harts are nonexistent.
16	anyresumeack	Indicates that any selected hart has acknowledged a resume request.
17	allresumeack	Indicates that all selected harts have acknowledged a resume request.
18	anyhavereset	Indicates that any hart has been reset since dmcontrol.dmactive was last asserted.
19	allhavereset	Indicates that all selected harts have experienced a reset.
20-21	Reserved	Hardwired to 0.
22	impebreak	Set if the core can halt due to an ebreak instruction in debug mode. Not enabled in this design.
23	stickyunavail	Sticky indication that some selected hart was unavailable at some point since last clear. Cleared only by dmactive deassertion.
24	ndmresetpending	Indicates that a system reset (ndmreset) is pending. Set by dmcontrol.ndmreset.

Design Notes:

- **Single-Hart Simplification:** In this single-core design, the *any** and *all** fields naturally mirror each other.
- **Reset Tracking:** The *anyhavereset* and *allhavereset* fields help verify whether a reset has occurred after *dmactive* was last set, which is crucial for debugging post-reset behavior.
- **No Authentication:** Since this implementation operates in a trusted and controlled environment, no authentication mechanism is required. Accordingly, *authbusy* is hardwired to 0, and *authenticated* is hardwired to 1.
- **impebreak Support:** This implementation does not support halting the core via the *ebreak* instruction in debug mode. Consequently, the *impebreak* field is hardwired to 0. Debug entry is achieved through external control via the debug module (e.g., *haltreq*) rather than software breakpoints.

The *dmstatus* register serves as a vital tool for observing and coordinating the debug flow and ensures that the external debugger can operate in a synchronized and state-aware manner.

3.12.9.3 hartinfo (Address: 0x12):

The *hartinfo* register provides static information about a given hart's debug capabilities and configuration. In a multi-hart system, this register would offer per-hart debug support characteristics; however, in this single-hart implementation, it conveys basic information applicable to the lone core, as detailed in table 3.24.

Bits	Field Name	Description
0-11	dataaddr	Base address offset of the first data register (typically data0). This field is valid and reflects the actual location used by the debug logic.
12-15	datasize	Indicates the number of implemented data registers. In this implementation, only data0 and data1 are present, hence datasize = 1 (meaning 2 registers).
16	dataaccess	Indicates whether the data registers are memory-mapped and accessible outside of abstract commands. Not supported in this design; the field is hardwired to 0.
17-19	Reserved	Reserved for future use. Must be zero.
20-23	nscratch	Number of implemented dscratch registers. This design does not implement any dscratch registers; the field is zero.
24-31	Reserved	Reserved for future use. Must be zero.

Table 3.24: Bit fields and description of hartinfo register

Design Notes:

- **Data Registers:** Only two data registers (*data0*, *data1*) are implemented, and they are exclusively used within abstract commands for operand and result exchange.
- **No dscratch:** The optional scratch registers (*dscratch0*, *dscratch1*) are not implemented, as all transient data handling during debug is managed internally or via custom registers.
- **Non-Memory-Mapped Data Access:** The data registers are not memory-mapped for external visibility and are instead accessed exclusively through debug commands (e.g., access register, access memory).

This lean configuration simplifies the debug module design while remaining compliant with essential portions of the RISC-V Debug Specification.

3.12.9.4 abstractcs (Address: 0x16):

The *abstractcs* (Abstract Control and Status) register provides information about the capabilities of the abstract command interface, as well as real-time status of the abstract command

execution engine. It plays a central role in managing the execution of abstract commands such as register and memory access operations, as detailed in table 3.25

Bits	Field Name	Description
0-3	datacount	Indicates the number of implemented data registers available for abstract commands. In this implementation, two registers are implemented (data0, data1), hence datacount = 2.
4-7	Reserved	Reserved for future use. Must be zero.
8-10	cmderr	Indicates the result of the last abstract command. The values are encoded as: 0 = success 1 = busy 2 = cmd not supported 3 = exception 4 = halt/resume error 5 = bus error 6 = reserved 7 = failed due to other reasons The debugger is expected to clear this field by writing 1s to it.
11	relaxedpriv	Indicates whether abstract commands are permitted to access registers across privilege boundaries. Not supported in this design (hardwired to 0).
12	busy	Set to 1 if an abstract command is currently executing. This design sets and clears this bit as needed during command handling.
13-23	Reserved	Reserved for future use. Must be zero.
24-28	progbufsize	Indicates the size of the implemented program buffer. As the program buffer is not implemented in this design, this field is hardwired to 0.
29-31	Reserved	Reserved for future use. Must be zero.

Table 3.25: Bit fields and description of abstractcs register

Design Notes:

- **Program Buffer:** This design does not implement a program buffer, and all abstract operations are executed directly via the debug module logic.

- **Error Handling:** The *cmderr* field plays a critical role in communicating failure states such as unsupported commands or unresponsive core behavior. Errors are latched until explicitly cleared by the debugger.
- **Busy Flag:** During execution of abstract commands, the *busy* flag is set to prevent concurrent command issuance. It is cleared once the command completes or fails.
- **Simplicity and Conformance:** By omitting complex features like *relaxedpriv* and *program buffer*, the implementation remains lightweight while fulfilling required functionality as per the RISC-V Debug Specification.

3.12.9.5 command (Address: 0x17):

The *command* register is used by the debugger to issue abstract commands to the debug module. It defines both the type of operation to perform and the parameters necessary to execute that operation. In this implementation, only Access Register and Access Memory command types are supported (corresponding to *cmdtype* = 0x0 and *cmdtype* = 0x2 respectively). The *command* register is 32 bits wide and must be written before setting the dmcontrol.haltreq bit or issuing a command through the abstract interface, as shown in table 3.26 and table 3.27.

Bits	Field Name	Description
0-15	regno	Register number to be accessed
16	write	If set, writes data0 to the register; if clear, reads from register into data0
17	transfer	If set, initiates the actual register transfer
18	postexec	If set, the program buffer (not implemented here) would execute after transfer
19	aarpostincrement	If set, regno is incremented after each access
20-22	aarsize	Size of the register access: 2 = 32-bit (as used in this design).
23	reserved	Must be zero
24-31	cmdtype	Should be 0x00 for Access Register

Table 3.26: Bit fields and description of the command register for Access Register (cmdtype = 0x00)

Bits	Field Name	Description
0-14	reserved	Must be zero
14-15	target-specific	Reserved for future use
16	write	1 = Write operation, 0 = Read.
17-18	reserved	Must be zero
19	aampostincrement	Increments memory address after access if set.
20-22	aamsize	Indicates access size (0 = 8-bit, 1 = 16-bit, 2 = 32-bit, etc.)
23	aamvirtual	Set to 0, as all accesses are to physical memory.
24-31	cmdtype	Should be 0x02 for Access Memory

Table 3.27: Bit fields and description of the command register for Access Memory (cmdtype = 0x02).

Execution Semantics:

- **Command Lifecycle:** A command is written to the *command* register, and its associated operands (such as register or memory address) are placed in *data0* and/or *data1*. The command execution begins immediately and sets the *abstractcs.busy* bit.
- **Completion and Error Reporting:** Upon completion, the *busy* bit is cleared and the result is indicated in *abstractcs.cmderr*. If a command fails due to an invalid regno, unsupported size, or bad cmdtype, appropriate error codes are latched.
- **Post-Increment:** This feature allows automated iteration through consecutive registers or memory addresses, simplifying multi-word transactions.

3.12.9.6 data0, data1 (Addresses: 0x04–0x05):

The *data0* and *data1* registers are 32-bit wide memory-mapped registers used as the primary data interface between the debugger and the target core through the debug module. These registers are part of the abstract command mechanism and are used for both register and memory access operations.

Purpose and Usage (see 3.28**):**

- ***data0:*** This is the primary data register used for transferring values to/from the RISC-V core or memory.

- ***data1:*** This auxiliary register is primarily used to hold memory addresses during abstract memory access commands.

These registers are critical for read and write transactions initiated through the command register. Their content is interpreted based on the cmdtype field in the command, which determines whether the access is targeted at a core register or memory.

Register	Width	Purpose	Access Context
data0	32-bit	Main operand/result data register	Register and Memory Access
data1	32-bit	Memory address or auxiliary data	Mainly Memory Access

Table 3.28: Description of dataN registers

Implementation Notes

- Both registers are implemented with simple latching mechanisms.
- They are not cleared after command execution and retain their last value until overwritten.
- No locking mechanism is enforced; coherence is ensured by command flow control and the busy signal in abstractcs.

3.12.9.7 Custom Registers (custom0 to custom6) (Addresses: 0x70–0x76):

To support staged bring-up and validation of the debugger independently of the RISC-V core, a set of custom debug registers (*custom0* through *custom6*) has been integrated. These registers implement two essential mechanisms:

1. **Bypass:** Simulates responses from the RISC-V core for abstract commands, enabling the debugger to function even in the absence of a running core.
2. **Probing:** Observes and captures signal values as they arrive at the Debug Module from the core, without interfering with normal operation.

Both mechanisms operate simultaneously, ensuring that while the debugger can function using simulated inputs, the actual signal paths from the core can still be verified for correctness and consistency (refer to table 3.29).

Register	Role	Description
custom0	Core Bypass (Access Register)	Bypasses the data read from a register (regno) when executing Access Register abstract commands.
custom1	Core Bypass (Access Memory)	Bypasses the memory data read from the address pointed by data1 during Access Memory commands.
custom2	Core Bypass (Other Signals)	Bypasses all non-data signals from the core to the DM, such as halted, cmd_err, busy, etc.
custom3	Lock / Core Bypass Selector	Acts as a security lock: core bypass mode is only enabled if this register holds a specific pre-defined hex sequence (e.g., 0xABCD1234). Prevents accidental core bypass activation.
custom4	Probe (Access Register Data)	Probes the data being routed to the DM from the core as a result of an Access Register command.
custom5	Probe (Access Memory Data)	Probes the data read from memory (via data1) that is routed to the DM from the core for Access Memory commands.
custom6	Probe (Other Signals)	Probes additional debug-related signals flowing from the core to the DM—mirrors what custom2 bypasses but from the DM's observation point.

Table 3.29: Description of the custom registers

Simultaneous Bypass and Probing:

The design allows bypass mode (enabled via *custom3*) and signal probing to operate in parallel:

- When the debugger is in bypass mode, it relies entirely on values from *custom0*, *custom1*,

and *custom2* to respond to abstract commands.

- At the same time, *custom4*, *custom5*, and *custom6* do not interfere with signal flow but passively record what the core would have sent to the DM, allowing for diagnostic and verification use.

This design ensures:

- Debugger development and testing can proceed independently of core integration.
- Real-time visibility into core-DM interactions, improving traceability and observability.
- Bypass logic does not suppress or overwrite actual core signals; both views coexist for comprehensive testing.

3.13 Debug Core integration

3.13.1 Debug Mode Entry

When the debugger requests for the debug entry, the processor enters in the debug mode. Currently we have three main debug entry points.

1. ***Haltreq* goes high (Async debug entry):** If the processor is in normal operation (pipeline operation) and it detects the *haltreq* signal, then we also enter the Debug state if the processor is not freezed. As soon as we enter the debug mode, *pending_async_dbg* goes low.
2. *resethaltreq* goes high (Reset debug entry)
3. Single Stepping
4. EBREAK (Sync debug entry)

Debug Mode is a special processor mode used only when a hart is halted for external debugging. Because the hart is halted, there is no forward progress in the normal instruction stream. How Debug Mode is implemented is not specified here.

Following things happen during debug mode entry:

1. Kills all the instructions in the pipeline.
2. All interrupts are masked using *interrupt_dbg_global* signal and also by writing mstatus[MIE] = 1'b0
3. All traps are disabled
4. All the debug instructions are executed as they do in M-mode except control instructions (**Branch and Jumps**) irrespective of their destination address, all PC related instructions like **AUIPC** act as illegal istructions.

3.13.2 RESET

If the *halt* signal (driven by the hart's halt request bit in the Debug Module) or *hasresethaltreq* are asserted when a hart comes out of reset, the hart must enter Debug Mode before executing any instructions, but after performing any initialization that would usually happen before the first instruction is executed.

3.13.3 HALT

Before entering the debug mode, the processor is halted when it encounters any of the debug entry indication through CSR or through external signals. In HALT mode, we do following things:

1. *cause* is updates with the cause of debug
2. *prv* is set to 3 and *v* is set to 0 to indicate Machine Mode.
3. *dpc* is set to the next valid instruction which needs to be executed when the core leaves the debug mode.
4. Kill all the instructions in the pipeline
5. write mstatus[MIE] <= 1'b0, to disable all the Software, timer and external interrupts.
6. Assert *interrupt_dbg_global* signal
7. PC <- dm_halt_addr (Program buffer)

When the processor is halted, it waits for signal from debugger that whether it has to execute the abstract command or start the execution from the program buffer. If processor wants to single step the instruction, then the debugger has to write the DCSR[step] = 1'b1 using the abstract command and then it can go in the single stepping mode. If processor recevies the *resumereq_i*, then it goes back to the functional (normal runnning) mode.

If while executing the program buffer EBREAK occurs, then the PC goes to the starting of program buffer and the processor is again halted. All the interrupts are disbaled in the HALT mode.

3.13.4 DEBUG EXIT SEQUENCE

When we get *resumereq_i* as 1'b1, following sequence is followed.

1. Fetch PC is retored with the one from *dpc*.
2. Previllage levels are restores as it was before entering the debug mode.
3. Leave the debug mode.

3.13.5 Single Steping

This method is only available to external debuggers, and is the preferred way to single step.

1. If the external debugger sets the ***step*** bit before resuming a halted hart, the processor will execute exactly one instruction before automatically re-entering Debug Mode.
2. If a trap (e.g., exception or interrupt) occurs while executing the instruction, the processor will enter Debug Mode right after the PC is updated to the trap handler address. The *mtval* and *mcause* CSRs will be updated however, the trap handler is not executed; it's immediately halted again.
3. If the executed instruction changes the PC to a location that would cause an exception (e.g., instruction fetch fault), that exception is deferred and will only be taken next time the hart resumes.

3.13.6 Debug FSM

Figure 3.45 shows the debug FSM implemented in the core.

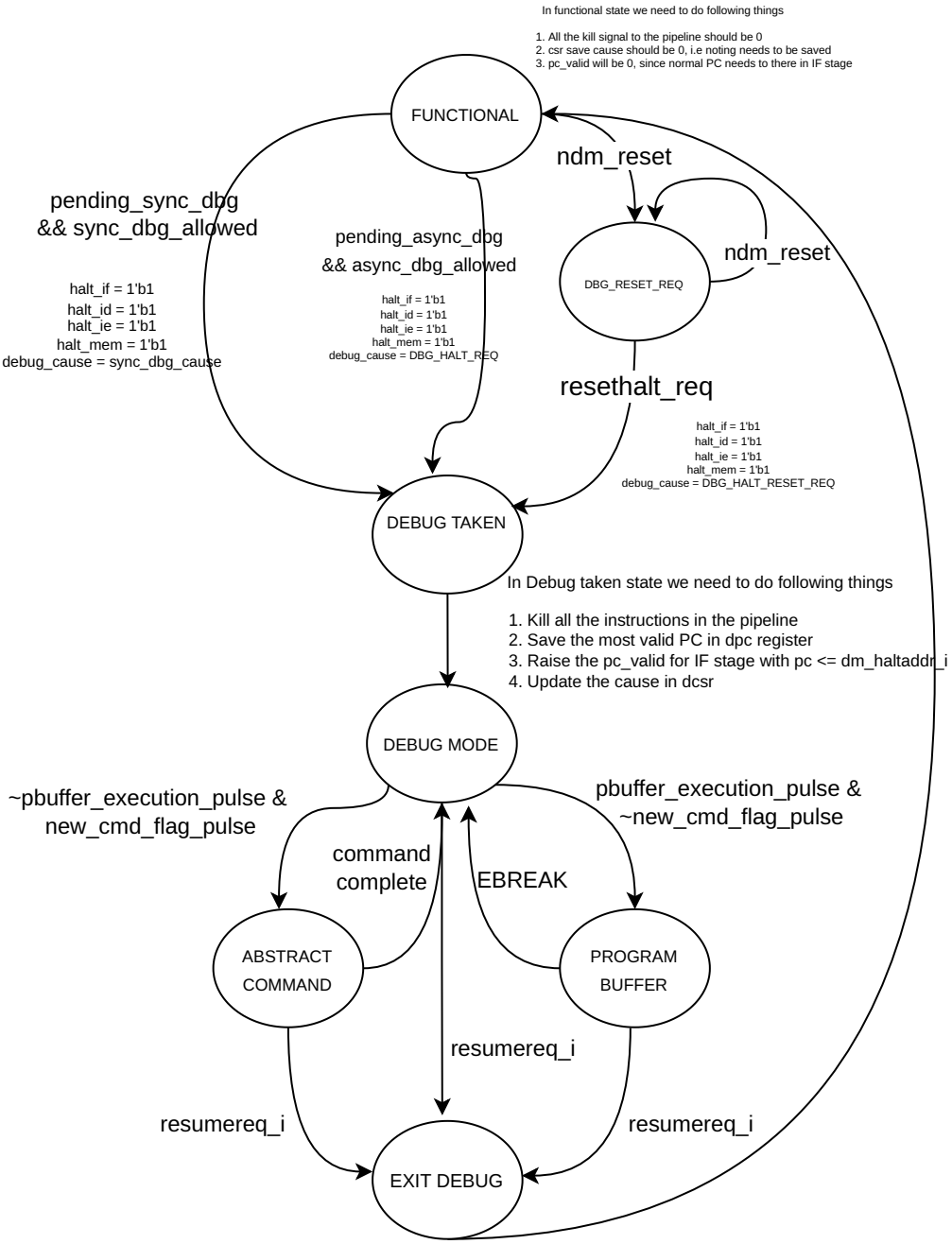

Figure 3.45: Debug FSM

FUNCTIONAL State: Upon coming out of reset, the processor checks the *haltreq_i* signal. If this signal is not asserted, the processor enters the FUNCTIONAL state. In this state, the processor behaves normally—fetching instructions from instruction memory, executing them through the pipeline, and writing results back to the register file or memory. Entry into debug mode is not permitted while the processor is stalled, such as during memory latency (ICACHE/DCACHE) or FPU operations. Even if debug requests such as *ndmreset* or *haltreq_i* are asserted during this time, the FSM defers action until the processor becomes idle and resumes normal flow. This behavior ensures that the processor only transitions into debug when it is safe to do so, preserving pipeline integrity and consistency.

DBG_RESET_REQ State:

If a non-debug module reset (*ndmreset*) is received while in the FUNCTIONAL state, the FSM transitions to the DBG_RESET_REQ state. In this state, the processor remains in reset, in accordance with the debug spec, until the *ndmreset* signal is deasserted. Once the reset is lifted, the FSM checks the *resethaltreq_i* signal. If this is asserted, indicating a request to enter debug mode immediately after reset, the FSM transitions to the DBG_TAKEN state. Otherwise, it stays in DBG_RESET_REQ or returns to FUNCTIONAL depending on other pending conditions. This mechanism allows a debugger to take control of the processor directly out of reset.

DBG_TAKEN State:

The DBG_TAKEN state handles the initiation of a debug session. The FSM transitions here if *haltreq_i* is asserted or if an appropriate debug entry condition is detected (e.g., from a triggered breakpoint, step event, or ebreak instruction). In this state, the cause of the debug event is written into the dcsr.cause CSR as defined in the RISC-V Debug Specification. If the debug cause is not due to single-step execution, the currently executing program counter is stored in the *dpc* register. The processor's PC is then loaded with the *dm_halt_address*, which points to the beginning of the debug program buffer, and the *debug_mode* signal is asserted. From here, the FSM transitions into the DBG_MODE state, indicating that the processor is now fully in debug mode and halted for external inspection or instruction.

DBG_MODE State:

In the DBG_MODE state, the processor is halted and remains idle, waiting for instructions from the debug host. It allows the debugger to inspect and modify state or initiate specific debug actions. If the debug module receives a *new_cmd_flag*, the FSM transitions to the ABSTRACT_COMMAND state to execute abstract commands, which may include register access or memory operations. Alternatively, if the *pbuffer_execution* signal is asserted, the FSM enters the PROGRAM_BUFFER state to begin executing a sequence of instructions from the program

buffer. The FSM stays in DBG_MODE as long as no execution requests are pending, providing a safe and passive state for debug interaction.

ABSTRACT_COMMAND State:

The ABSTRACT_COMMAND state manages the execution of abstract commands issued via the debug module interface (DMI). These commands are typically used to read/write registers or access memory. The processor executes these commands one at a time through the instruction pipeline. Once a command is successfully executed, the FSM sets the command_complete flag and returns to DBG_MODE to await the next command. If a resume request (*resumereq_i*) is detected during or after command execution, the FSM prepares to exit debug mode and transitions to the DBG_EXIT state.

PROGRAM_BUFFER State:

In the PROGRAM_BUFFER state, the processor begins executing instructions from a predefined buffer area (*dm_halt_address*) loaded during the DBG_TAKEN state. This mode is used for running short debug programs, such as test sequences or small memory routines. Execution continues until an EBREAK instruction is encountered, which serves as an exit condition. Upon reaching EBREAK, the FSM transitions back to DBG_MODE to await further commands. Similar to the ABSTRACT_COMMAND state, if *resumereq_i* is asserted during program buffer execution, the FSM will instead transition to DBG_EXIT.

DBG_EXIT State:

The DBG_EXIT state handles the safe exit from debug mode. Here, the PC is restored with the value saved in the *dpc* register, effectively resuming execution from the instruction where the core had been halted. The *debug_mode* signal is deasserted, and the FSM returns to the FUNCTIONAL state, resuming normal operation. This transition ensures that any modifications made during debug mode are correctly applied, and program control is seamlessly handed back to the running application.

Chapter 4

ASIC Design

4.1 ASIC Motivation

Designing an ASIC on the 65nm TSMC node offers a compelling balance between performance, power efficiency, and cost, making it an ideal choice for academic and research-driven processor development. Our motivation to implement a 32-bit RISC-V G-class core at this node stems from the opportunity to realize a silicon-proven, standards-compliant processor that demonstrates practical scalability and manufacturability. This experience forms a critical step toward bridging the gap between high-level architectural design and real-world VLSI implementation.

4.2 RTL Design

While the architecture largely adheres to the standard RISC-V specifications, certain optional components—specifically the Floating Point Unit (FPU), Memory Protection Unit (MPU), and Debug Unit—have been temporarily excluded from synthesis. These modules are in the early stages of RTL development and require further stabilization before they can be reliably synthesized. However, the rest of the core is complete and includes all major functional blocks necessary for full operation. These include the Instruction Fetch Unit, Decode Stage, Execute Stage, Load-Store Unit (LSU), Branch Prediction Unit (BPU), Exception Handling Unit, Control and Status Register (CSR) block, as well as fully integrated instruction and data caches, each with its respective Translation Lookaside Buffers (ITLB and DTLB). This subset of the design enables us to focus on synthesizing and validating the stable portions of the core while

continuing development and integration of the more complex auxiliary units.

4.3 RTL Verification

Following the RTL development, we proceeded with functional simulation using the Xcelium simulator from Cadence. The RV32IM test suite was executed on the RTL through a self-checking testbench that monitors and validates the processor's behavior against expected outputs. Each test was run individually, and detailed logs were captured to aid in future debugging, especially in the case of test failures. One of the key challenges encountered during this phase was adapting the test infrastructure from an earlier Vivado-based environment to the Cadence ecosystem. While both simulators serve the same purpose, subtle differences in how they handle memory initialization, test vector loading, and file I/O automation required careful adjustments. Ensuring consistent behavior across simulations and creating an efficient, automated flow for loading instruction and data memory files were crucial steps in establishing a robust and repeatable verification setup in Xcelium. The testcases and their status is shows in table 4.1.

Testcase Name	**Number of testcase**	**Status**
Integer instruction	20	20/20 PASS
Load Instructions	6	6/6 PASS
Store Instructions	6	6/6 PASS
CSR Instructions	1	PASS
Exceptions	1	PASS
Interrupts	1	PASS
Hello World printing	1	PASS
Cache Testing	1	PASS
Branching	6	6/6 PASS
Hazard testing	2	2/2 PASS

Table 4.1: Testcases Run

4.4 LINTING

After verifying the RTL for functional correctness, the next crucial step in ASIC design is to perform linting, which we carried out using the Cadence Jasper SuperLint tool. Linting plays a vital role in ensuring design quality by identifying inconsistencies between the RTL code and its intended behavior. It helps uncover issues such as combinational loops, simulation-synthesis mismatches, and poor design practices early in the flow, reducing the risk of downstream integration or timing failures.

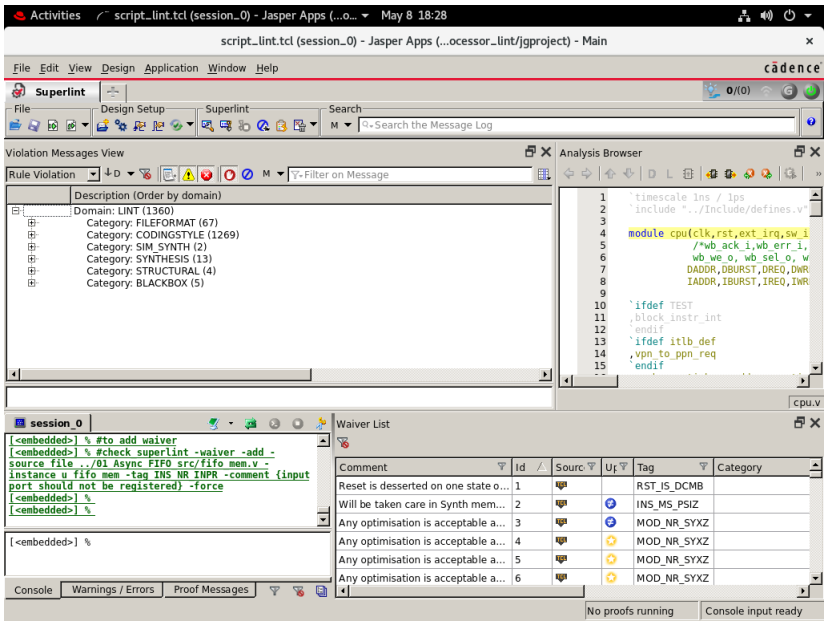

Figure 4.1: Linting using Jasper Gold

Based on the linting results, several modifications were made to improve the robustness and synthesis-friendliness of our design. Firstly, all **synchronous resets** were converted to **asynchronous resets** to ensure proper behavior during initialization and recovery. We also eliminated any **direct connections between input and output ports**, as these can lead to simulation and synthesis mismatches. All **registers were explicitly initialized**, and the design was thoroughly checked to ensure that **no combinational loops** were present. Additionally, any **inferred latches** were reviewed to confirm whether they were intentional or resulted from incomplete conditional statements. For all finite state machines (FSMs), **default values were assigned to outputs** to avoid unintended latch inference or undefined behavior. Other structural improvements were implemented to prevent potential issues such as synth-sim mismatches and race conditions, ensuring the design is clean, consistent, and ready for synthesis.

4.5 Genus Synthesis

Cadence Genus is a high-performance RTL synthesis tool used in ASIC design flows for converting RTL code (typically in Verilog or VHDL) into a gate-level netlist optimized for area, power, and timing. It is part of the broader Cadence digital implementation suite and supports advanced synthesis features like clock gating, datapath optimizations, and constraint-driven synthesis. Genus is commonly used in industry-standard ASIC flows due to its scalability and efficient optimization capabilities.

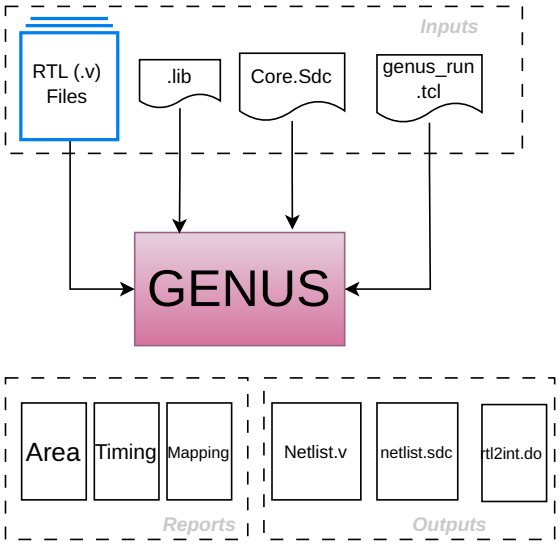

Figure 4.2: Genus input and output files

To perform synthesis with Genus, the following **input files** are typically required also shows in figure 4.2:

1. RTL Files: Verilog or VHDL source code describing the hardware design.
2. Constraint Files: Typically in SDC (Synopsys Design Constraints) format. Includes clock definitions, input/output delays, timing exceptions, and false paths.
3. Library Files: Technology-specific standard cell libraries in Liberty (.lib) format. May also include physical abstracts (.lef) if floorplanning follows.
4. Configuration/Tcl Scripts: Tcl scripts to automate flow control (read files, elaborate, apply constraints, synthesize, write outputs).

Following outputs are generated post synthesis:

1. Gate-Level Netlist (.v): The synthesized Verilog netlist mapped to standard cells.
2. Constraint-annotated Reports (.sdf): Setup/Hold violations, path slack, cell usage, etc.
3. Area and Power Reports: Gives insight into design utilization and power estimates (dynamic and leakage).
4. Timing Reports (.rep): Post-synthesis STA report based on the applied constraints.
5. Design Check Reports: Reports on combinational loops, unconnected ports, inferred latches, and other warnings.

The synthesis methodology using the Cadence Genus tool follows a structured and constraint-driven approach to transform RTL into a gate-level netlist optimized for area, power, and timing.

The process begins with the **preparation stage**, where all RTL design files, technology libraries, and constraint files (typically in SDC format) are gathered and organized. This is followed by the **elaboration** and analysis phase, during which Genus reads and elaborates the RTL to create an internal design representation and checks for structural correctness, connectivity, and design rule violations. After elaboration, **user-defined constraints are applied** to define clocking, input/output delays, and timing exceptions. These constraints guide the timing engine to establish the setup and hold relationships within the design. Once constraints are in place, the synthesis engine **maps the RTL logic to standard cell gates** using the provided technology library. Genus performs a series of logic optimizations, such as retiming, resource sharing, and constant propagation, to meet the design goals for area, power, and performance. In cases where aggressive optimization is needed, multi-pass synthesis strategies may be employed to iteratively improve results. After synthesis, the tool generates a **gate-level netlist** along with detailed **timing, area, and power reports**. These reports help identify critical timing paths, violations, and design bottlenecks. The final stage involves verifying the correctness of the synthesized netlist, checking for issues such as latch inference, combinational loops, or unused logic. Once verified, the synthesized netlist and associated reports are handed off to the physical design team for place and route using tools like Cadence Innovus. This methodology ensures a clean, optimized, and implementation-ready netlist that aligns with the design specifications and physical constraints of the target technology.

Figure 4.3 shows the syntheisized design schematic of a) Pipeline Design b) Icache c) Dcache

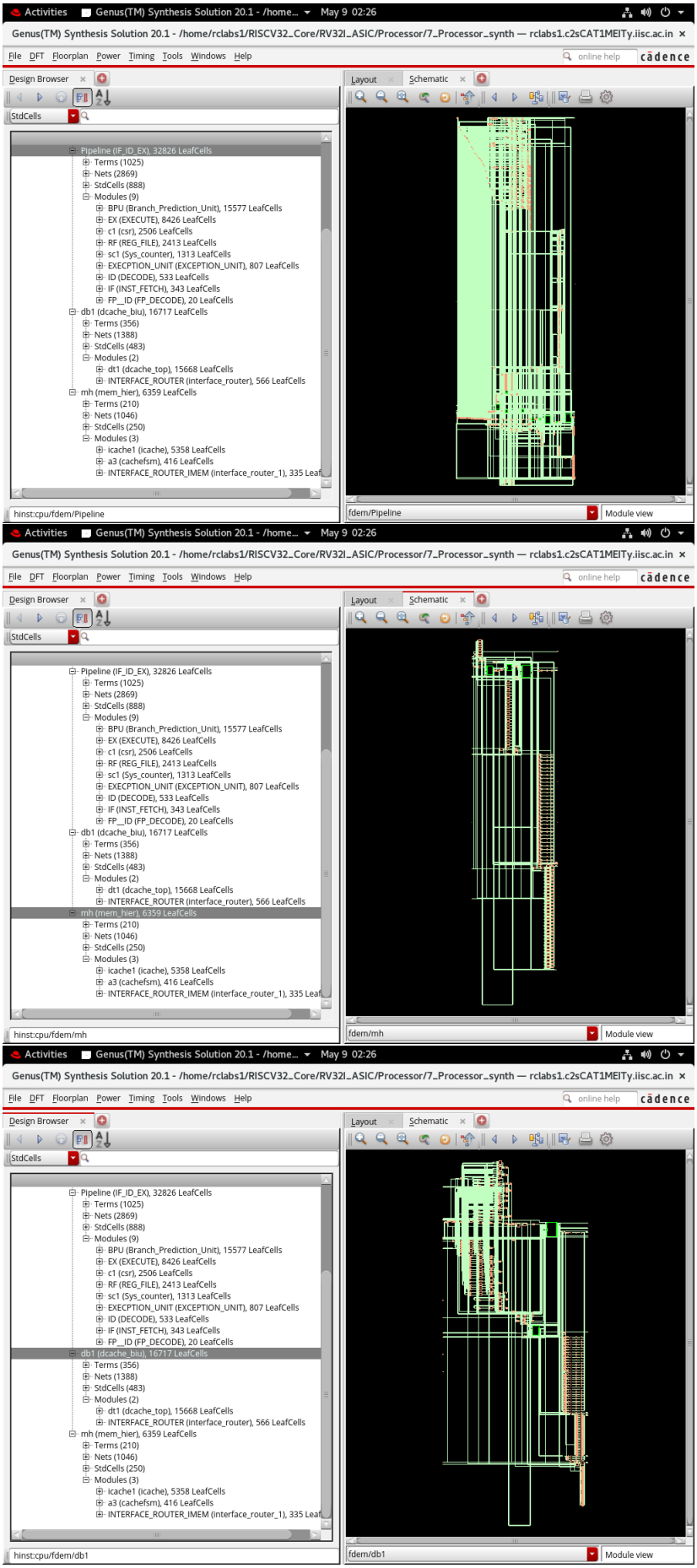

Figure 4.3: Post-synthesis schematic

4.6 Gate Level Simulations

Post-synthesis Gate-Level Simulation plays a vital role in the ASIC design process by verifying the functional accuracy of the synthesized netlist while considering real-world timing constraints. In contrast to functional simulation—which is based on RTL and assumes ideal, zero-delay conditions—GLS operates on the netlist generated after synthesis and incorporates actual timing data, typically provided through SDF (Standard Delay Format) annotation. This enables the simulation to account for realistic delays, setup and hold times, and any logic optimizations introduced during synthesis. As a result, GLS is effective at detecting issues such as race conditions, timing violations, and uninitialized registers—problems that often go unnoticed in RTL simulation. Moreover, GLS can uncover bugs that static timing analysis (STA) or formal equivalence checking tools might miss. GLS flow is shows in figure 4.4.

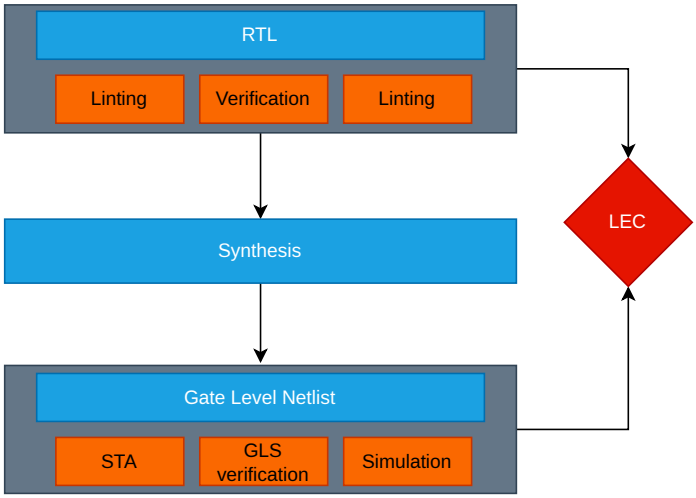

Figure 4.4: Gate Level simulation flow

GLS proves valuable in several scenarios, including:

- Addressing the shortcomings of Static Timing Analysis (STA), such as:
  - STA's inability to detect issues related to asynchronous interfaces
  - The need for precise validation of static timing constraints, including those involving false paths and multi-cycle paths
- Ensuring proper system startup by verifying the initialization process and confirming the correctness of the reset sequence

4.6.1 Zero Delay Simulations

In the initial phases of Gate-Level Simulation (GLS), while the design is still undergoing timing closure, zero-delay simulations are often used to validate functional correctness. These simulations run significantly faster than those with full timing. Zero-delay mode can be activated using the ***-NOSpecify*** option. However, since all signal transitions occur at the same simulation time, there's a higher risk of introducing race conditions. To mitigate this, some delay may be added to gates.

Cadence provides multiple delay modes for gate-level simulation, such as *delay_mode_path*, *delay_mode_distributed*, *delay_mode_unit*, and *delay_mode_zero*. Replacing detailed delay modes like path or distributed with global delays like zero or unit can substantially speed up simulation. These simplified delay modes are especially useful during the debugging phase, when verifying design functionality takes precedence over timing accuracy.

4.6.1.1 Identifying zero delay loops

The Xcelium simulator includes a built-in feature that identifies potential zero-delay combinational loops and generates a warning when such loops are found. Detecting these loops is crucial, as they can cause glitches that may propagate through the design. This feature can be activated for Verilog simulations by using the ***-GAteloopwarn*** option on the command line.

4.6.1.2 Handling Zero-delay race conditions

Race conditions in zero-delay mode can arise because all gates have zero propagation delay by default.

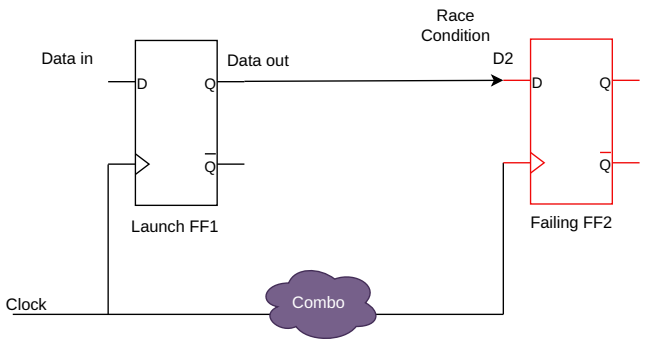

Figure 4.5: Zero delay loops

As shown in Figure 4.5, such issues can occur when there is combinational logic in the clock path during zero-delay simulation. In this scenario, a race may happen, causing the data at FF2 to be captured at the same clock edge as it exits FF1.

To resolve this, a unit delay (e.g., #1) can be introduced at the output of FF1 to ensure that its data is delayed and thus gets latched into FF2 on the following clock edge. One way to implement this is by inserting a buffer with a unit delay, as shown in the following example:

buf #1 buf _ prim _ delay1 (D2 _ FF2, Data _ OUT _ FF1); //Add buffer with unit delay.

Alternatively, you can use the -seq_udp_delay 50ps command-line switch. This option introduces a specified delay (in this case, 50 picoseconds) to the input/output paths of all sequential UDPs (User Defined Primitives) in the design. Figure 4.6 provides an example of this technique applied to a shift register.

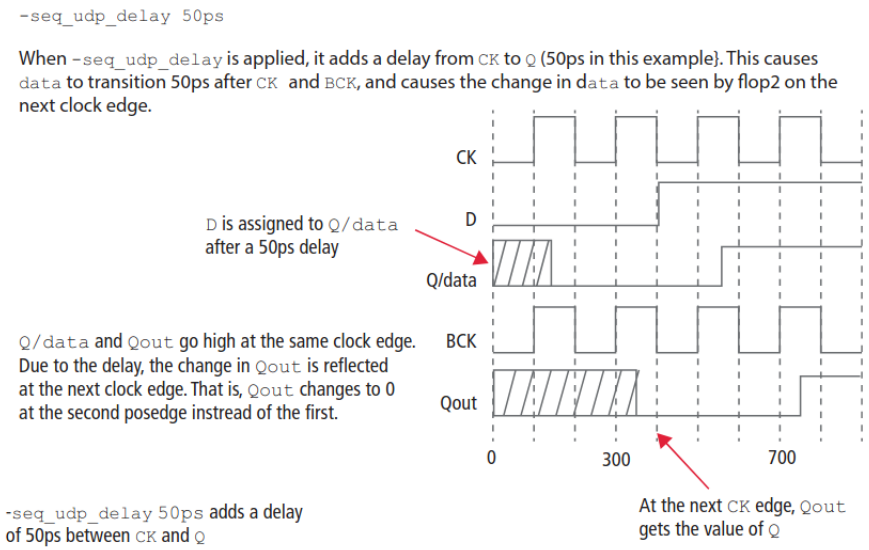

Figure 4.6: Sequential UDP delay

Once these settings are completed and design is verified for its functionality in zero delay GLS, we can go ahead with the SDF simulations. Figure 4.7 shows the status of testcase passing in zero delay simulations

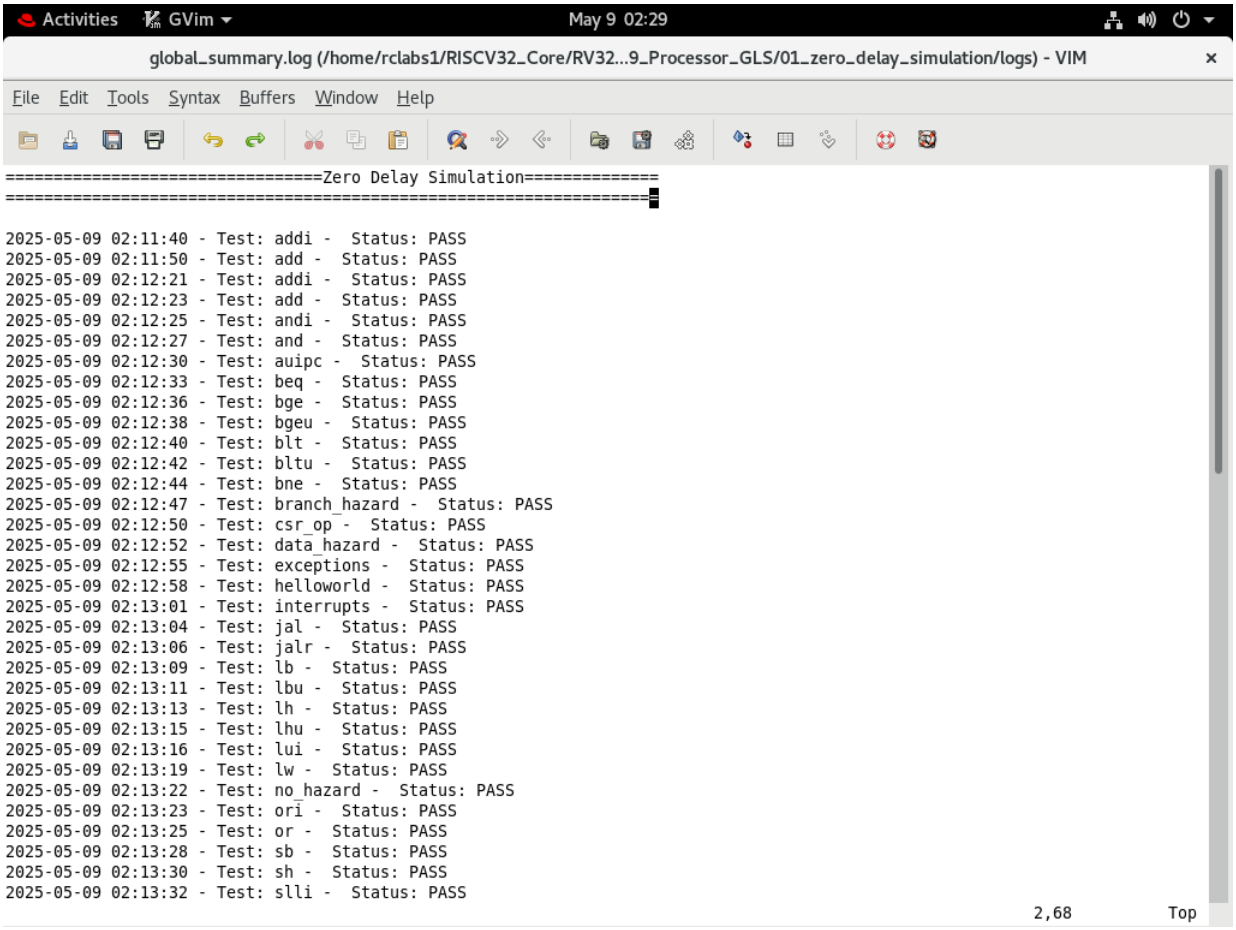

Figure 4.7: Zero Delay simulations

4.6.2 Standard Delay Format Simulations

SDF (Standard Delay Format) simulations are a specialized form of gate-level simulation where precise timing delays are back-annotated into the netlist using an .sdf file generated after synthesis or place-and-route. These delays represent real-world propagation times through gates and interconnects, capturing setup and hold checks, clock skew, and data path delays. SDF simulations are essential for verifying the timing behavior of a design under realistic conditions, ensuring that it meets all timing constraints such as clock-to-Q, combinational path delays, and input/output timing.

The testbench design may change as we move from functional simulation to SDF simulations, hence ‘define or ‘ifdef SDF kind of macros could be used if we selectively want a part of testbench to be active during that testing.

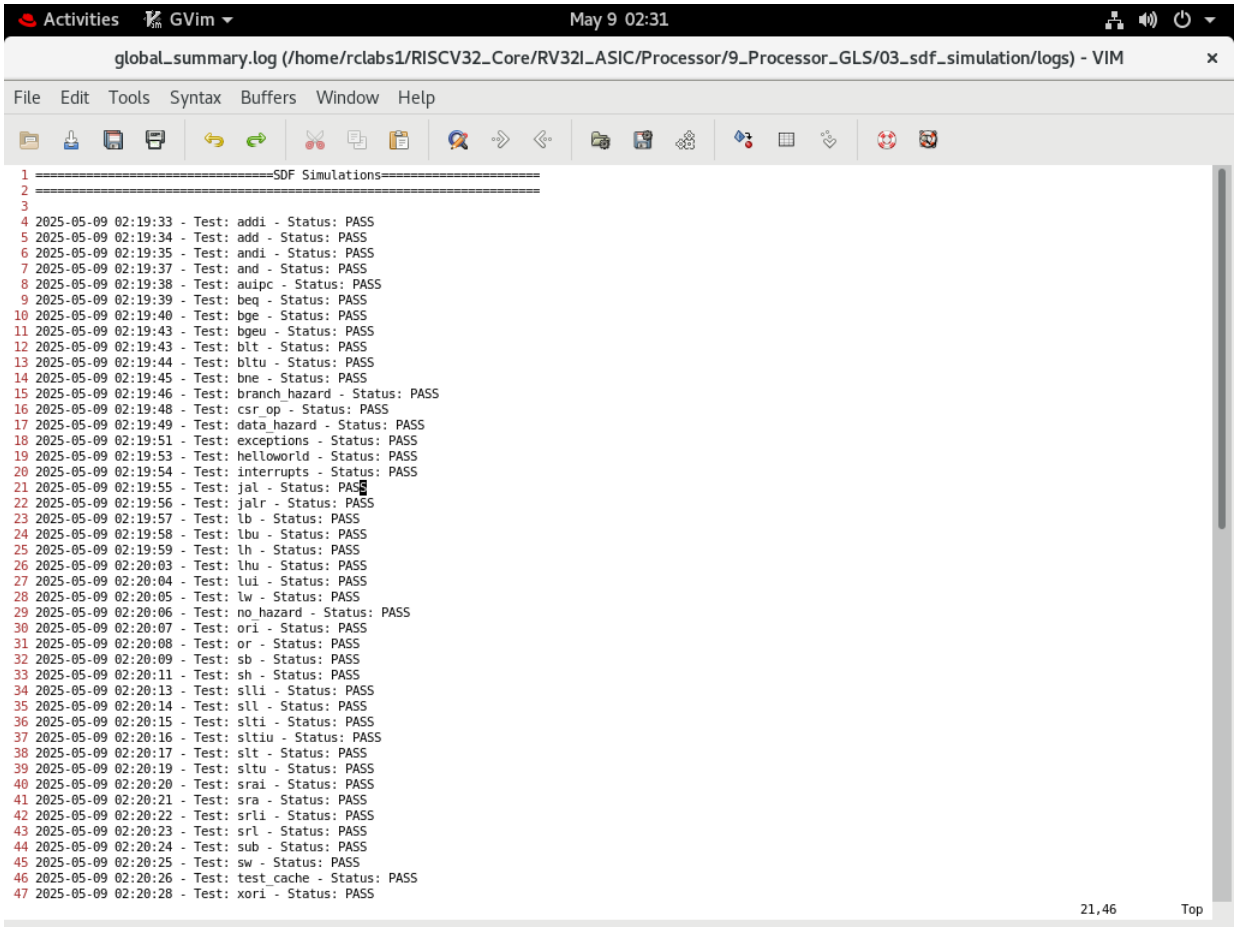

Figure 4.8: SDF Simulations

Chapter 5

Verification

5.1 Unit Testing

Initially, a directed testbench approach was employed during RTL development to inject instructions into the processor and observe its internal behavior. This allowed us to validate the pipeline stages, control flow, data forwarding, and exception handling in isolation before moving to full system-level testing. Behavioral logs and waveform analysis were critical during this phase, enabling detailed tracking of signal transitions, register file updates, memory transactions, and pipeline stalls, which facilitated quick debugging and refinement of the RTL. Following features were tested:

1. Verification of machine Mode privilege level and CSR instruction handling.
2. Verification of Memory Management Unit (MMU) with Page table walker and TLB filling.
3. I-Cache and D-Cache memory transactions
4. Various internal exceptions such as load and store page faults, misaligned memory accesses, and illegal instructions. The core was observed for correct trap handling behavior, accurate CSR updates (like mcause and mtval), and recovery via mret.
5. The interrupt controller was tested using both asynchronous interrupt generation and timer interrupts, verifying correct transition into and out of trap handlers, proper CSR updates, and handling of nested or masked interrupts.

6. Verification of processor debug module, with functionalities like halt, resumte, single-step, breakpoint and verified the execution of abstract command and program buffer.

7. Memory Protection Unit functionality was verified by configuring regions using CSRs like pmpaddr and pmpcfg, and testing enforcement of access permissions across privilege boundaries.

5.2 Formal Verification

Once the individual functionalities of the processor were thoroughly verified through directed test cases, the next phase of verification focused on evaluating the processor's behavior during full instruction execution. This involved testing how the core handles dynamic scenarios that arise during real program execution, such as data hazards, control hazards, and pipeline stalls caused by instruction dependencies or memory latency. These conditions cannot be fully captured through isolated unit-level tests, and therefore required system-level validation.

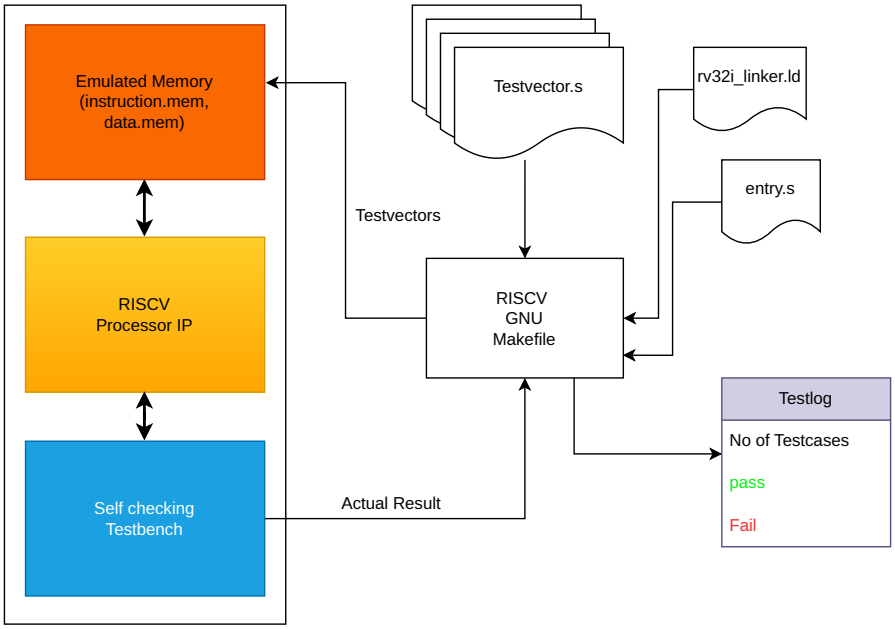

Figure 5.1: Formal Verification environment

To achieve this, we developed a comprehensive verification environment as shown in figure 5.1, combining a simulation testbench with the RISC-V GNU toolchain. Using this setup, we compiled C programs into RISC-V binaries that reflect realistic instruction mixes and execution

patterns. These binaries were loaded into the processor's memory, and the core was allowed to fetch, decode, and execute instructions autonomously.

We took testcases from *risc_testcase* github repository which is used for benchmarking the processors and performance measurements.

Below figure 5.2 shows the result of the formal verification of our processor. It is passing all the test cases.

```
Test Case Results (Table Format):
================================================
TEST CASE NAME | STATUS  | SIMULATION TIME | PC_BREAK VALUE
================================================
bne                  | PASS   | 36700 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000144
slti                 | PASS   | 38700 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
or                   | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
addi                 | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
sw                   | PASS   | 122200 ns  | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000019c
and                  | PASS   | 37100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000154
csr_op               | PASS   | 43300 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 000001a4
ori                  | PASS   | 38700 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
sra                  | PASS   | 39100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000016c
jal                  | PASS   | 41100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000184
data_hazard          | PASS   | 48200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 000001f8
sb                   | PASS   | 42200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000017c
sh                   | PASS   | 42400 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000184
andi                 | PASS   | 37000 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000150
beq                  | PASS   | 37000 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000148
jalr                 | PASS   | 41300 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000018c
xori                 | PASS   | 37000 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000150
srl                  | PASS   | 40900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000017c
lh                   | PASS   | 40500 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000174
bgeu                 | PASS   | 39100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
sltiu                | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
lw                   | PASS   | 40500 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000174
lb                   | PASS   | 40200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000168
srai                 | PASS   | 38700 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
lbu                  | PASS   | 40200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000168
sll                  | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
no_hazard            | PASS   | 43800 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 000001b8
bltu                 | PASS   | 37400 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000158
sub                  | PASS   | 39000 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000168
exceptions           | PASS   | 55300 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000248
auipc                | PASS   | 39200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000170
interrupts           | PASS   | 53300 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 000001ec
helloworld           | PASS   | 49500 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000160
bge                  | PASS   | 39100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
sltu                 | PASS   | 39100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000016c
slt                  | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
add                  | PASS   | 39200 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000170
branch_hazard        | PASS   | 45600 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 000001f8
lhu                  | PASS   | 40500 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000174
xor                  | PASS   | 37100 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000154
lui                  | PASS   | 38700 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 0000015c
srli                 | PASS   | 38900 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000164
blt                  | PASS   | 37400 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000158
slli                 | PASS   | 37000 ns   | DONE=1 TIMEOUT=0 PASS=1 FAIL=0  | PC_BREAK_VALUE= 00000150
```

Figure 5.2: Formal Verification Result

Chapter 6

Results

The RISC-V processor developed in this work has been evaluated across several key design metrics including area, operating frequency, power consumption, and performance. The initial version of the core was implemented and validated on an FPGA platform, where its functionality and performance were verified through simulation and execution of directed test cases. This allowed early validation of the instruction pipeline, control logic, and integration features such as interrupts and debug support. After ensuring functional correctness on FPGA, the design was synthesized using the Cadence Genus synthesis tool, targeting a 65nm technology node. This enabled accurate estimation of gate-level area, timing characteristics, and resource utilization under realistic ASIC constraints. The synthesis reports form the basis for the quantitative analysis presented in this chapter.

6.1 Processor Performance

Processor's performance is measured in terms of number of instructions executed per cycle (CPI). The number of instructions executed and the number of clock cycles required for the execution are mesured using cycle and instret performance counter CSRs. Processor's throughput is measured through IPC (Instructions per clock cycle). The table 6.1 mention the CPI from 4 difference benchmark programs.

Code	No of Instructions executed	No of clock cycles taken	IPC	CPI
QuickSort	846	1173	0.72	1.38
Binary Search	220	454	0.48	2.06
Selection Sort	753	1029	0.73	1.36
Exchaning Number	155	376	0.41	2.42

Table 6.1: Benchmark Codes

The values of CPI (or IPC) varies from code to code and depends how many number of loops, number of branches and number of memory instructions are there.

6.2 Processor Area

We used Cadence Genus to evaluate the area, timing, and gate-level characteristics of our RISC-V processor. After successful synthesis, the area report generated by Genus provided a detailed breakdown of the design's physical footprint. Given table 6.2 shows the total area of our Processor, during 3 different phase of synthesis.

Metric	generic	map	Incremental	final
Cell Area	1,394,233	1,164,744	1,160,756	1,160,756
Total Cell Area:	1,394,233	1,164,744	1,160,756	1,160,756
Leaf Instances:	114,980	58,394	56,052	56,052
Total Instances:	114,980	58,394	56,052	56,052

Table 6.2: Processor area metrix

Genus synthesize the design in 3 phases: Generic, map and final. Generic synthesis is the initial phase where the RTL (e.g., Verilog or VHDL) is converted into a technology-independent

representation using generic logic gates like AND, OR, MUX, and DFF. Mapping is the process of transforming the generic netlist into a technology-specific netlist using a standard cell library (e.g., 45nm or 7nm cells). Incremental synthesis is an iterative or localized re-synthesis approach applied when changes are made to a portion of the design (e.g., ECOs or constraint changes), without re-optimizing the entire design.

The given table 6.3 shows the detail distribution of area among modules in our design (CPU).

Instance	**Cell count**	**Total Area**
CPU	56052	1160756
FDEM	55908	1160578
Pipeline	32826	379391
BPU		
EX		
I_DECODE	533	1116
EXECPTION_UNIT	807	3669
I_FETCH	343	1514
CSR	2506	18405
REG_FILE	2413	13952
Sys_counter	1313	5582
dcache_biu	16717	421222
Interface_Router	566	3799
dcache_top	15668	415131
dcache_dpram	6190	375915
DTLB	3622	21002
dcache_ram_fsm	5524	22314
dcache_ram_fsm_1	2580	10698
mem_hier	6359	359926
Interface_Router	335	3134
cache_fsm	416	1154
Icache	5358	353853
ITLB	3407	19817

Table 6.3: Submodules Area

6.3 Processor Timing

In terms of timing performance, the processor was first deployed and tested on an FPGA platform using Xilinx Virtex-7 series. On this platform, the maximum operating frequency achieved was **28 MHz.** However, this relatively lower frequency is primarily influenced by the routing delays inherent in FPGA architectures, where the majority of the critical path delay arises from net delays rather than the actual logic delays. FPGAs typically introduce higher parasitic capacitances and less optimized interconnects compared to ASICs, which impacts achievable clock speeds.

To obtain a more accurate measure of the processor's true timing potential, the design was later synthesized using Cadence Genus targeting a 65 nm technology node. Post-synthesis analysis revealed that the processor can operate at a frequency of 50 MHz, with a positive setup slack of 56 ps, indicating that the timing constraints are comfortably met. This result demonstrates the improved efficiency and timing closure achieved in an ASIC environment where placement and interconnects are optimized for performance. The timing report of the design in given in table 6.4

Metric	**generic**	**map**	**Incremental**	**final**
Slack(ps)	11,447	580	54	54
R2R (ps)	14,135	10,369	9,750	9,750
I2R (ps)	17,843	14,399	13,974	13,974
R2O (ps)	11,447	580	54	54
I2O (ps)	15,155	4,611	4,278	4,278

Table 6.4: Processor timing metrix

From the timing metrics summarized in the table, it is evident that the most critical paths in the design originate from register-to-output (R2O). The R2O slack progressively tightens from 11,447 ps in the generic stage to just 54 ps in the final synthesized netlist, indicating that output paths are the primary timing bottleneck. Detailed path analysis reveals that these critical paths terminate at the WDATA output pin of the processor, which is responsible for driving write-back data onto the external interface.

This bottleneck is expected in a soft IP core, where output drive and boundary logic often form the longest combinational paths. However, since the processor is intended to be used as a soft-IP block, it provides the flexibility to tightly integrate or couple the processor directly with the system interconnect or memory subsystem in the SoC design.

6.4 Processor Power

In Cadence Genus, power estimation is performed by analyzing the synthesized netlist in conjunction with the technology library (.lib) files, which contain detailed models of standard cell characteristics, including switching capacitance, leakage currents, and net parasitics. Genus uses this information along with activity data—either from functional simulation or static vectorless estimation—to compute different components of power consumption.

57.53 mW (Net power) and 89.456 mW Switching power

Module Design: cpu Operating Conditions: ss1p08vm40c (Worst case tree)					
Instance	Cells	Leakage (nW)	Internal(nW)	net (nW)	Switching (nW)
cpu	56052	302900.077	31919464.489	57537343.086	89456807.575

Table 6.5: Processor power metrix

The total dynamic power is broken down into two primary components:

1. Switching Power, which accounts for the energy consumed due to toggling of logic gates and nets during clock transitions.

2. Net Power, which represents the power dissipated across the interconnects and routing resources as signals propagate through the design.

In our synthesis results, Genus reported a Switching Power of 89.456 mW, indicating the significant dynamic activity in the processor due to data-path operations and clocking logic. Additionally, the Net Power was reported as 57.53 mW, which highlights the energy consumption attributed to signal routing across the design. These values are derived from RTL switching activity estimates and standard cell-level power models, providing a realistic and technology-specific view of power consumption, critical for evaluating the design's suitability for low-power or thermally constrained applications.

Chapter 7

Appendix

7.1 ICACHE Design

The processor boots from the address 32'h0000_0000, which is the base address of the instruction memory (BOOT ROM) in our case.

When the first time processor sends the BASE_ADDRESS, we'll have a icache miss. Then the virtual address is given to the TLB block and physical address is given out with a tag_valid signal. In our case, since we have disabled the translation for ease of use, we'll have physical address = virtual address. When the tag_hit is 1'b1 and hit_out is 1'b0, then the state machine inside the **cachefsm** block goes to the service miss state(2'b01).

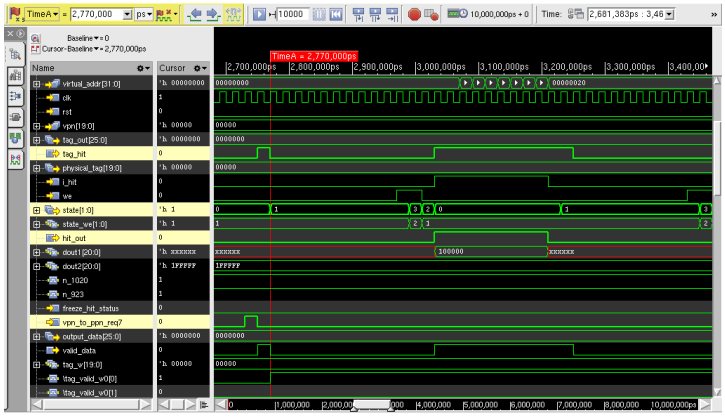

Figure 7.1: Initial icache miss

In the state SERVICE_MISS, the physical address is given to the instruction memory for fetching the instruction in burst. We have a cache line of 256bits, that is 8 words (32-bits).

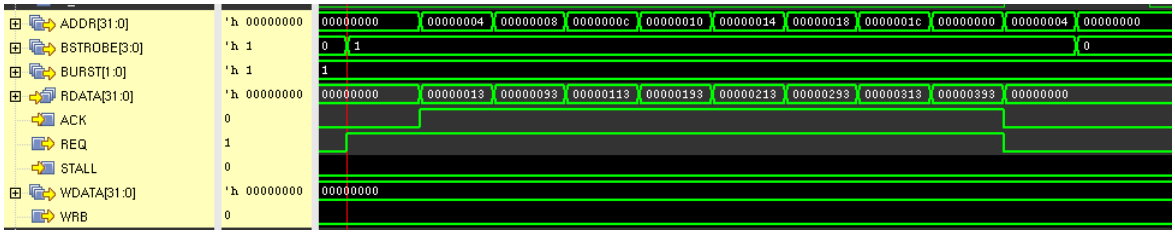

Figure 7.2: Address burst to fetch instruction

When the whole cache line is fetched, the cachefsm signals we to the icache controller for writing the cache line in to the memory. Physical tag along with the tag_valid is also written to the tag array memory. When *we* comes from the cachefsm, the FSM inside the icache controller state *state_we* goes to 2 (2'b10). In state 2, it cache line and tag is written on to the ICACE SRAM, and in next cycle it again comes to state 1.

Parallely, states of cachefsm are also moving. When the state of cachefsm is 2'b10, it raises the vpn_to_ppn_req_3 signal, which goes to the icache controller and the state goes to 0.

This vpn_to_ppn_req_3, is used to read the tag again from the TLB and compare it with the tag coming from the TAG SRAM ARRAY. It it matches, then it is a hit otherwise miss.

When the cache hits, it sends the *instruction[31:0]* to the pipeline unit using block select counter.

Now then the PC goes to 32'h0000_0020, the index increments and we encounter a cache miss, since the tag from the TLB(physical) doesnt matches with that from the memory. So signal i_hit goes down. Now the state in cachefsm again goes to SERVICE_MISS. At this moment, the tag_valid[VA] for the next upcoming way is written 1'b1 and cache again fetched the next line from the memory.

7.2 GLS Issues

1. **Hit issue during CACHE miss for the 1st time**

CURRENT WAVEFORM

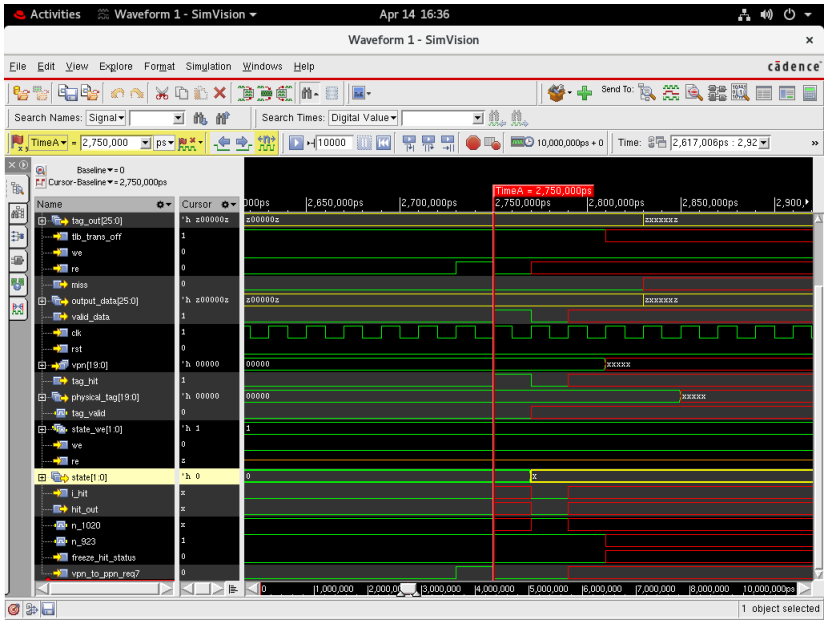

Figure 7.3: Current Waveform

EXPECTED WAVEFORMS

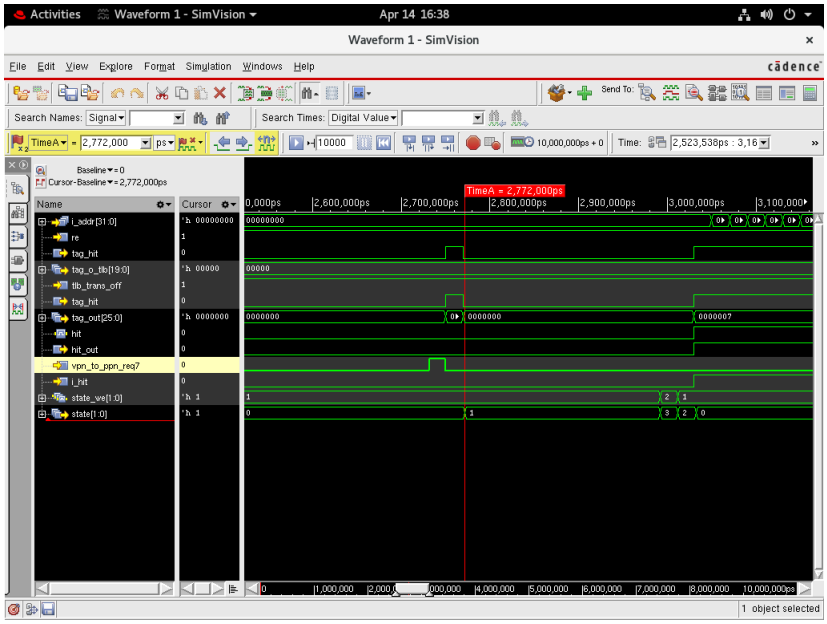

Figure 7.4: Expected Waveforms

For the moment, I have initialised all the Memory output of dout1_mem[20:0]

force tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout1_mem = 21'h1fffff; force tb_platform.riscv_platform
= 21'h1fffff; xcelium> force tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout1_mem =
21'h1fffff; force tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout2_mem = 21'h1fffff;

After that, hit goes fine.

Then we have the issue at the Address line of the ICACHE (fdem.mh.interface_router.ADDR).

Lower 2 bits of the address is going "Z"

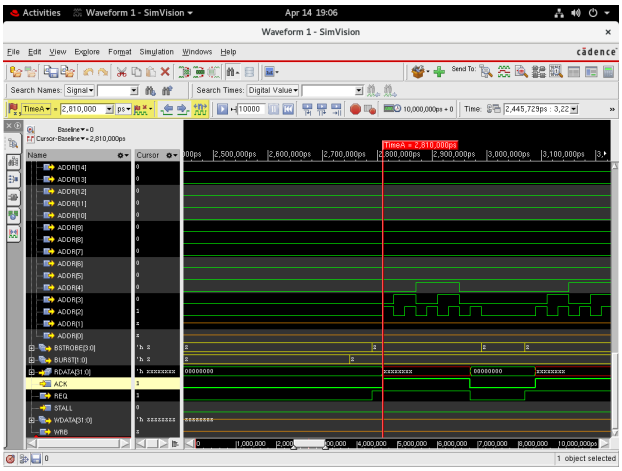

Figure 7.5: Address lower bits

0000ns: >> At 0ns, put the force on both dout1_mem and dout2_mem, run till **2800ns**

2800ns: >> At 2800ns, Release the force on dout1, and run the simulations for **400ns** more

3200ns: >> At 3200ns, Again force the dout1_mem to 21'h1fffff, and run the simulations for 200ns

3400ns: >> At 3400ns, Release the force on dout1_mem

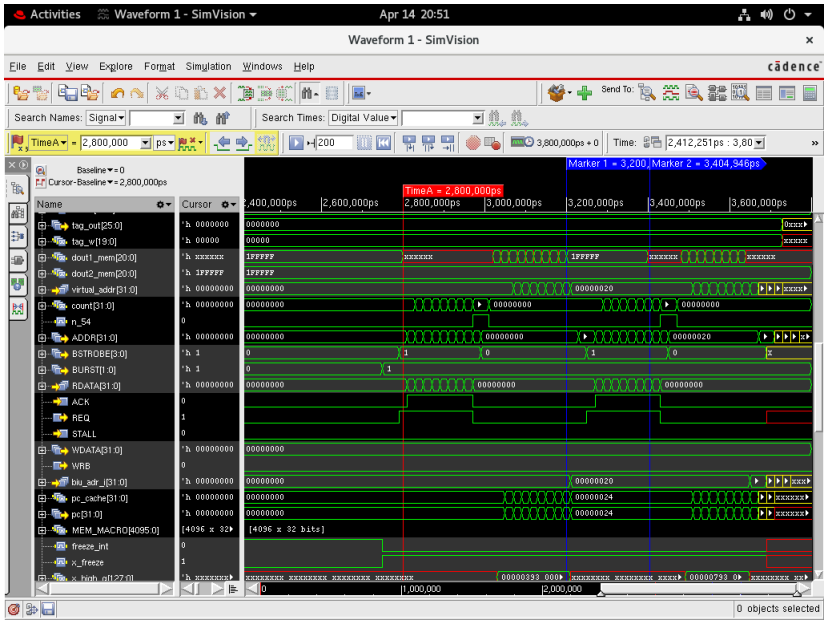

Figure 7.6: Force Mem output

SOLUTION: All the memories will be initialised to X in the GLS. Hence the RTL code needs to be designed in such a way that it doesnt read the memory before it writes something on to it. Hence we designed the tag_valid register which holds the value 1'b1, if that cache line is valid.

```
hit = (tag_hit ? ((tag_valid_w1_read && (tag==dout2[(tag_last_bit - tag_start_bit):0])) ? 1'b1 :
                  (tag_valid_w0_read && (tag==dout1[(tag_last_bit - tag_start_bit):0])) ? 1'b1 : 1'b0)
                  : 1'b0);
```

Figure 7.7: RTL fix

Here when the tag_hit is 1'b1, and tag_valid_w1_read is 1'b0, it doesnt matter what the dout values are and the hit will remain to 1'b0 cleanly.

2. **Delay Loops inside the designs:** Designs contain many paths with sequential elements, which have no clock to q delay. All the # delays will be removed out from the design once the synthesis is done.

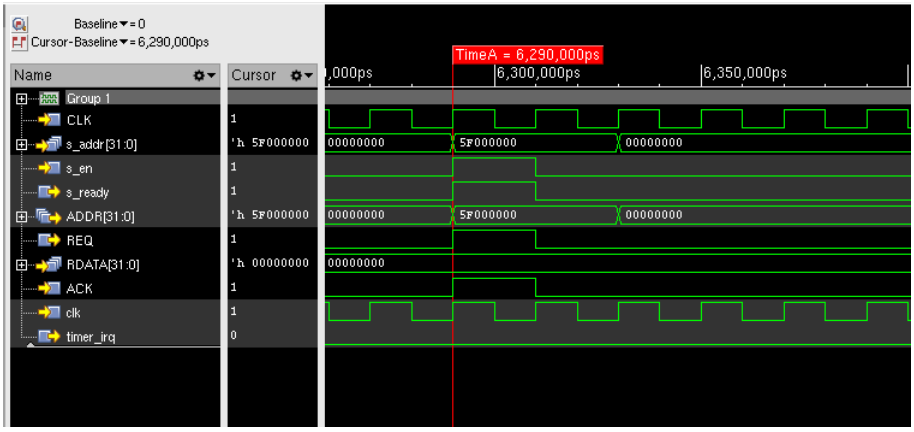

Figure 7.8: CLINT Req-Ack issue

In the above case, *s_en,* comes along with the address *s_addr.* The s_ready has to be 1'b1 in the next clock cycle, but in functional it works fine since we have given # delay to the flops, but doesnt work in zero delay or unit delay simulations.

```
// READY assertion
always @(posedge CLK or posedge RST) begin
  if(RST) s_ready <= 1'b0;
  else begin
    if(s_en && ~s_ready) s_ready <= 1'b1;
    else s_ready <= 1'b0;
  end
end
```

Figure 7.9: RTL code

SOLUTION: add the ***-seq_udp_delay 200ps*** to the script to make the output of all the flops to come after a delay of 200ps to model the tcq for correct functionality.

7.3 Genus elaboration script

```
### Template Script for RTL->Gate-Level Flow
######### Preset global variables and attributes ###########
set DESIGN cpu
set GEN_EFF high
set MAP_OPT_EFF high
set clockname clk     #set clockname write_clk set DATE [clock format [clock seconds] -format "%b%d-%T"]
set _OUTPUTS_PATH output_files set _REPORTS_PATH report_files
set _LOG_PATH logs

###################################
## Library setup
```

```
#################################

set_db / .script_search_path
{. ./Scripts} set_db / .init_hdl_search_path {. ./Include ./RTL_source}
set_db / .init_lib_search_path
{/home/rclabs1/RISCV32_Core/RV32I_ASIC/Processor/7_Processor_synth/Synthesis}
set_db / .library
"tsdn65lplla128x128m4f_200a_ss1p08vm40c.lib \
tsdn65lplla128x21m8f_200a_ss1p08vm40c.lib \
tsdn65lplla128x32m8f_200a_ss1p08vm40c.lib \
tsdn65lplla128x56m8f_200a_ss1p08vm40c.lib \
tsdn65lplla2048x2m8m_200a_ss1p08vm40c.lib \
tcbn65lplvtwc_ccs.lib"

set_db auto_ungroup none
set_db / .information_level 7
set_db / .max_cpus_per_server 20


#################################
## Load Design
#################################
puts "load RTL"

set verilog_files [glob ./RTL_source/*.v]
read_hdl $verilog_files

puts "elobrate design" elaborate $DESIGN
#################################33
##memory black box and include area details
#################################3

# Mark memory instances as black boxes (do not synthesize, but include in area calculation)
# Prevents optimization #set_ dont_use false [get_lib_cells TSDN65LPLLA2048X16M8M] ;# Ensures area is considered
puts "Runtime & Memory after 'read_hdl'" time_info Elaboration check_design -unresolved


#################################
## Constraints Setup
#################################
read_sdc ./Constraints/Core.sdc
report_timing -lint -verbose #break
puts "The number of exceptions is [llength [vfind "design:$DESIGN" -exception *]]"
 if {![file exists ${_OUTPUTS_PATH}]} {
  file mkdir ${_OUTPUTS_PATH}
  puts "Creating directory ${_OUTPUTS_PATH}"
  }
    if {![file exists ${_REPORTS_PATH}]} {
  file mkdir ${_REPORTS_PATH}
  puts "Creating directory ${_REPORTS_PATH}"
  }
 if {![file exists ${_LOG_PATH}]} {
  file mkdir ${_LOG_PATH}
  puts "Creating directory ${_LOG_PATH}"
  }

set_db lp_power_analysis_effort high
#################################
## Synthesizing to generic
#################################

set_db / .syn_generic_effort $GEN_EFF
syn_generic
puts "Runtime & Memory after 'syn_generic'"
time_info GENERIC
report_dp > $_REPORTS_PATH/generic/${DESIGN}_datapath.rpt
write_snapshot -outdir $_REPORTS_PATH -tag generic
report_summary -directory $_REPORTS_PATH
write_hdl  > ${_OUTPUTS_PATH}/${DESIGN}_generic.v
write_sdc > ${_OUTPUTS_PATH}/${DESIGN}_generic.sdc
#break
#################################
```

```
## Synthesizing to gates
################################

set_db / .syn_map_effort $MAP_OPT_EFF
syn_map
puts "Runtime & Memory after 'syn_map'"
time_info MAPPED
write_snapshot -outdir $_REPORTS_PATH -tag map
report_summary -directory $_REPORTS_PATH
report_dp > $_REPORTS_PATH/map/${DESIGN}_datapath.rpt
write_hdl  > ${_OUTPUTS_PATH}/${DESIGN}_map.v
write_sdc > ${_OUTPUTS_PATH}/${DESIGN}_map.sdc

#write_do_lec -revised_design fv_map -logfile ${_LOG_PATH}/rtl2intermediate.lec.log > ${_OUTPUTS_PATH}/rtl2intermediate.lec.
    do
write_do_lec -revised_design ${_OUTPUTS_PATH}/${DESIGN}_map.v -logfile ${_LOG_PATH}/rtl2intermediate.lec.log > ${
    _OUTPUTS_PATH}/rtl2intermediate.lec.do

################################
## Optimize Netlist
################################
set_db / .syn_opt_effort $MAP_OPT_EFF
syn_opt
write_snapshot -outdir $_REPORTS_PATH -tag syn_opt
report_summary -directory $_REPORTS_PATH

puts "Runtime & Memory after 'syn_opt'"
time_info OPT

write_snapshot -outdir $_REPORTS_PATH -tag final
report_summary -directory $_REPORTS_PATH
write_hdl  > ${_OUTPUTS_PATH}/${DESIGN}_incremental.v
## write_script > ${_OUTPUTS_PATH}/${DESIGN}_m.script
write_sdc > ${_OUTPUTS_PATH}/${DESIGN}_incremental.sdc

###### SDF file Generation ############
write_sdf -version 2.1 -recrem split -setuphold merge_when_paired -edges check_edge > ${_OUTPUTS_PATH}/Core.sdf

################################
### write_do_lec
################################

write_do_lec -golden_design fv_map -revised_design ${_OUTPUTS_PATH}/${DESIGN}_incremental.v
-logfile  ${_LOG_PATH}/intermediate2final.lec.log > ${_OUTPUTS_PATH}/intermediate2final.lec.do
##Uncomment if the RTL is to be compared with the final netlist..
#write_do_lec -revised_design fv_map -logfile ${_LOG_PATH}/rtl2intermediate.lec.log > ${_OUTPUTS_PATH}/rtl2intermediate.lec.
    do

puts "Final Runtime & Memory."
time_info FINAL
puts "============================"

puts "Synthesis Finished ........"
puts "============================"
```

Chapter 8

Concluding remarks

Github Repository of our project: https://github.com/SanidhyaSaxena21/RV32G_Core

8.1 User instructions

This brief manual outlines the steps required to compile and simulate a C program on the custom-designed RISC-V 32-bit soft-core processor. To successfully compile and simulate programs on this processor, ensure the following tools are installed and properly configured:

- GNU RISC-V 32-bit Toolchain (target: riscv32-unknown-elf)
- Xilinx Vivado Design Suite

The flow of running a C code on the processor involves two main stages:

1. Compilation of C Code to Instruction and Data Memory Files: Before simulation, your C source code needs to be compiled into RISC-V assembly and then converted to .mem files required for initializing the FPGA memory.
   (a) Compile .c to .s (Assembly): Use the following command to compile your C code:
      i. **riscv32-unknown-elf-gcc -S -march=rv32im -mabi=ilp32 -o program.s program.c** : This will generate an assembly file program.s from your C source program.c.
   (b) Generate instruction.mem and data.mem: Run the provided script:

i. **./generate_testcase.sh program.s rv32i_linker.ld entry.s**: This script will generate instruction.mem and data.mem (if your program contains a data section).

2. Running Simulation on Vivado: Once you have the memory files, run the simulation using Vivado in batch mode:

   (a) **vivado -mode batch -source run_test.tcl**

Important Notes

- Ensure that your assembly code or compiled program ends with an ebreak instruction. This is essential for the processor to halt gracefully after program execution and for the testbench to detect completion.
- You can inspect simulation logs and waveform data (if enabled) to verify register values, memory states, and instruction flow.

To compile and run the design for ASIC synthesis, follow these instruction:

1. RTL Compilation (*Xcillium*): xrun -f filelist.f -top riscv_platform -access+rwc
2. RTL Simulation (*Xcillium*): RTL simultion can be run in 2 ways, either a directed test case or a regression.

   (a) Direct Test: ./run_sim.tcl +test=<test_name> -gui

   (b) Regression: ./run_test_all.tcl

3. LINTINT (*Jasper gold*): jg -superlint -script run_lint.tcl
4. Genus synthesis: For genus synthesis we should have following files

   (a) RTL Files: ./RTL_Sources/*

   (b) Library Files: ./Synthesis/lib...

   (c) SDC Constraints: ./Constraints/core.sdc

   (d) genus run script: ./Scripts/core_run.tcl

To run the genus synthesis, follow these commands

- genus -gui

- source ./Scripts/core_run.tcl

This will generate the netlist.v files, along with the sdf annotation files required for the Gate level simulations. Report files like timing report, area report, power report, critical STA paths will be dumped by the tool. LEC do files will also be dumped

8.2 Corrections

We are currently making the Soc from our processor which will integrate core with peripherals like UART, CLINT, SPI, PWM etc. These will be used for the DEMO purposes for the FPGA board. Also the debug is currently under testing, hence commands for invoking debug is still not mentioned.

8.3 Suggestions for next gen

Currently the RTL is matured enough to be used as a soft-core IP. Also the RTL is synthesizable for ASIC flow. For further batches, it is strictly recommended to go ahead with the ASIC flow in following direction:

1. Understand the RTL and design thoroughly, run vivado simulations and testcases for better understanding
2. Understand the ASIC flow, go through the documentation of CDAC and sample FIFO project to get a hands-on practice of the flow.
3. We'll be moving to the lower technology nodes (45nm or 28nm), that will introduce mutiple challenges like STA, hold violations, new memory models etc. So replace all the previous memory models with the newer ones, and update the library files in the genus scripts.
4. Synthesize the RTL and check whether all the testcases are passing SDF simulation, there should be no 'X' propogation in design.
5. Look for critical paths in the design and try to optimize the design for performance. Majorly the paths via Dcache needs to be optimised for increasing the clock frequency.

6. Once the synthesized RTL is verified, go for placement of macros, power-net connectivity, routing and layouting of the design. Clear all the DRC and LVS errors (if any).

7. Run the post-route timing simulations on different PVT corners.

8. If everything works fine, we can go ahead with the GDS (Tape-out).

8.4 Future scope

The core can be outsourced to different vendors for creating a Soc for low-power application like power meters and watches. Also the core can be used as a microcontroller, which can be used for various embedded applications. We can build a PCB around the processor, which can serve the need of in-house computings and various applications then can be build upon it. It will eliminate the need for relying on ARM based microcontrollers.

Bibliography

[1] RISC-V International: https://riscv.org/

[2] BBC Research: https://www.bccresearch.com/market-research/semiconductor-manufacturing/global-risc-v-technology-market.html

[3] https://semico.com/content/risc-v-cpu-market-analysis-sip-socs-ai-and-design-starts

[4] https://riscv.org/technical/specifications/

[5] https://wiki.riscv.org/display/HOME/RISC-V+Technical+Specifications

[6] C. Boulet et al., "VexRiscv: A configurable and efficient RISC-V CPU for FPGAs," GitHub Repository, [Online]. Available: https://github.com/SpinalHDL/VexRiscv

[7] T. Paravan, "NEORV32: An open-source, customizable RISC-V microcontroller," GitHub Repository, [Online]. Available: https://github.com/stnolting/neorv32

[8] Codasip, "Codasip L110 Core Datasheet," Codasip, 2022. [Online]. Available: https://www.codasip.com/product/l110/

[9] D. Patterson et al., "SweRV Core EH1," Western Digital, 2019. [Online]. Available: https://github.com/chipsalliance/Cores-SweRV

[10] SiFive, "U84 Core Complex," SiFive Documentation, 2021. [Online]. Available: https://www.sifive.com/cores/u84

[11] Micro Magic Inc., "RISC-V CPU runs at 5 GHz using 200 mW," EE Times, 2020. [Online]. Available: https://www.eetimes.com/micro-magic-unveils-5-ghz-risc-v-core/

[12] A. Waterman, Y. Lee, D. A. Patterson, K. Asanovic, V. I. U. level Isa, A. Waterman, Y. Lee, and D. Patterson, "The risc-v instruction set manual," 2014.

[13] K. Asanovi and D. A. Patterson, “Instruction sets should be free: The case for risc-v,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug 2014. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[14] A. Waterman, K. Asanovic and SiFive Inc., “The RISC-V Instruction Set Manual Volume I: Unprivileged ISA,” Document Version 20191213, EECS Dept, University of California, Berkeley, CA, USA, December 13, 2019

[15] B. Zimmer et al., "A RISC-V Vector Processor With Simultaneous-Switching Switched-Capacitor DC–DC Converters in 28 nm FDSOI," in IEEE Journal of Solid-State Circuits, vol. 51, no. 4, pp. 930-942, April 2016, doi: 10.1109/JSSC.2016.2519386.

[16] Milanesi, L., 2023. Study and Development of RISC-V Mitigation Approach Based on Redundancy (Doctoral dissertation, Politecnico di Torino).

[17] Berry, G., and Gonthier, G. The Esterel synchronous programming language: Design, semantics, implementation. Science of Computer Programming 10, 2 (1992).

[18] Bluespec Inc. Bluespec(tm) SystemVerilog Reference Guide: Description of the Bluespec SystemVerilog Language and Libraries. Waltham, MA, 2004

[19] Goldstein, S., and Budiu, M. Fast compilation for pipelined reconfigurable fabrics. ACM/FPGA Symposium on Field Programmable Gate Arrays (1999).

[20] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: constructing hardware in a Scala embedded language. In Proceedings of the 49th Annual Design Automation Conference (DAC ’12). Association for Computing Machinery, New York, NY, USA, 1216–1225. https://doi.org/10.1145/2228360.2228584

[21] Odersky, M. e. a. Scala programming language. http://www.scala-lang.org/.

[22] RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213 (December 13, 2019), https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf

[23] RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203 (December 4, 2021), https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf

[24] RISC-V Debug Support, version 1.0-STABLE, ece9f185e7e348aafca75418d0f1540b2dfd0ff6, November 7 2023, https://github.com/riscv/riscv-debug-spec/blob/24d2ea065737fc184670a6d09fb314e1a66109f5/riscv-debug-stable.pdf